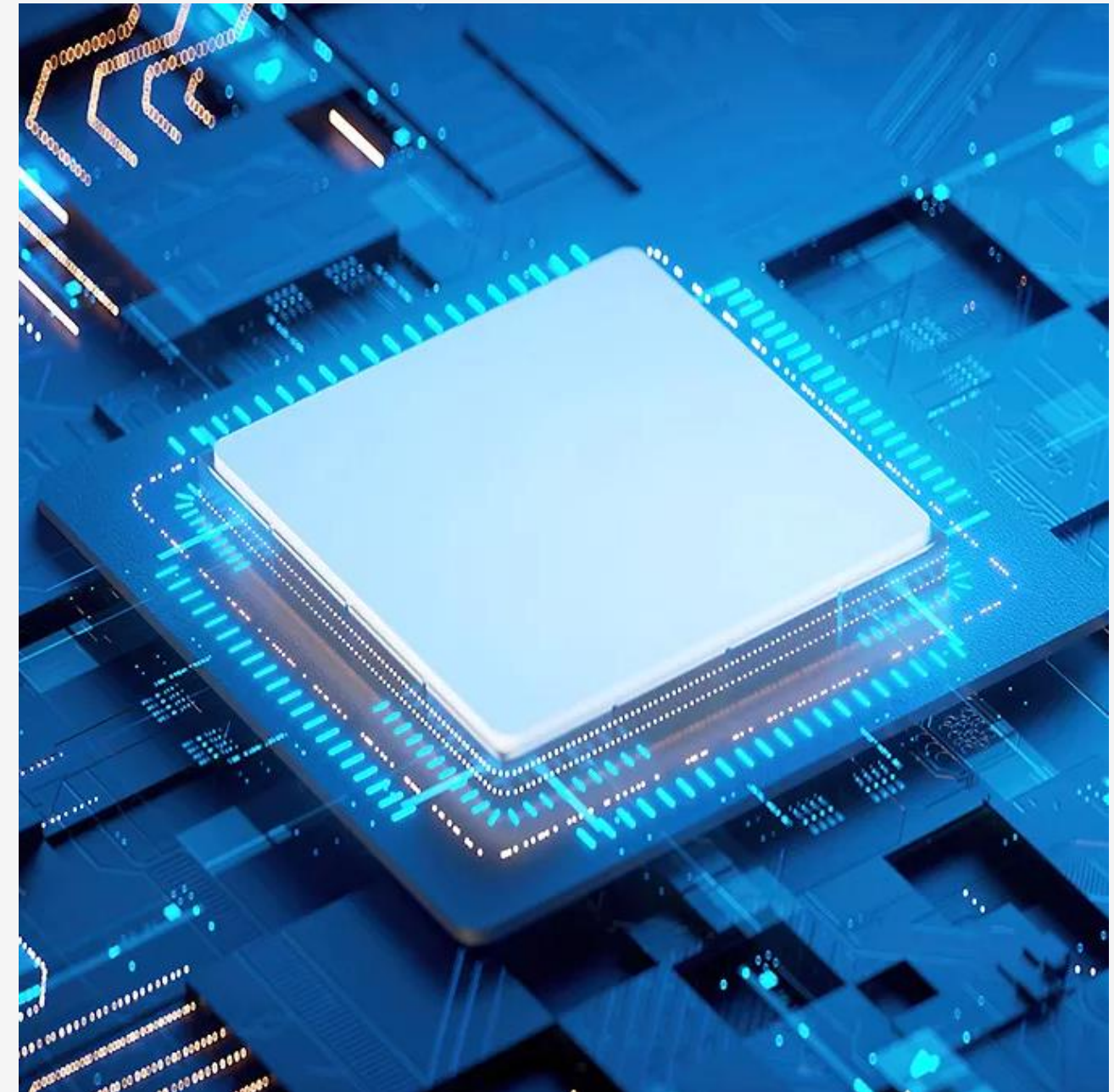


FPGA

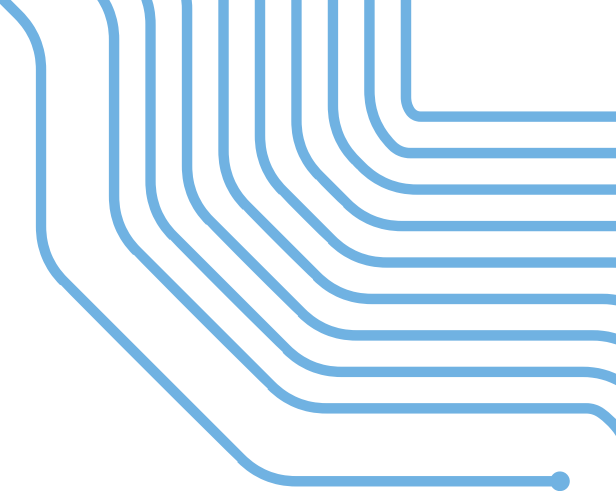
Eng. Ahmed Hussein



Agenda

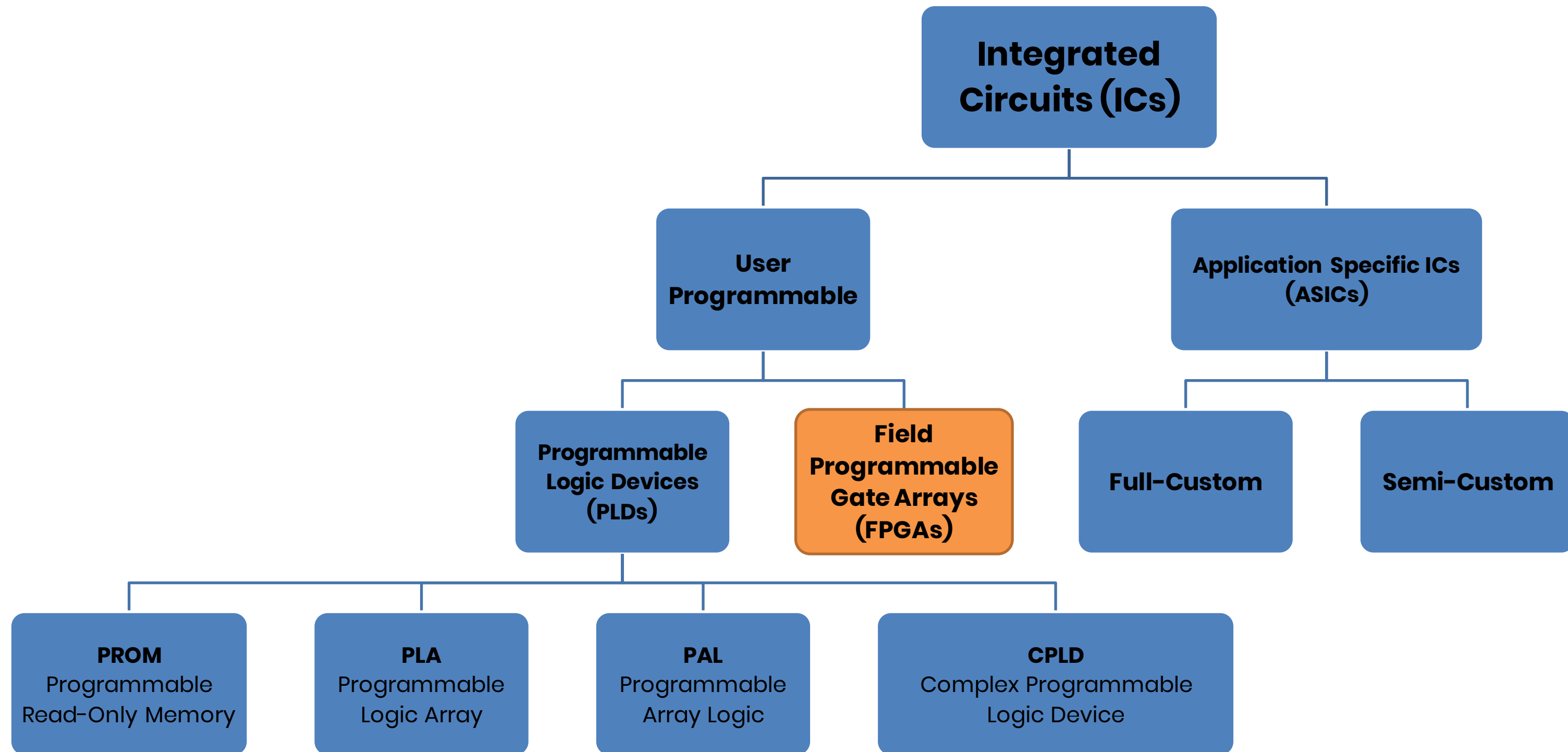
1. FPGA....WHY?
2. FPGA Design Flow
 - a. Design Entry
 - b. Analysis & Synthesis
 - c. Implementation
 - d. Timing Closure
 - e. Device programming
3. IP CORES
4. DE10-Standard FPGA
5. Hello world labs
 1. Play with bottoms and LEDs
 2. 7-Segment decoder in Verilog HDL
 3. 10-bit binary switch values on three 7-segment displays
 4. Displaying 10-bit binary as BCD digits on the 7-segment displays
 5. Vga driver in verilog





Take one step back

VLSI Design Styles (Physical Implementation Styles)



User Programmable Devices

- **Programmable Logic Devices (PLDs)**
 - Programmable Read-Only Memory (PROM)
 - Programmable Logic Arrays (PLAs)
 - Programmable Array Logic (PALS)
 - Complex Programmable Logic Devices (CPLDs)
- **Field Programmable Gate Arrays (FPGAs)**

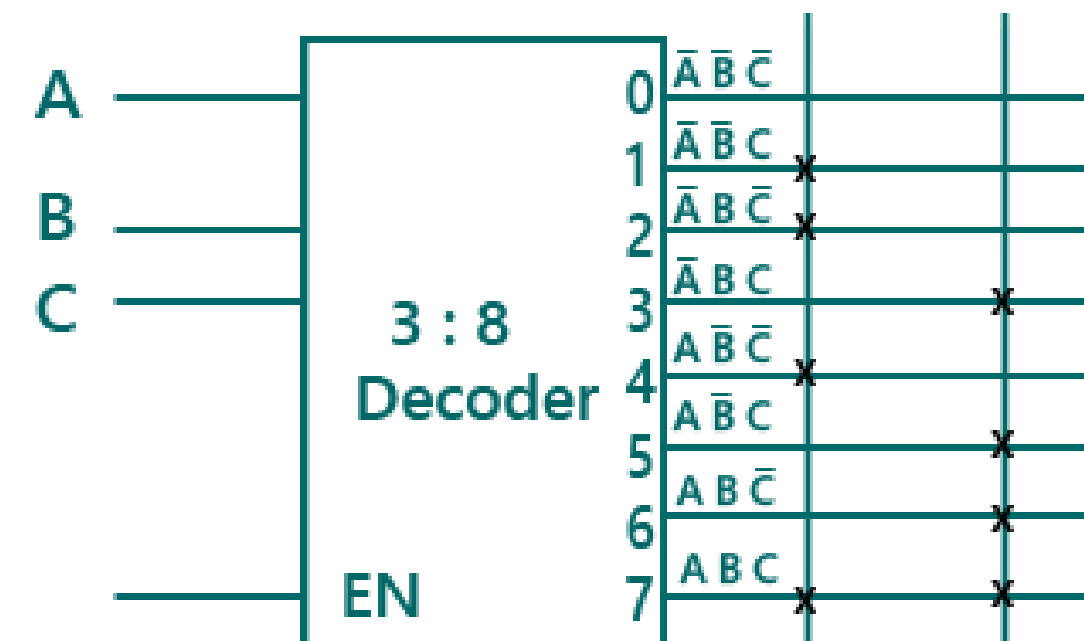
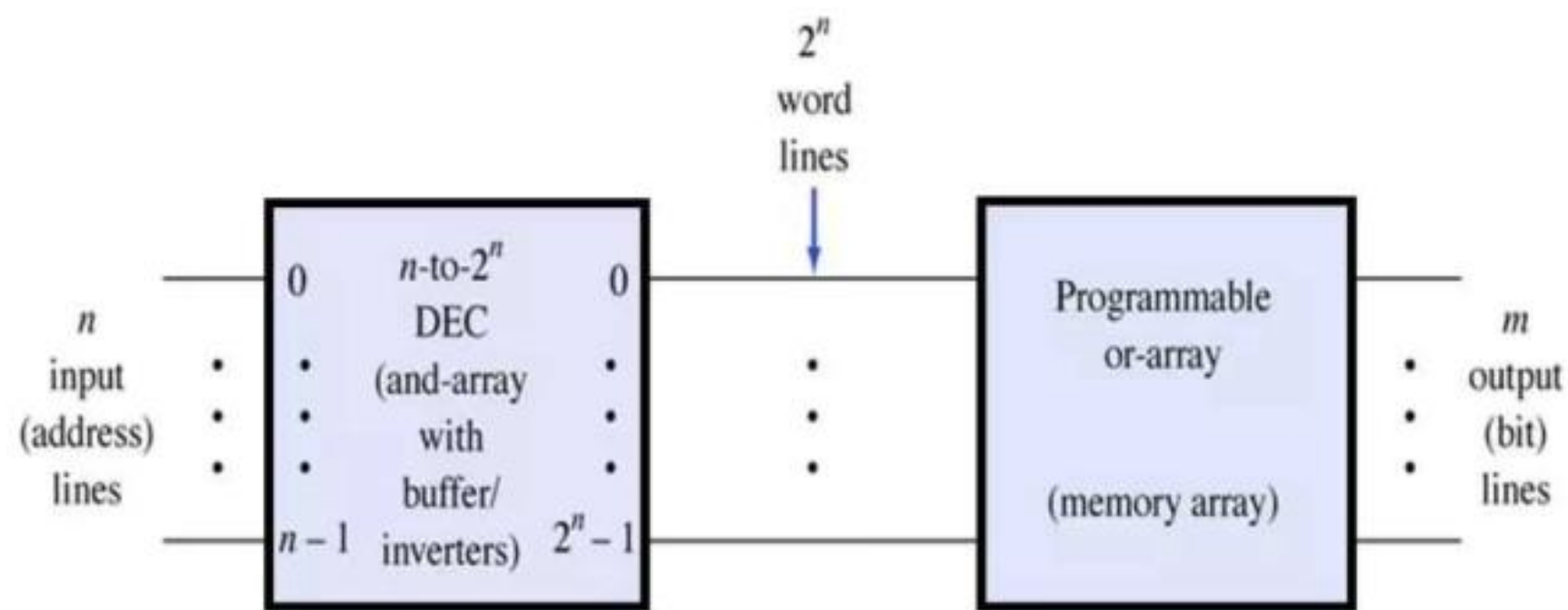
**Simple Programmable
Logic Devices (SPLDs)**

Programmable Read-Only Memory (PROM)

- **Structure:** Fixed AND Array → **Programmable** OR Array.
- **How it works:** The "Fixed AND" acts just like a memory address decoder (activating one row at a time). The "Programmable OR" stores the data bits.
- **Pros:** Very **easy** to manufacture; **low cost** for storing data.
- **Cons:** Not efficient for logic functions because you cannot choose which input combinations trigger the output (you must decode all of them).



Fig:- Basic Configuration of PROM



$$\begin{aligned} \text{Sum} &= \sum m(1, 2, 4, 7) \\ &= \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}\bar{C} + ABC \end{aligned}$$

$$\begin{aligned} \text{Carry} &= \sum m(3, 5, 6, 7) \\ &= \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC \end{aligned}$$

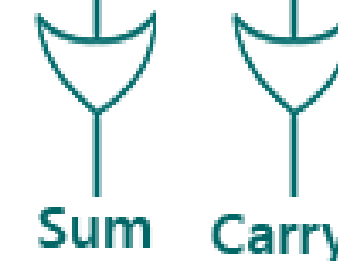
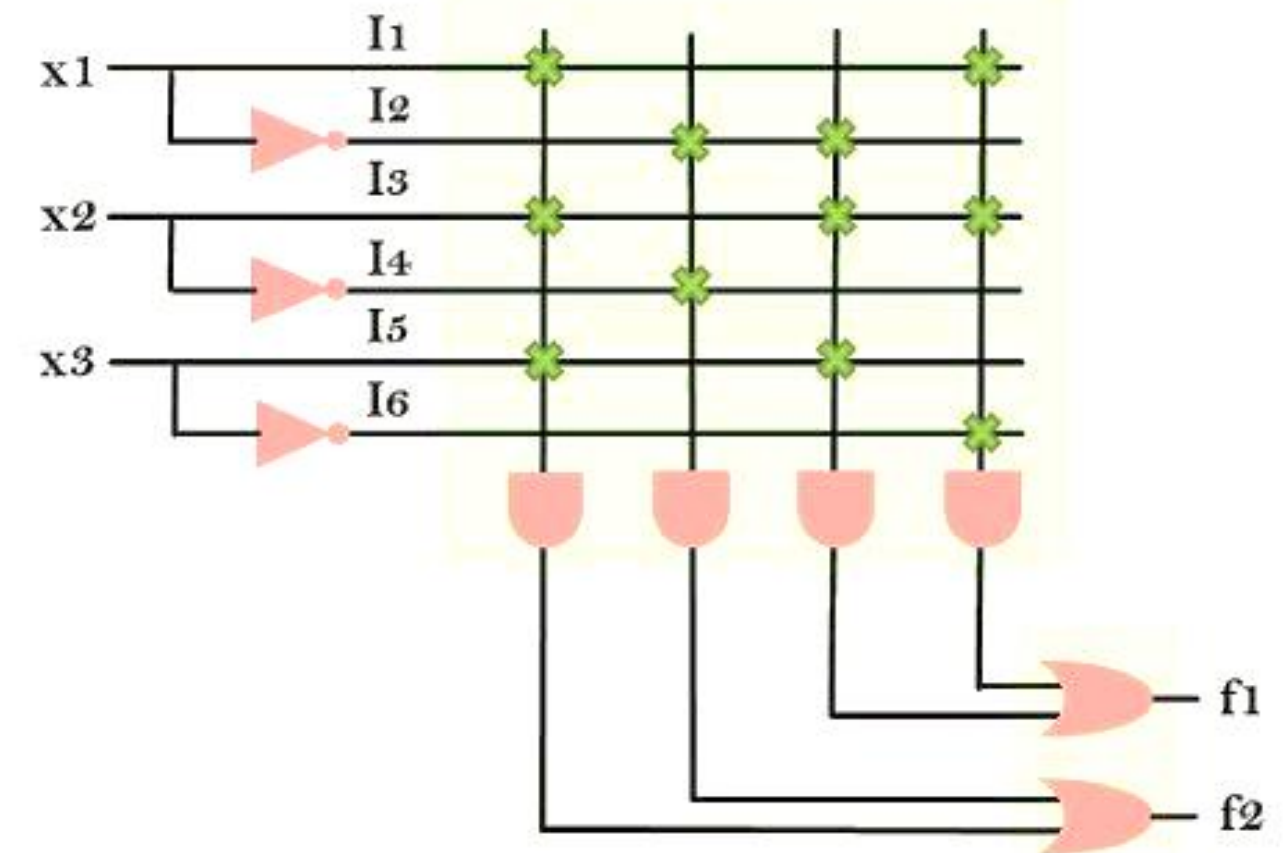
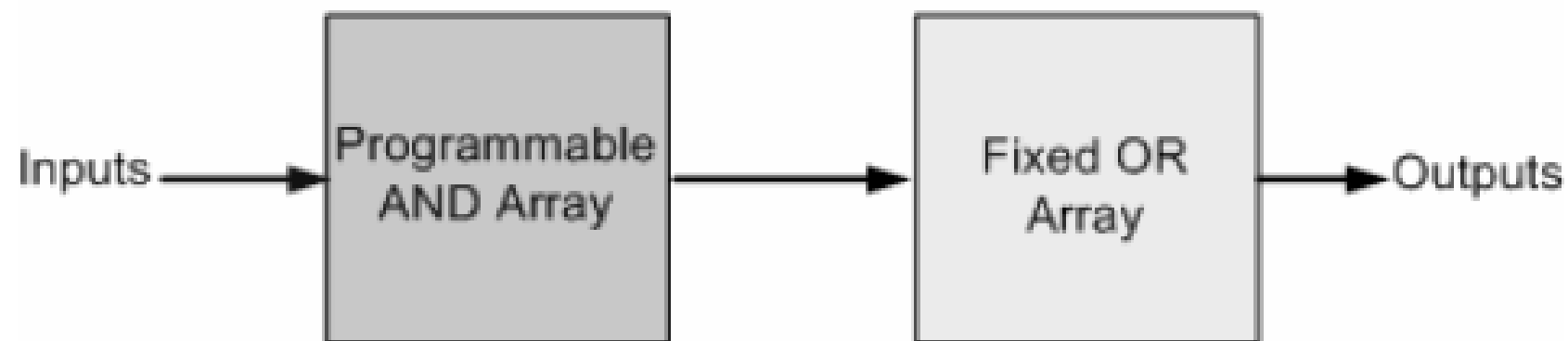


Fig . Full Adder using PROM

Programmable Array Logic (PALs)

- **Structure: Programmable** AND Array → **Fixed** OR Array.
- **How it works:** This is the opposite of PROM. You can choose exactly which complex inputs you want to detect (Programmable AND), but the way they are combined is hard-wired (Fixed OR).
- **Pros: Faster** than PLA (signals only pass through one programmable layer). **Cheaper** to build than PLA.
- **Cons: Less flexible.** If a function needs 4 OR terms but the chip is wired for only 3, you can't implement it.



$$f_1 = x_1x_2x'_3 + x'_1x_2x_3$$

$$f_2 = x'_1x'_2 + x_1x_2x_3$$

Programmable Logic Arrays (PLAs)

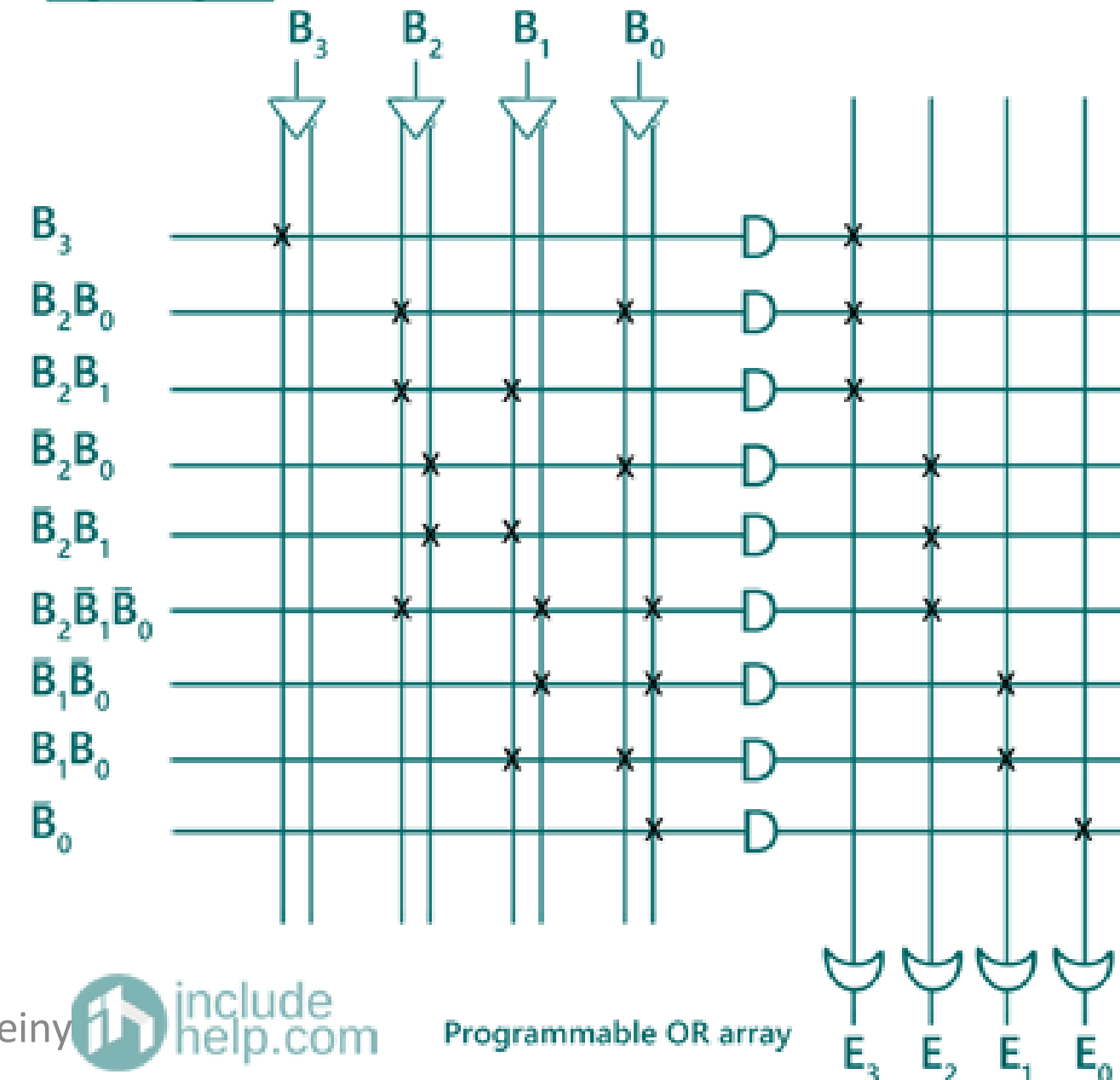
- **Structure: Programmable** AND Array $\rightarrow$ **Programmable** OR Array.
- **Pros: Most flexible.** Can implement complex logic efficiently by sharing product terms between different outputs.
- **Cons: Slower** (signals go through two programmable fuses) and historically more expensive to manufacture

Logic diagram:



Fig:-

Basic configuration of Programmable logic array.

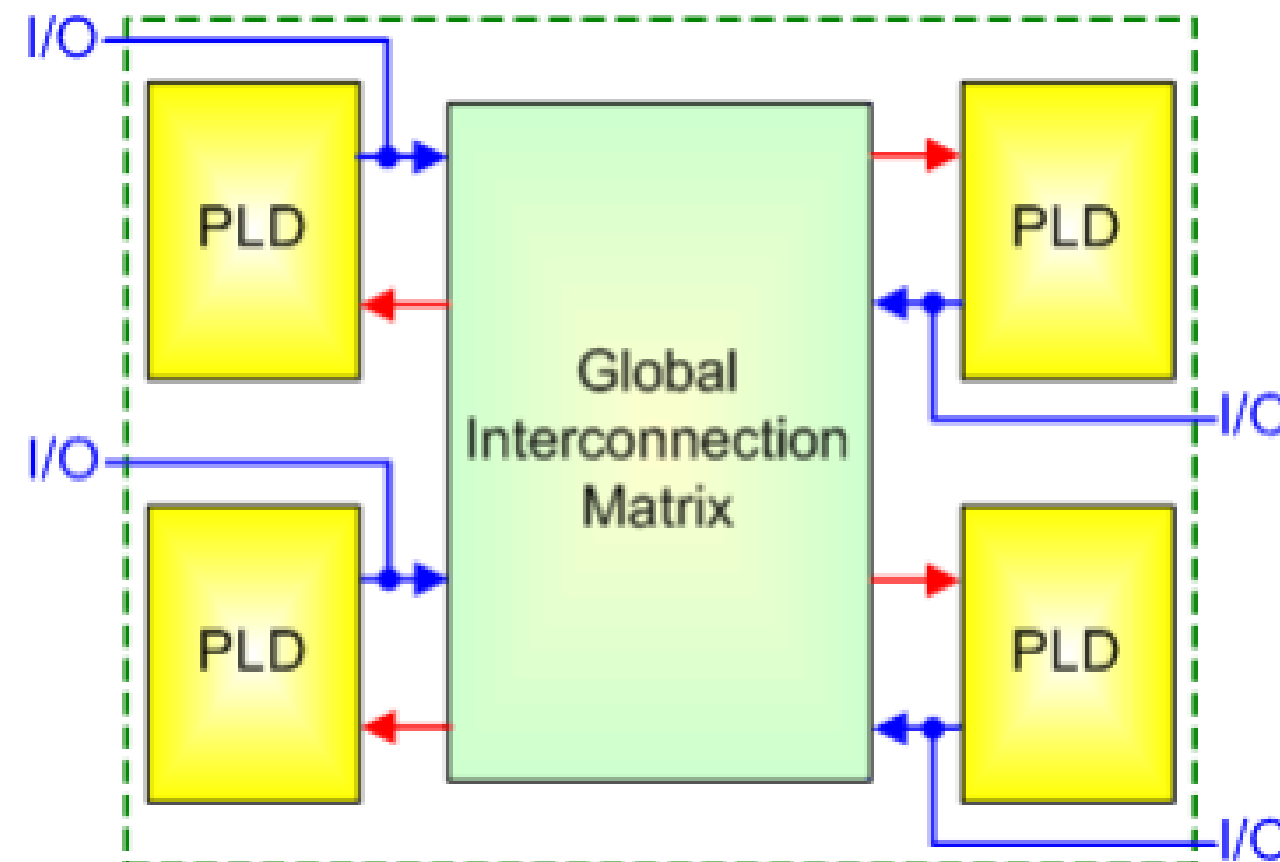


Complex Programmable Logic Devices (CPLDs)

- CPLDs are another way to **extend the density** of the SPLD.
- CPLDs use **less board space**, **improve reliability**, and **reduce cost**.
- **Multiple PLDs + Prog. interconnection** in a single chip
- **Two levels of programmability**: each **PLD** block can be programmed, and then **the interconnections** between the PLDs can be programmed.



MAX 5000 Erasable Programmable Logic Devices (EPLDs)
by Altera in 1988



Field Programmable Gate Arrays (FPGAs)

PLD

- SPLD
- CPLD



ASIC

- FULL-CUSTOM
- SEMI-CUSTOM

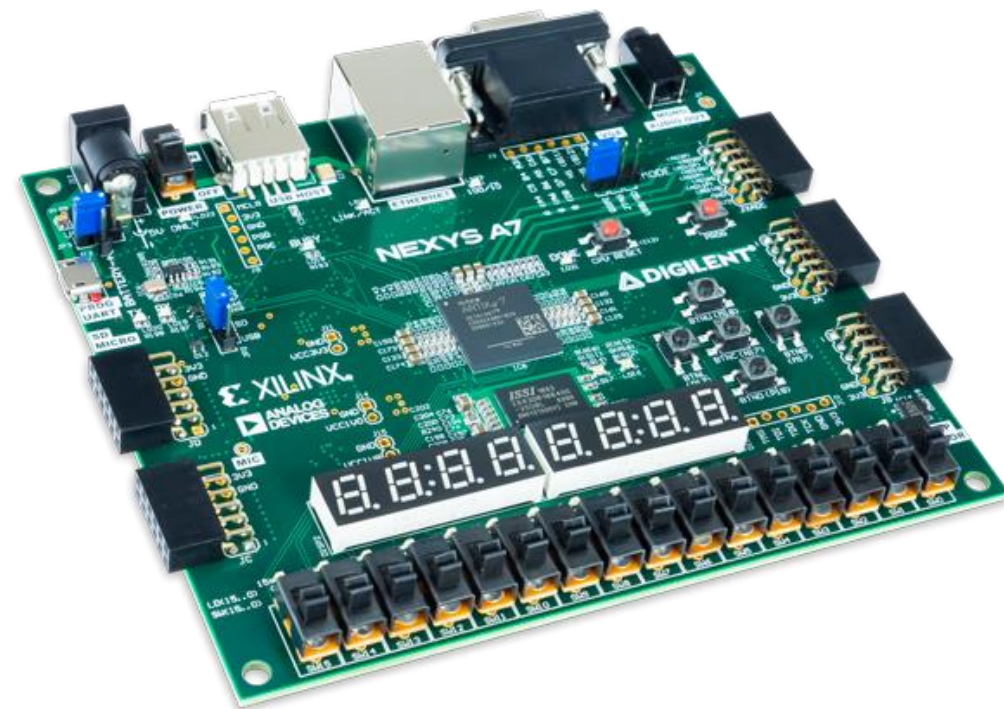


in 1985, **Xilinx** identified this gap and introduced new type of IC, Called **XC2064™ FPGA family**

The first **FPGAs** were based on **CMOS** and used **SRAM** cells for configuration purposes

Field Programmable Gate Arrays (FPGAs)

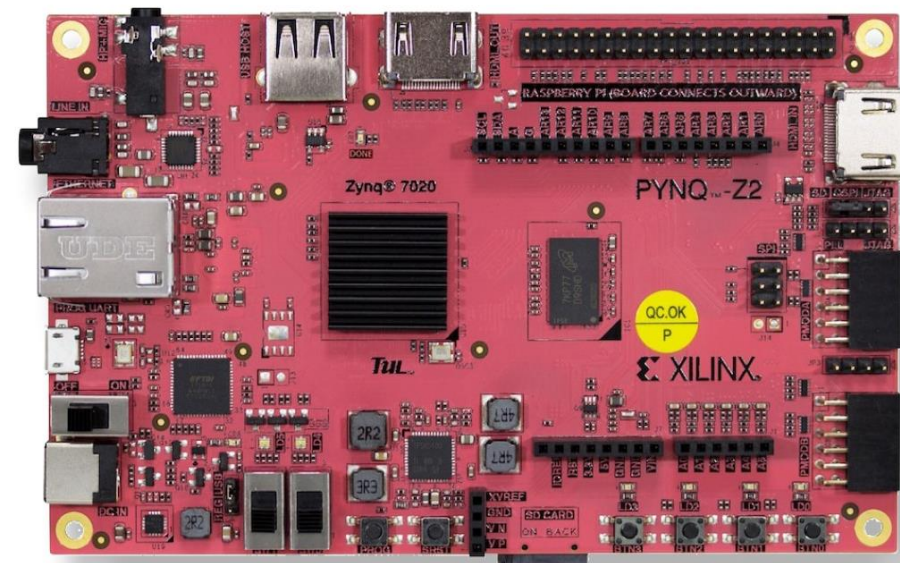
- Field-programmable gate arrays (FPGAs) act as the **boundary** between **hardware** and **software**.
- FPGAs are available as **part** of circuit boards (**FPGA Boards**) which contain multiple optional other **peripherals** such as **switches**, **LEDs**, **7-segment displays**, **ADCs** and **DACs**, **VGA**, etc.



Nexys- A7

AMD Artix™ 7 FPGA

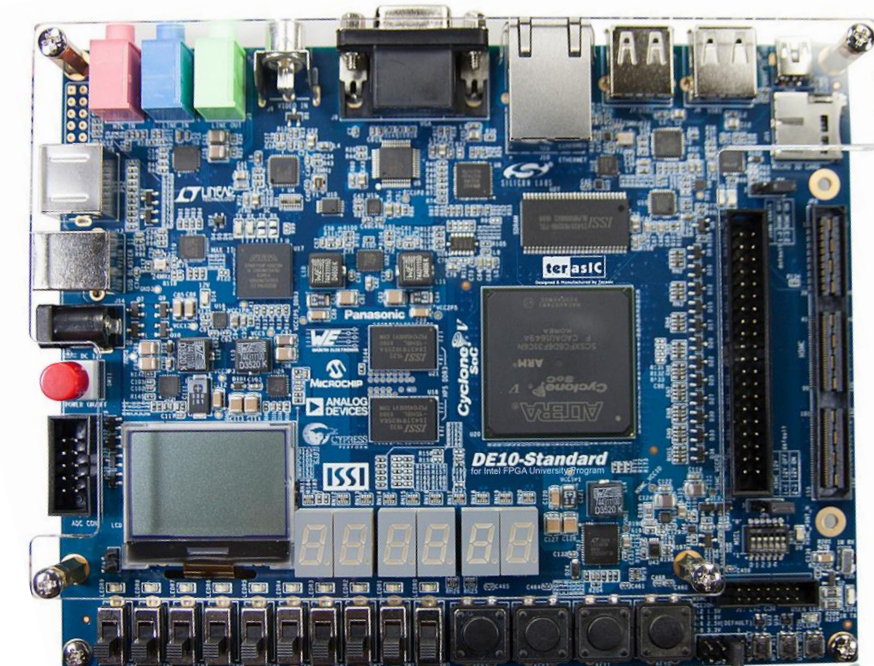
Digilent Inc.



AUP PYNQ-Z2

Zynq-7000 SoC

© Eng. Ahmed Hussein
AMD



DE10-Standard

Intel Cyclone® V SX SoC

Terasic

FPGA Companies

ARTIX⁷ KINTEX^{UltraSCALE} VIRTEX^{UltraSCALE+} ZYNQ^{UltraSCALE+}

VIVADO™

XILINX
VITIS™

XILINX
ALLIANCE PROGRAM
CERTIFIED

intel
FPGA Design
Solutions Network
Platinum

Quartus® Prime
Design Suite

Libero
System-on-Chip

MICROCHIP

LATTICE
SEMICONDUCTOR

LATTICE
DIAMOND
DESIGN SOFTWARE

MICROCHIP
SmartFusion® 2
SoC FPGA

MICROCHIP
IGLOO® 2
FPGA

MICROCHIP
ProASIC®3
FPGA

MICROCHIP
PolarFire®
FPGA

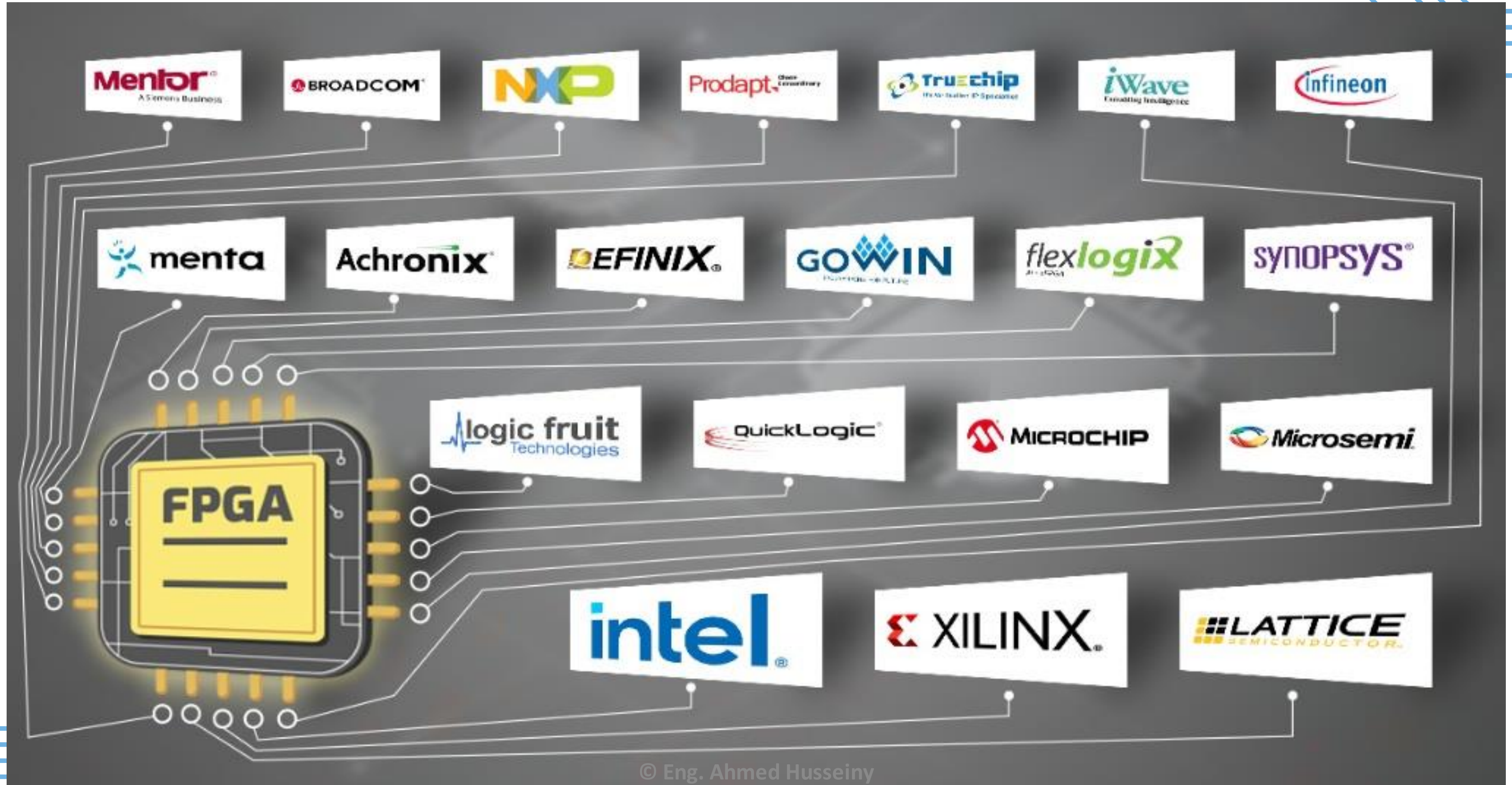
LATTICE
LatticeECP3

LATTICE
LatticeSC/M

LATTICE
iCE40
UltraPlus

LATTICE
ECP5

FPGA Companies



Why FPGA?

- **Prototyping**

- Before a company spends millions of dollars manufacturing a custom chip (ASIC), they need to make sure the design actually works.

- **Emulation**

- mapping the much larger systems onto a massive bank of FPGAs. the system can run at speeds much closer to real-time. This allows software developers to write and test drivers/firmware before the actual hardware chip is even manufactured.

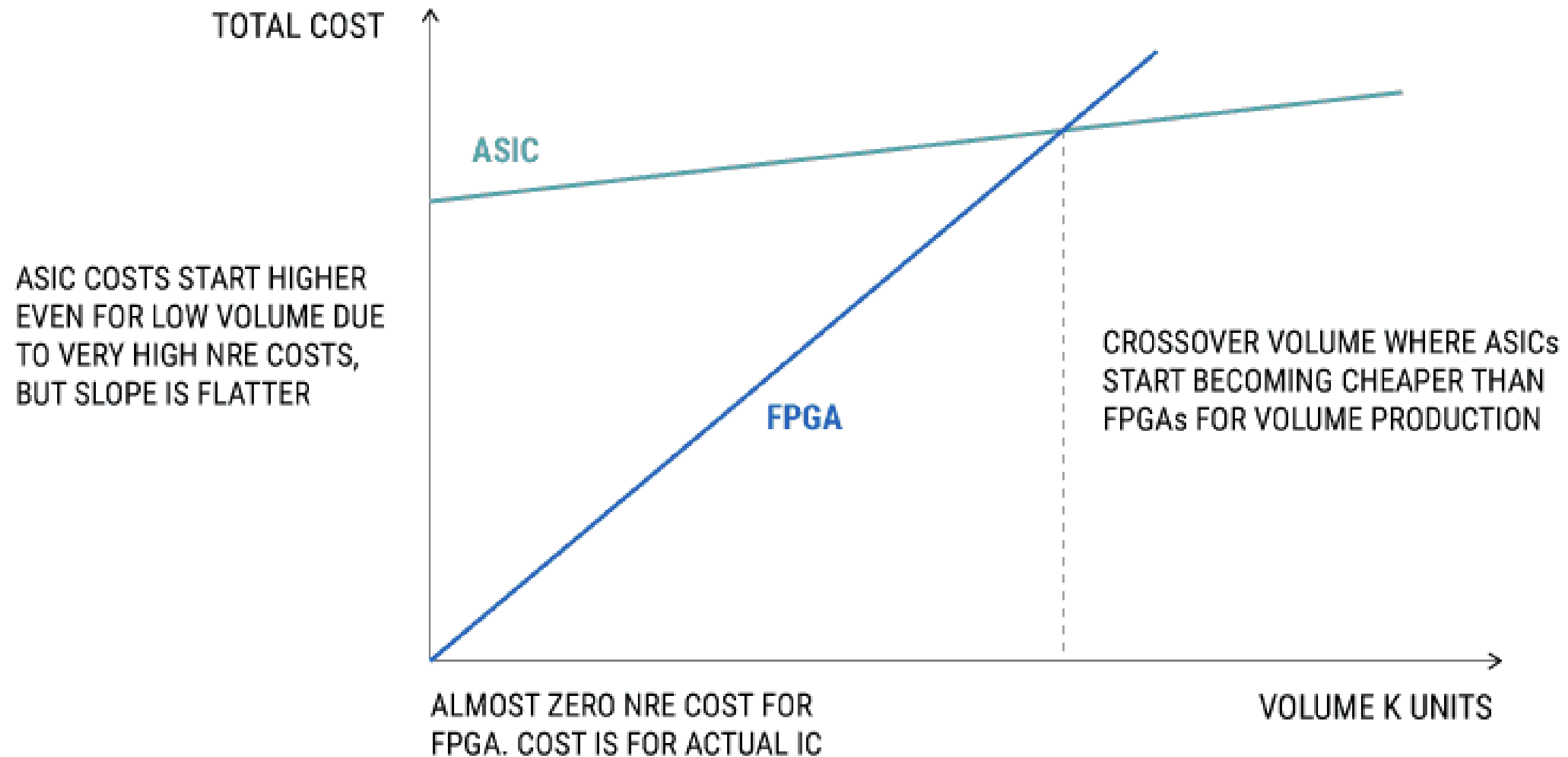
- **Niche Products (Low Volume)**

- For low-volume products, it is cheaper to buy off-the-shelf FPGAs (even if they cost \$100 each) than to pay \$5 million upfront to design a custom chip that costs \$5 to make.

- **Reconfigurability**

- you can send a software update that physically rewires the internal connections of the chip.

FPGA VS ASIC



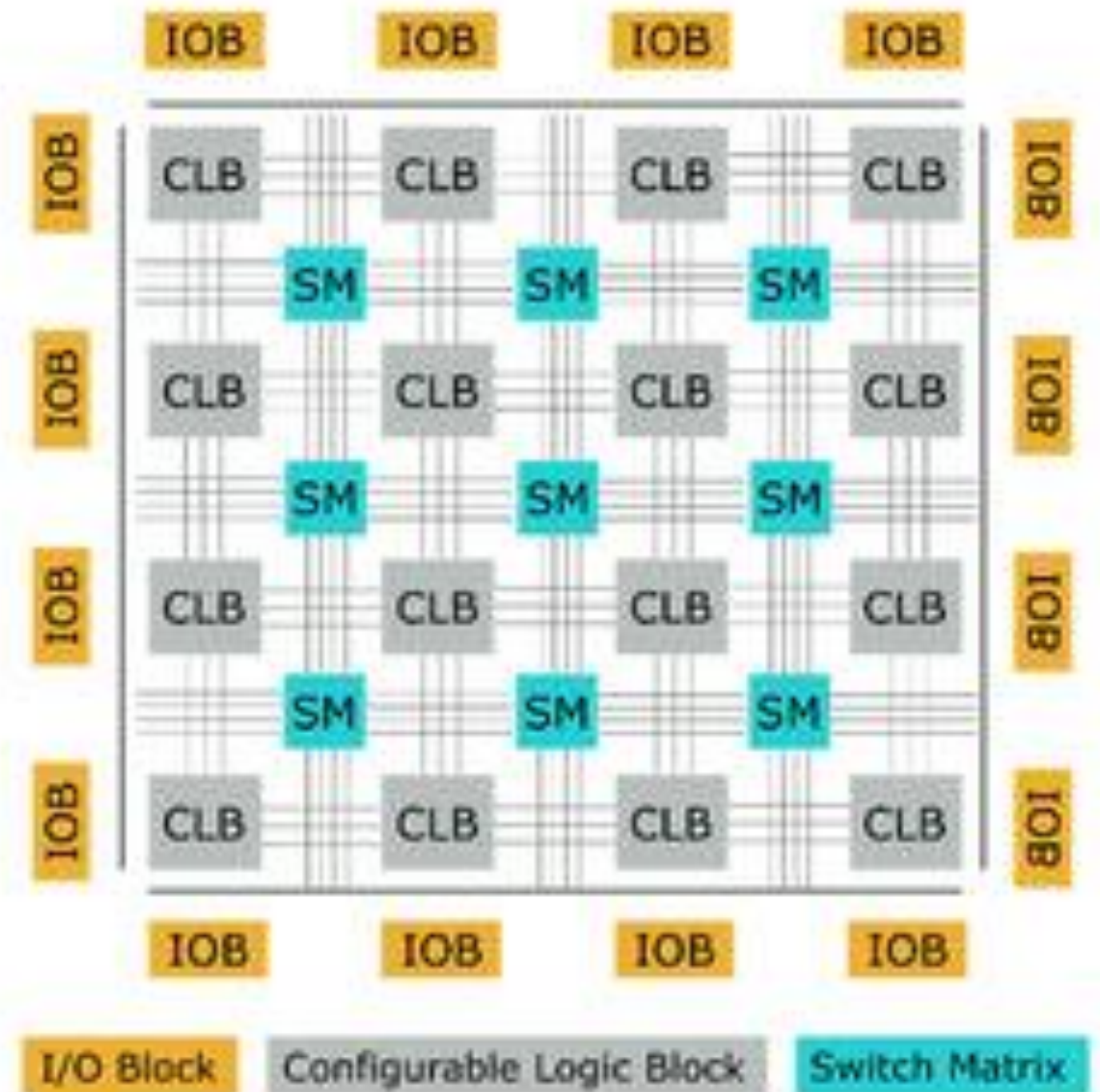
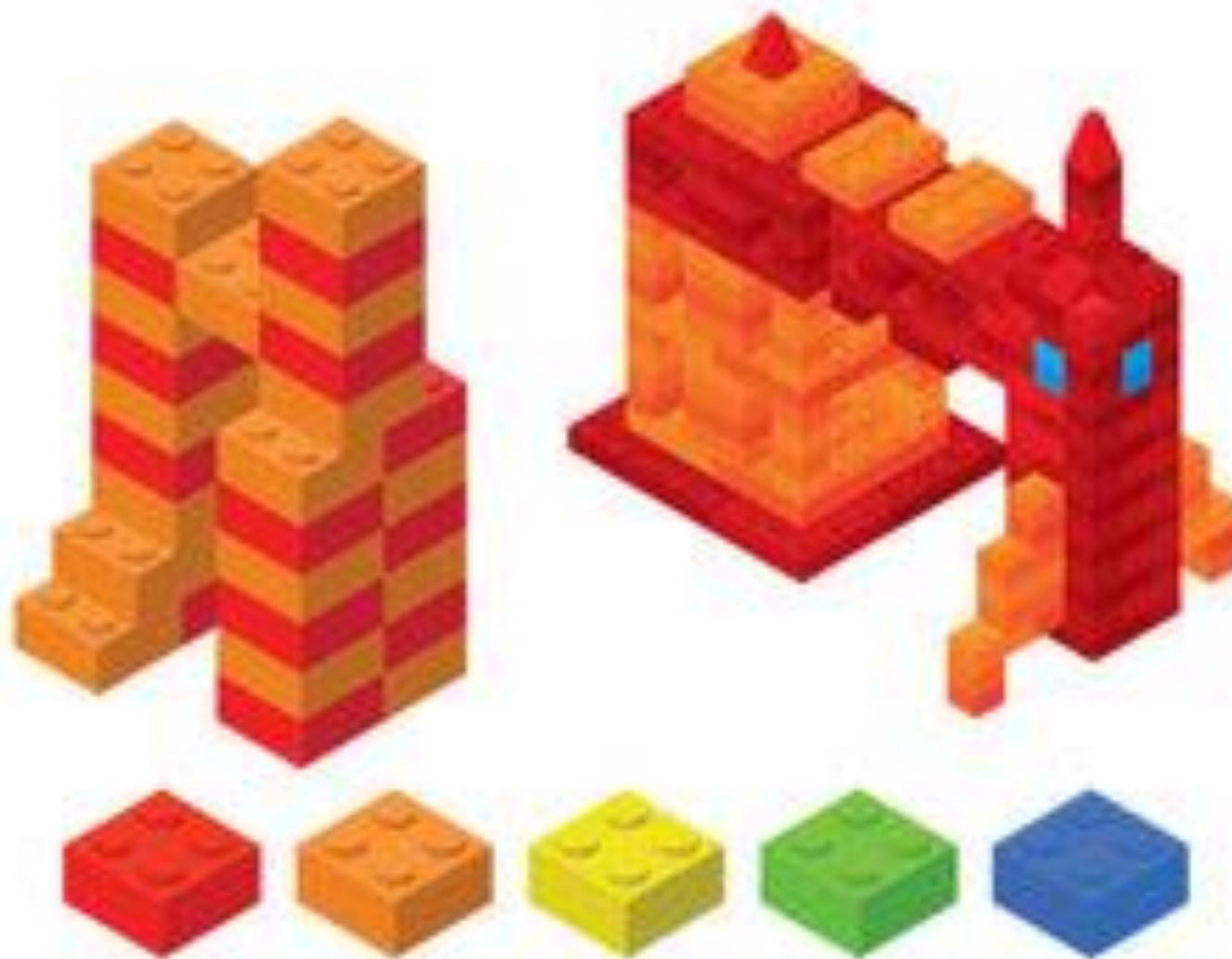
FPGA vs ASIC COST ANALYSIS

FPGA VS ASIC

	FPGA	ASIC
Analogy	FPGA is like LEGO	ASIC is like a DIE-CAST TOY
NRE Cost	Low. You just buy the chip off the shelf. No factory setup fee.	Very High. Millions of dollars for masks and setup before making a single chip.
Unit Cost	High. A single FPGA can cost \$10 to \$10,000 depending on size.	Low. In mass production , a chip might cost pennies (e.g., \$0.50).
Time to Market	Fast. You can code and deploy in weeks or months.	Slow. Design to physical chip takes 12–24 months.
Performance (Speed)	Slower. Routing through programmable switches adds delay.	Fastest. Optimized physical connections allow maximum GHz.
Power Consumption	High. Unused logic blocks and routing switches leak power.	Low. You only build exactly what you need, wasting no energy.
Flexibility	High. Can be reprogrammed /updated in the field (like a firmware update).	None. Once manufactured, the logic is frozen forever.
Typical Usage	Complex products in small volume (ex. Medical and Defense) Digital HW Prototyping	Products in large volume

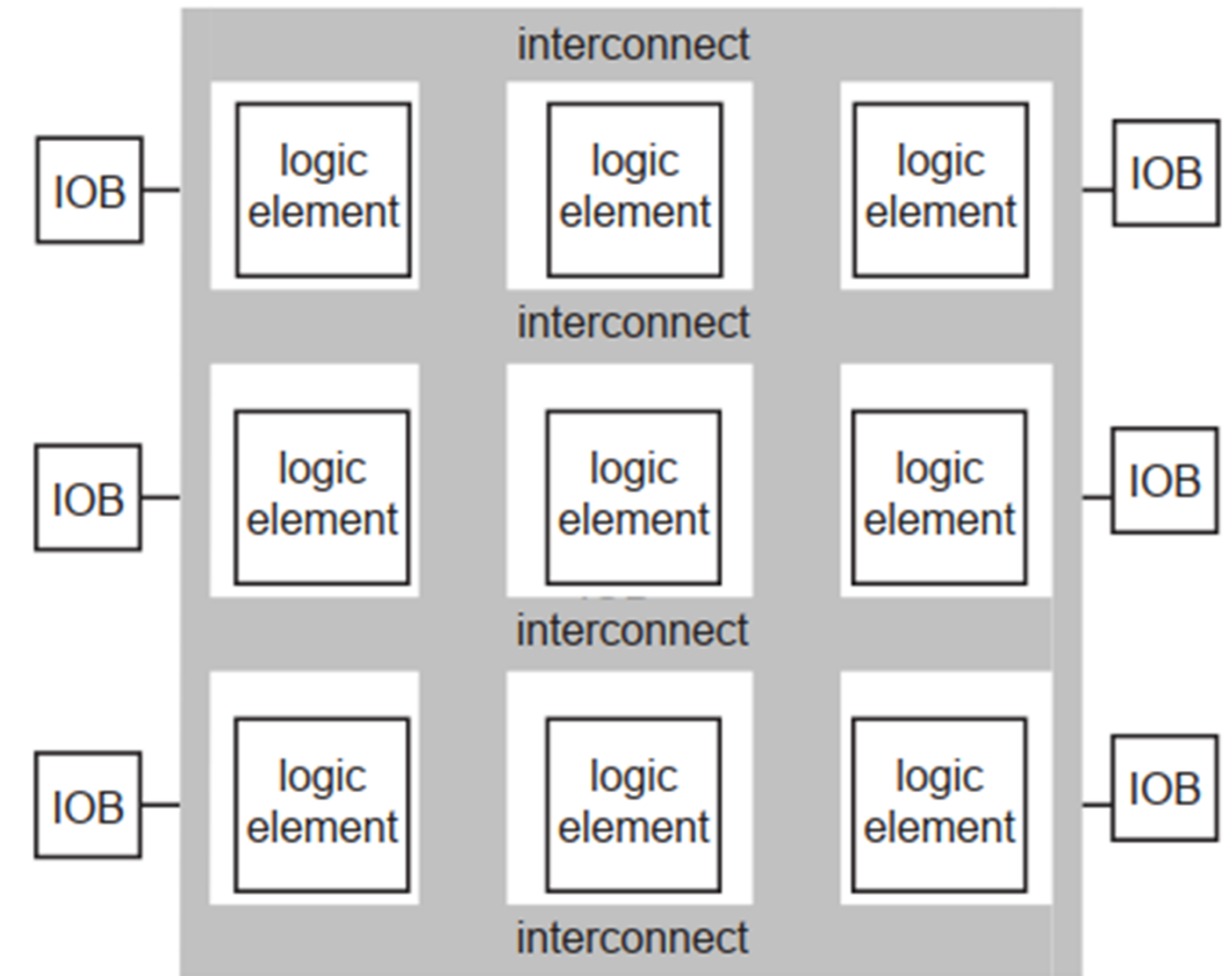
Why FPGA?

Think of **FPGAs** as **digital LEGOs**

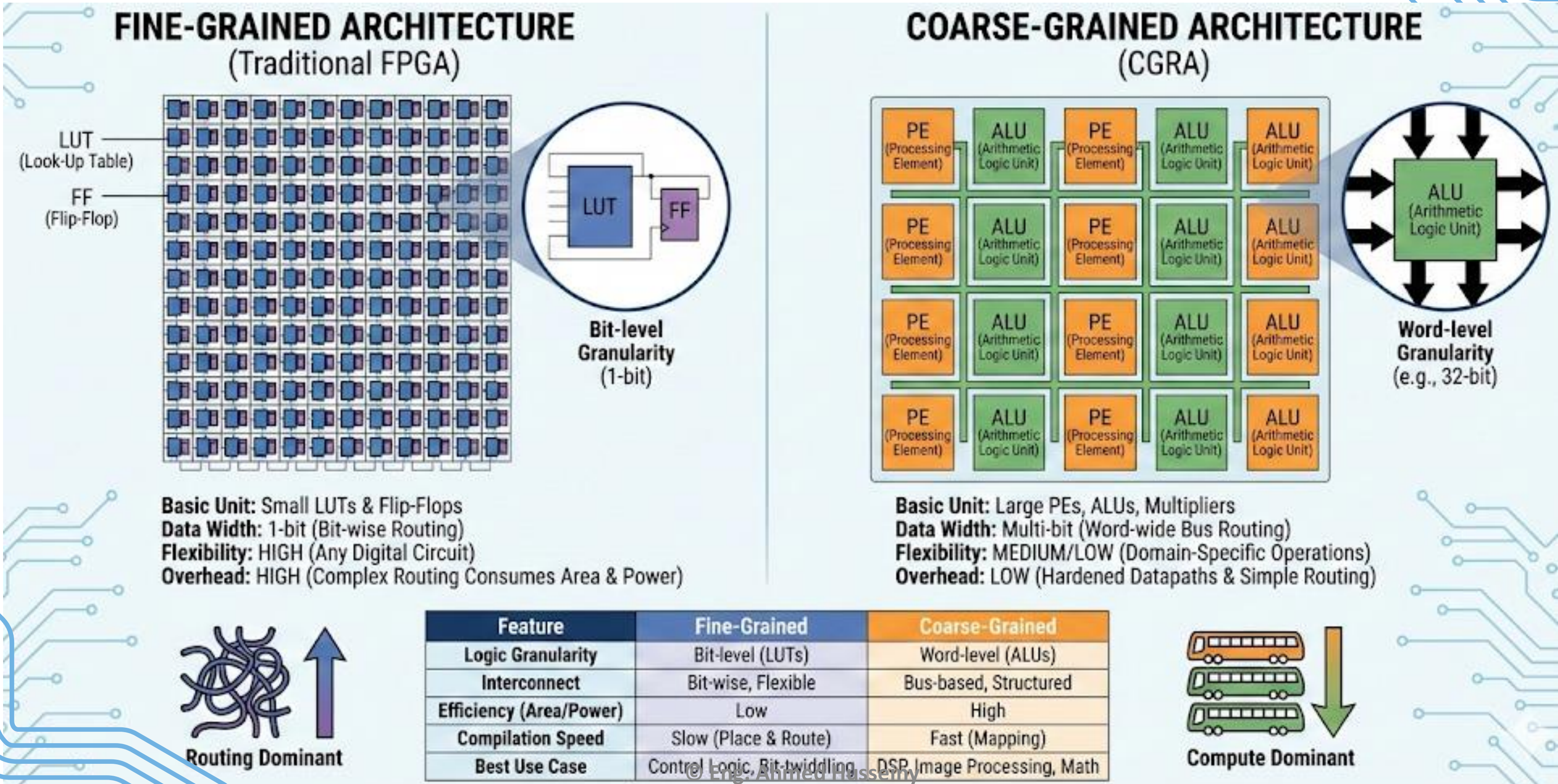


FPGAs Fabric

- **Field Programmable Gate Array (FPGA)** consist of,
 - **Basic Section of FPGA :**
 - Configurable Logic Blocks (CLBs) / Logic Elements (LEs)
 - Programmable Interconnect (Routing Fabric)
 - I/O Blocks (IOBs)
 - **More Advanced FPGA Contains:**
 - Block RAM (BRAM)
 - DSP Blocks (Digital Signal Processing)
 - Clock management tiles (CMTs) (PLL/DLL)
 - Hard Processor System (HPS)

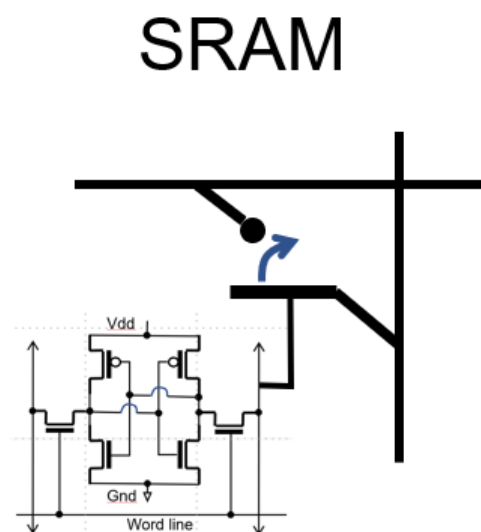


FINE-GRAINED VS COARSE-GRAINED ARCHITECTURES

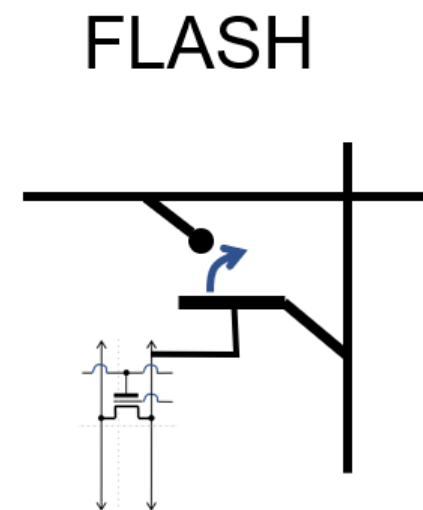


FPGA configuration

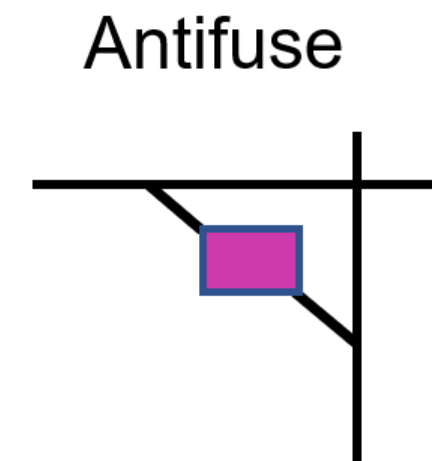
- All FPGAs need to be **programmed** or **configured**.
- There are **three** major circuit **technologies** for **configuring** an FPGA:
 - **SRAM based.**
 - **One Time Programmable FPGAs (Anti Fusing).**
 - **PROM/EPROM/EEPROM/FLASH based.**



Reprogrammable
and volatile

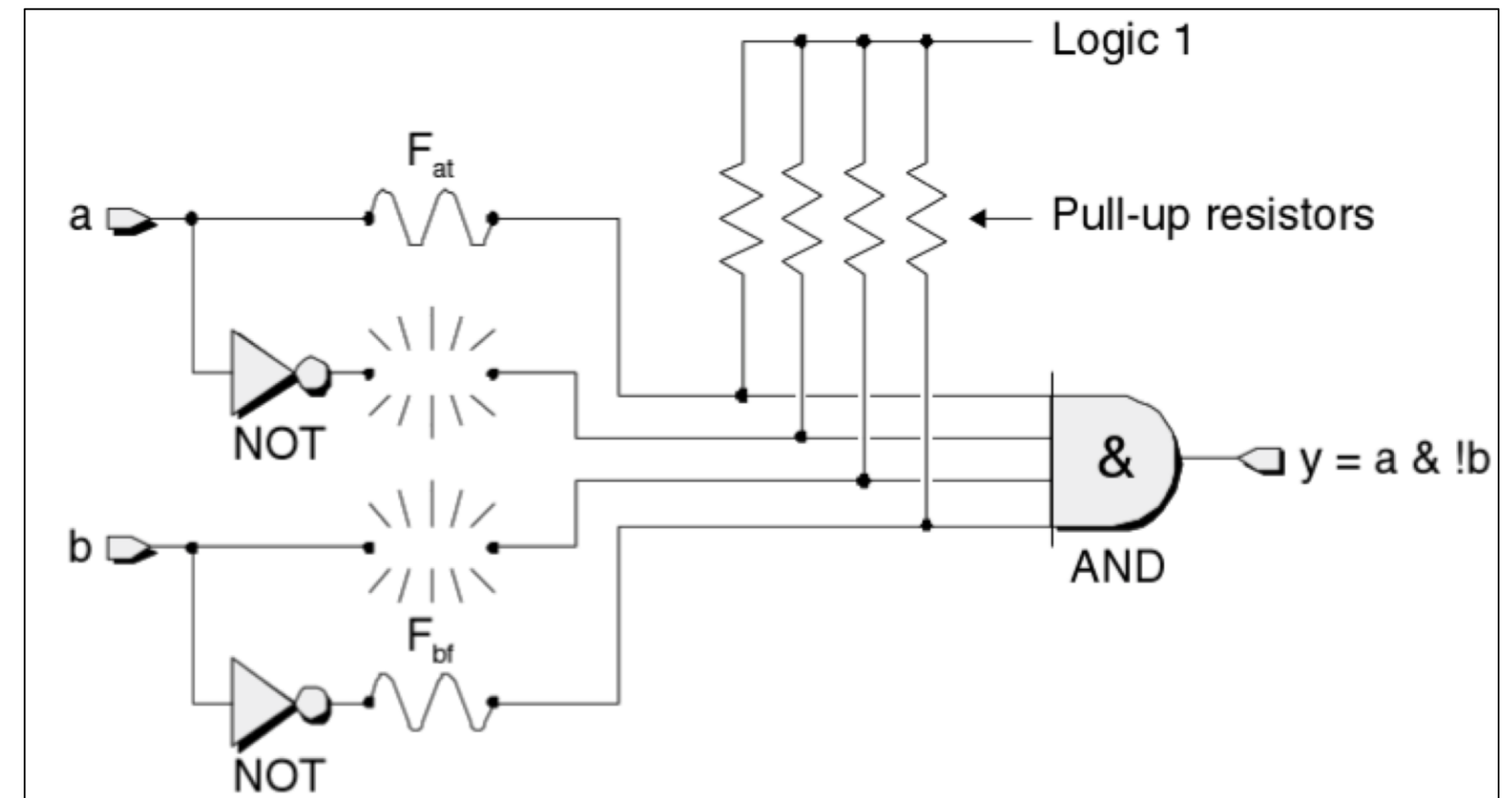
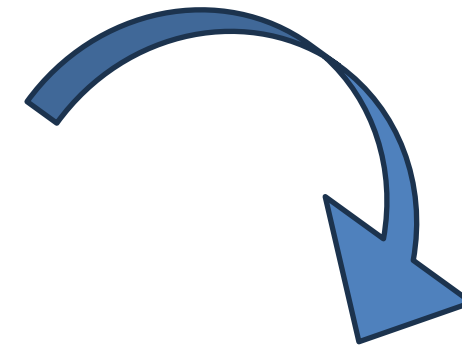
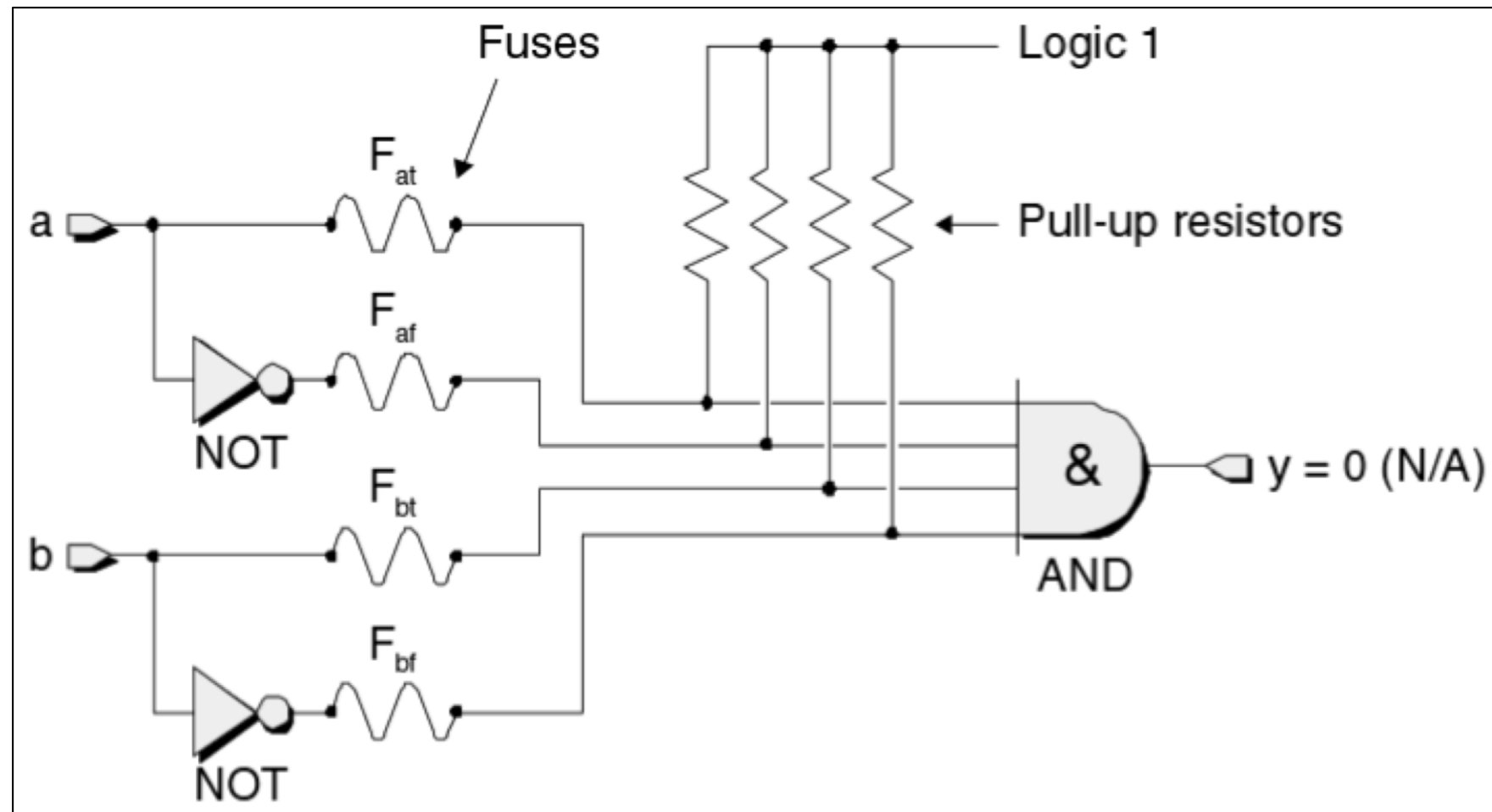


Reprogrammable
and Non-volatile
© Eng. Ahmed Hussein

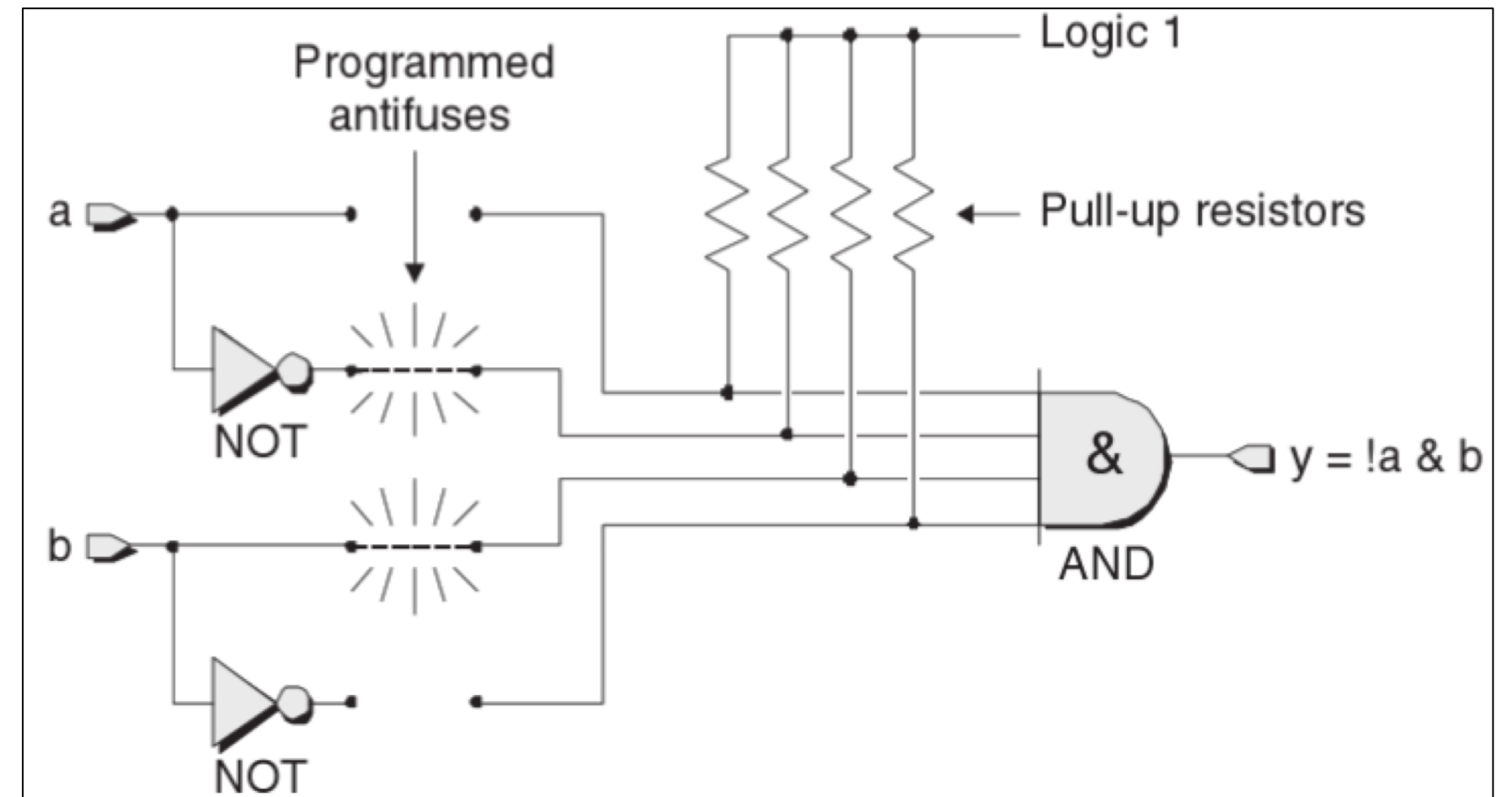
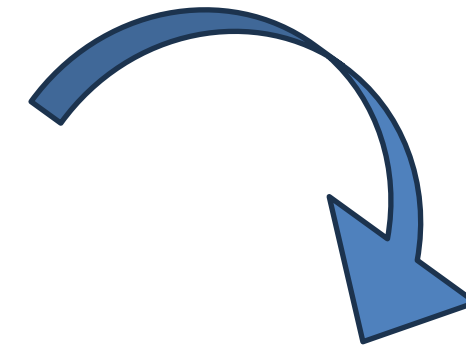
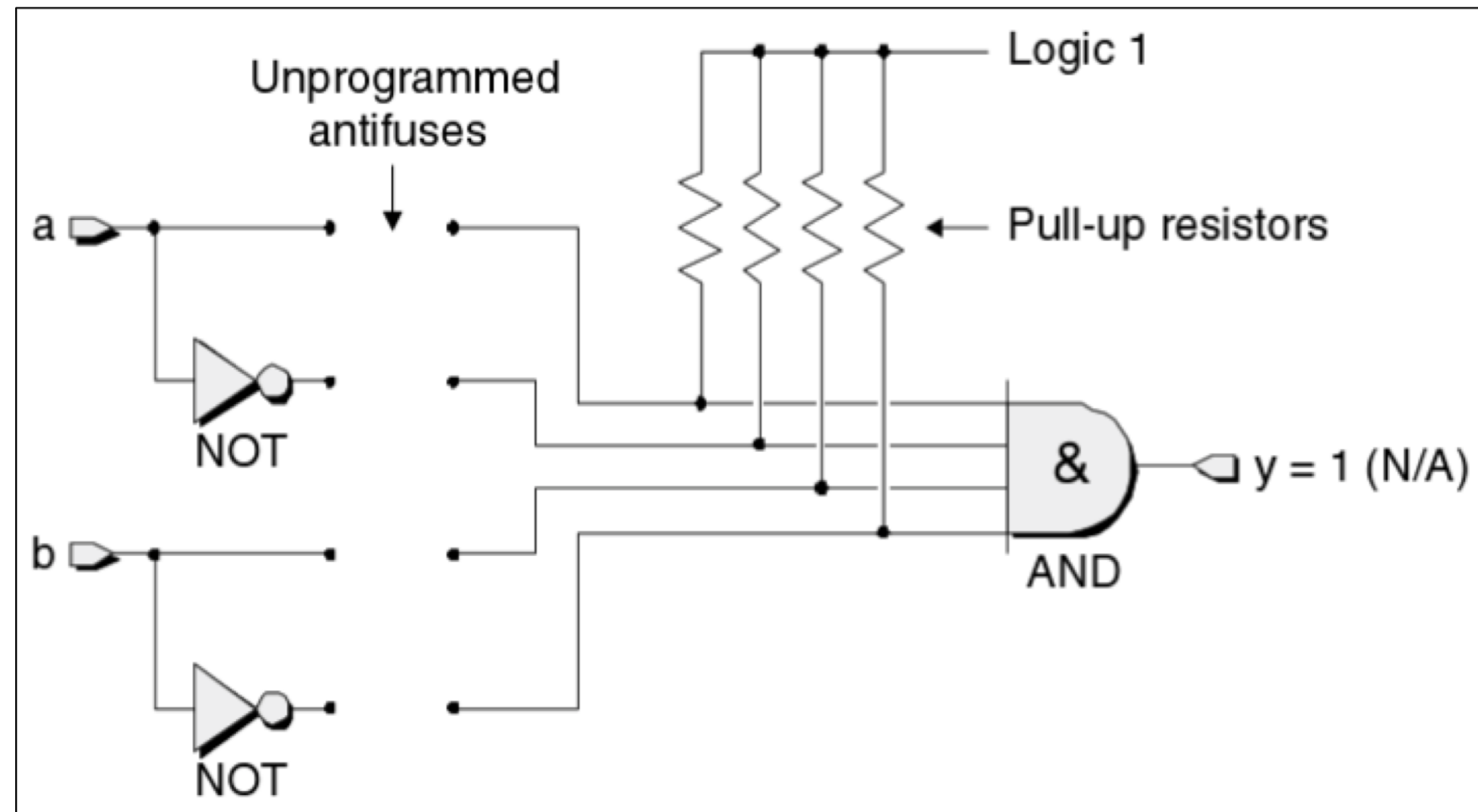


One-time programmable
and Non-volatile

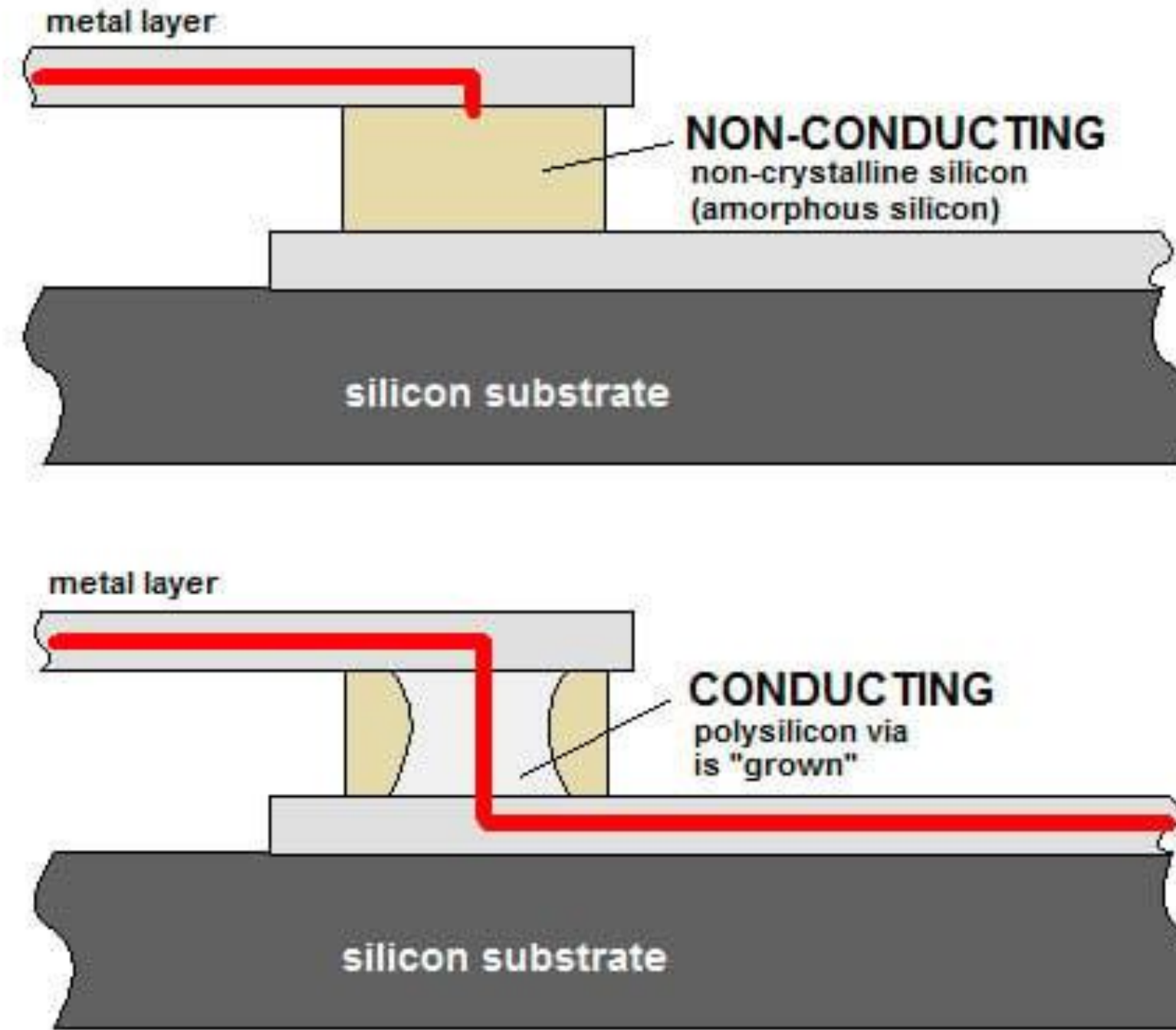
FPGA configuration : Fuses



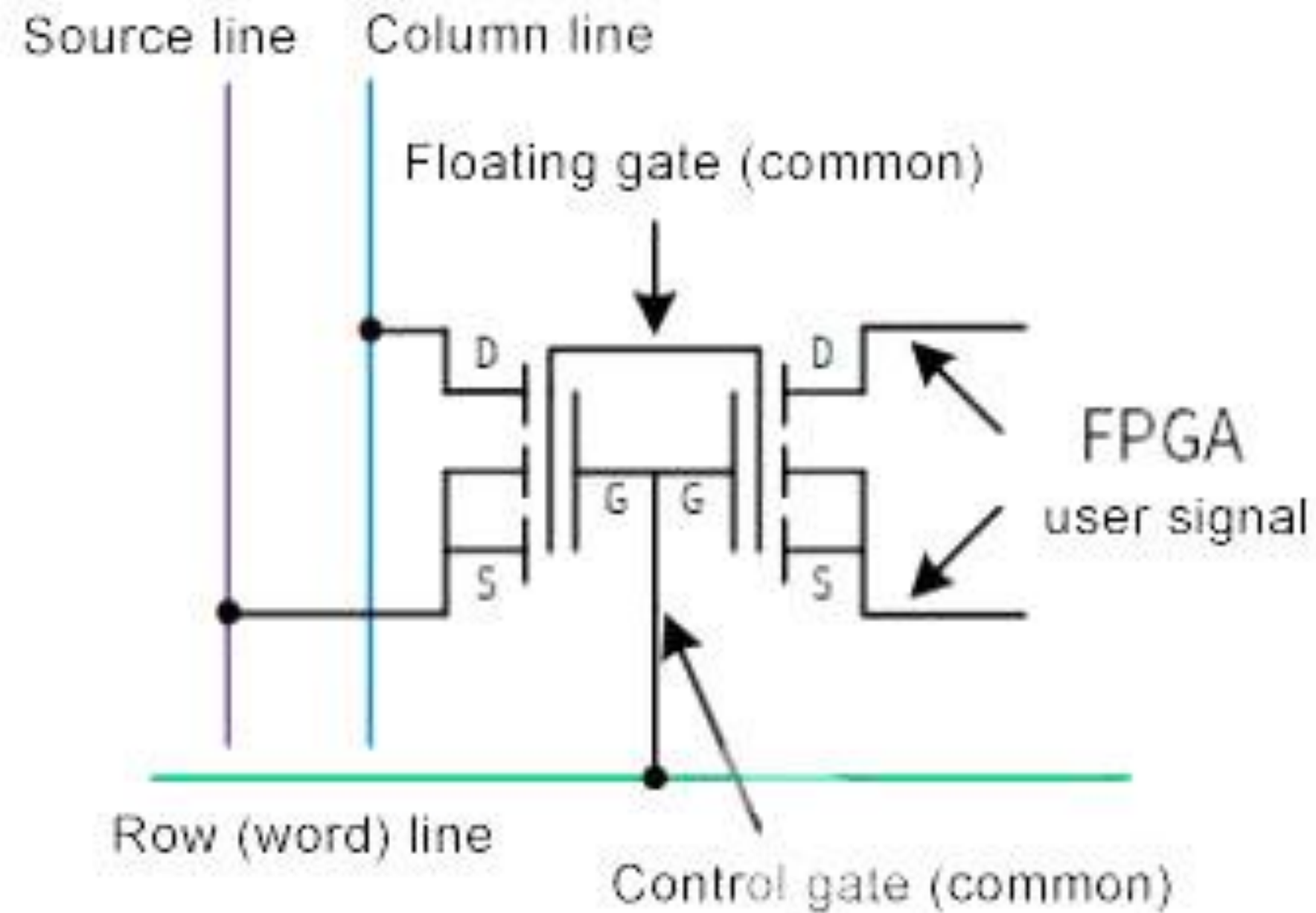
FPGA configuration : Anti-fuses



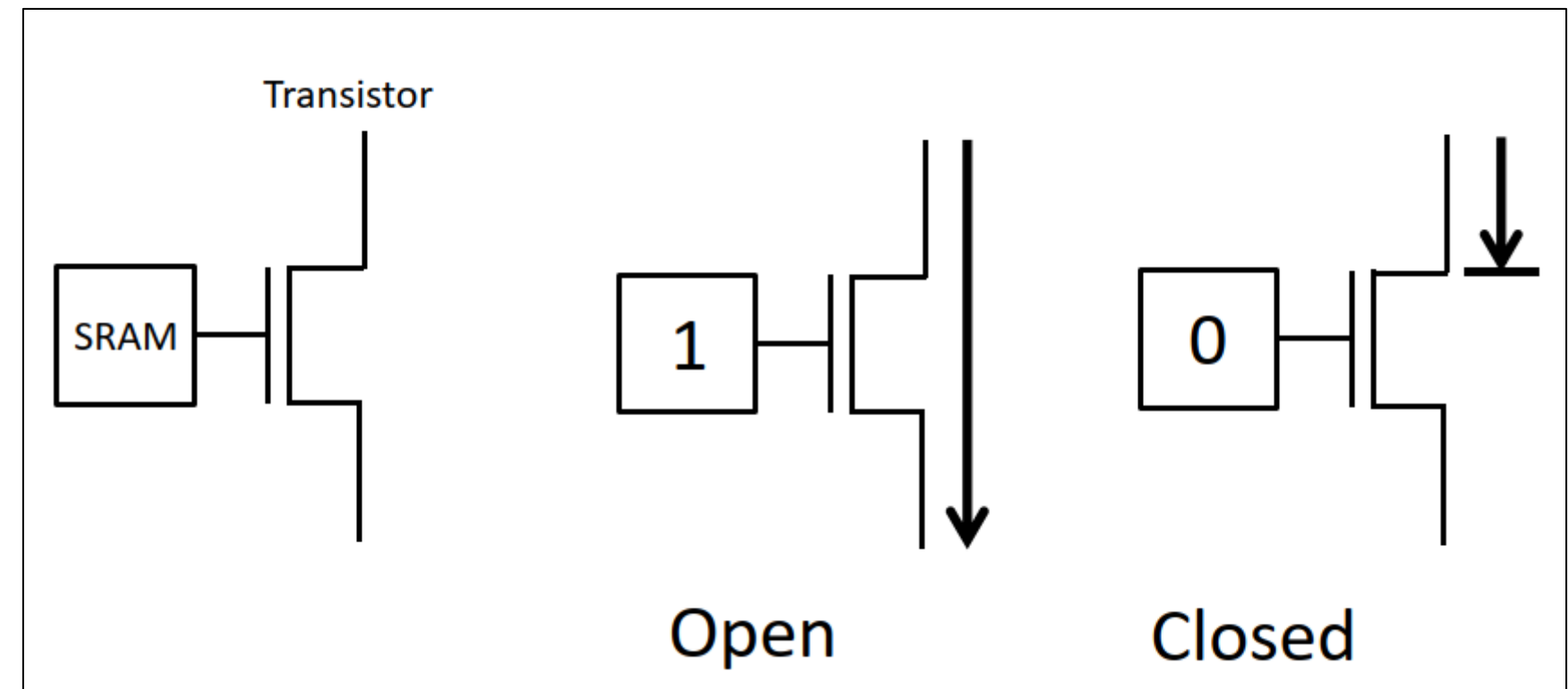
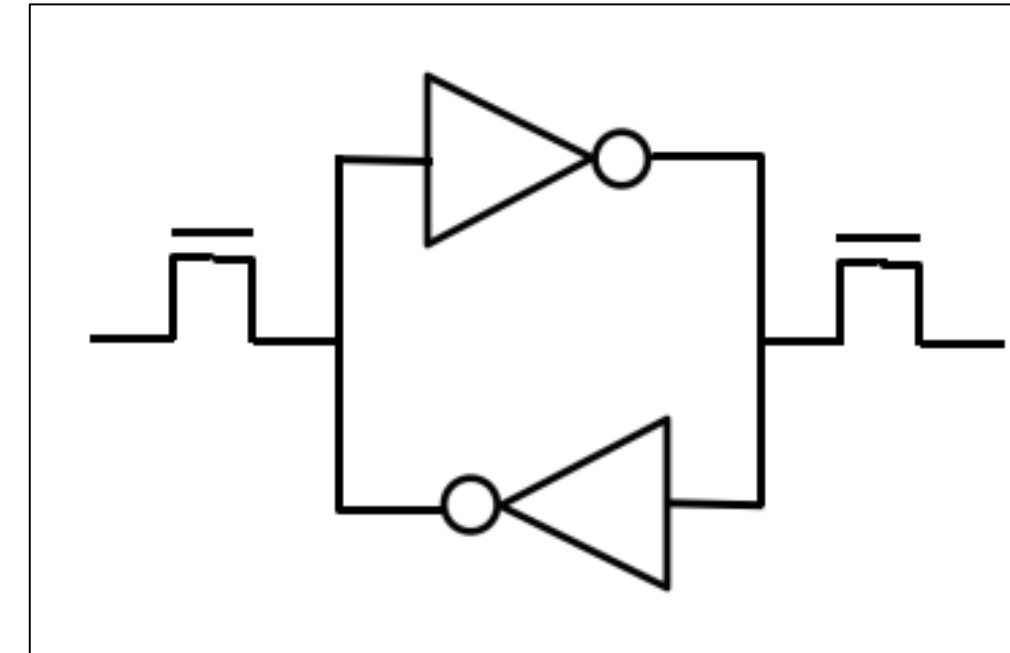
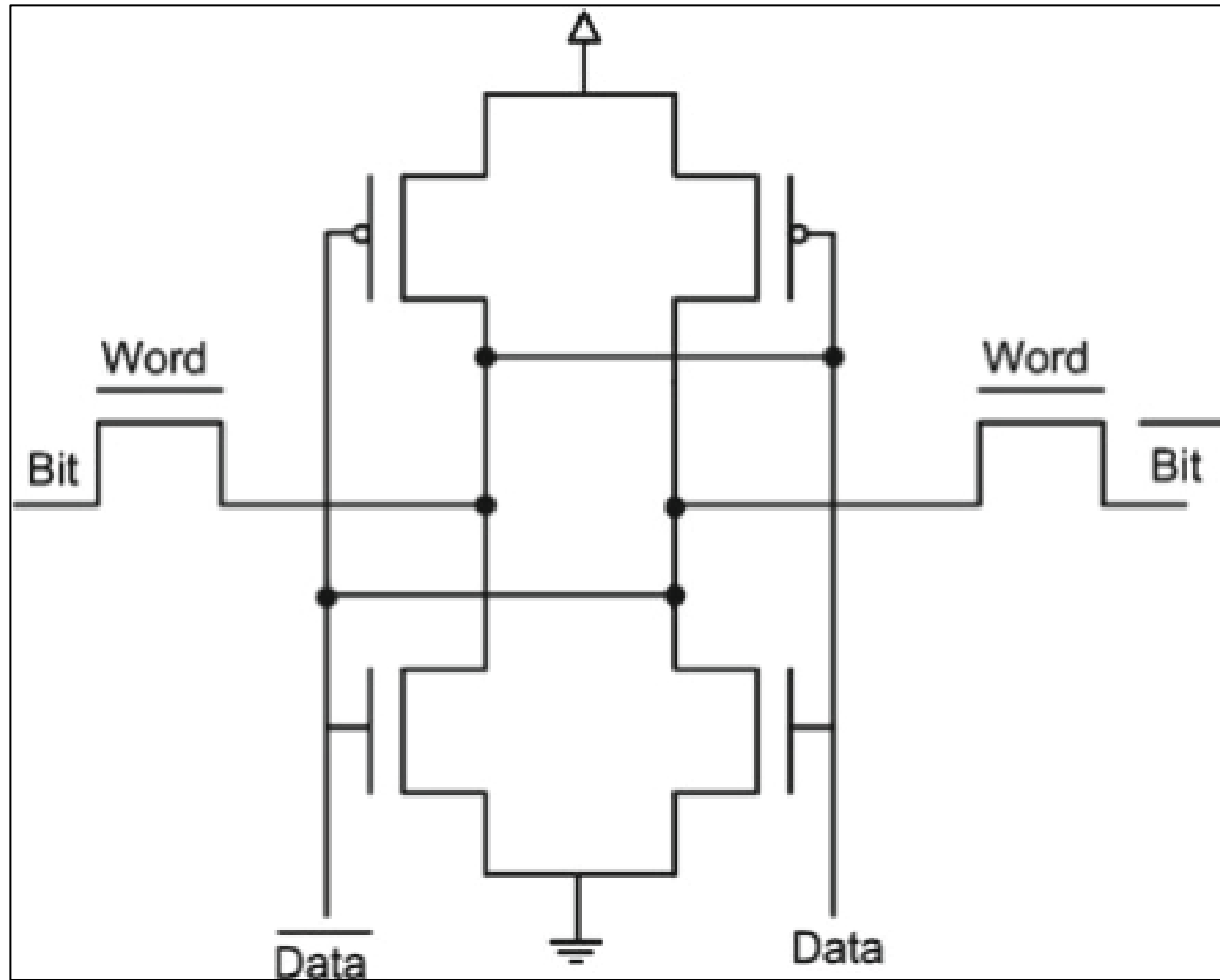
FPGA configuration : Anti-fuses



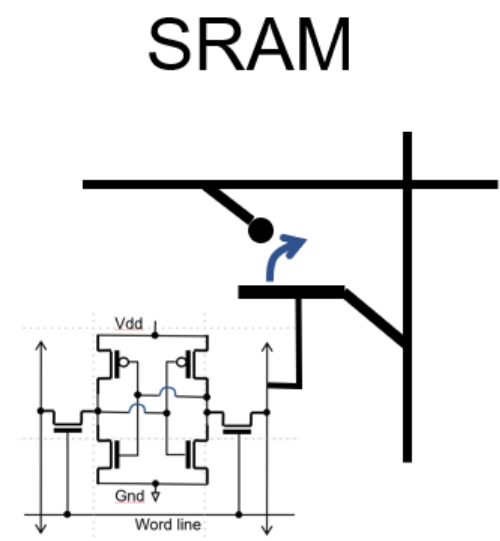
FPGA configuration : Flash



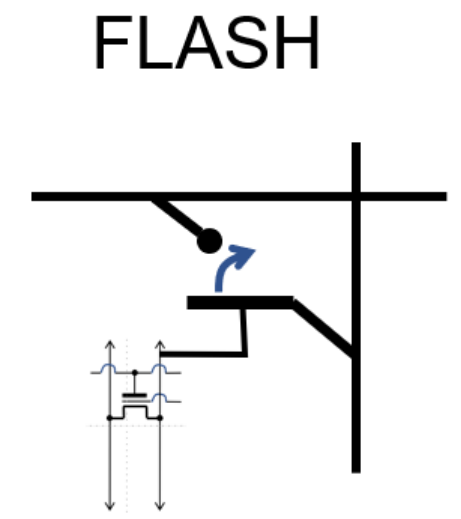
FPGA configuration : SRAM



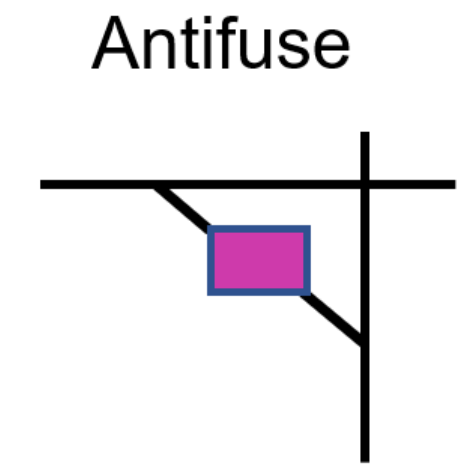
FPGA configuration



Reprogrammable
and volatile



Reprogrammable
and Non-volatile



One-time programmable
and Non-volatile

Name	Volatile	Re-programmable	Delay	Area
Flash	No	In-circuit	Large	Medium
SRAM	Yes	In-circuit	Large	Large
Anti-fuse	No	No	Small	Small

FPGAs Fabric

- **Field Programmable Gate Array (FPGA) consist of,**
 - **Basic Section of FPGA :**
 - Configurable Logic Blocks (CLBs) / Logic Elements (LEs)
 - Programmable Interconnect (Routing Fabric)
 - I/O Blocks (IOBs)
 - **More Advanced FPGA Contains:**
 - Block RAM (BRAM)
 - DSP Blocks (Digital Signal Processing)
 - Clock management tiles (CMTs) (PLL/DLL)
 - Hard Processor System (HPS)

Field Programmable Gate Arrays (FPGAs)

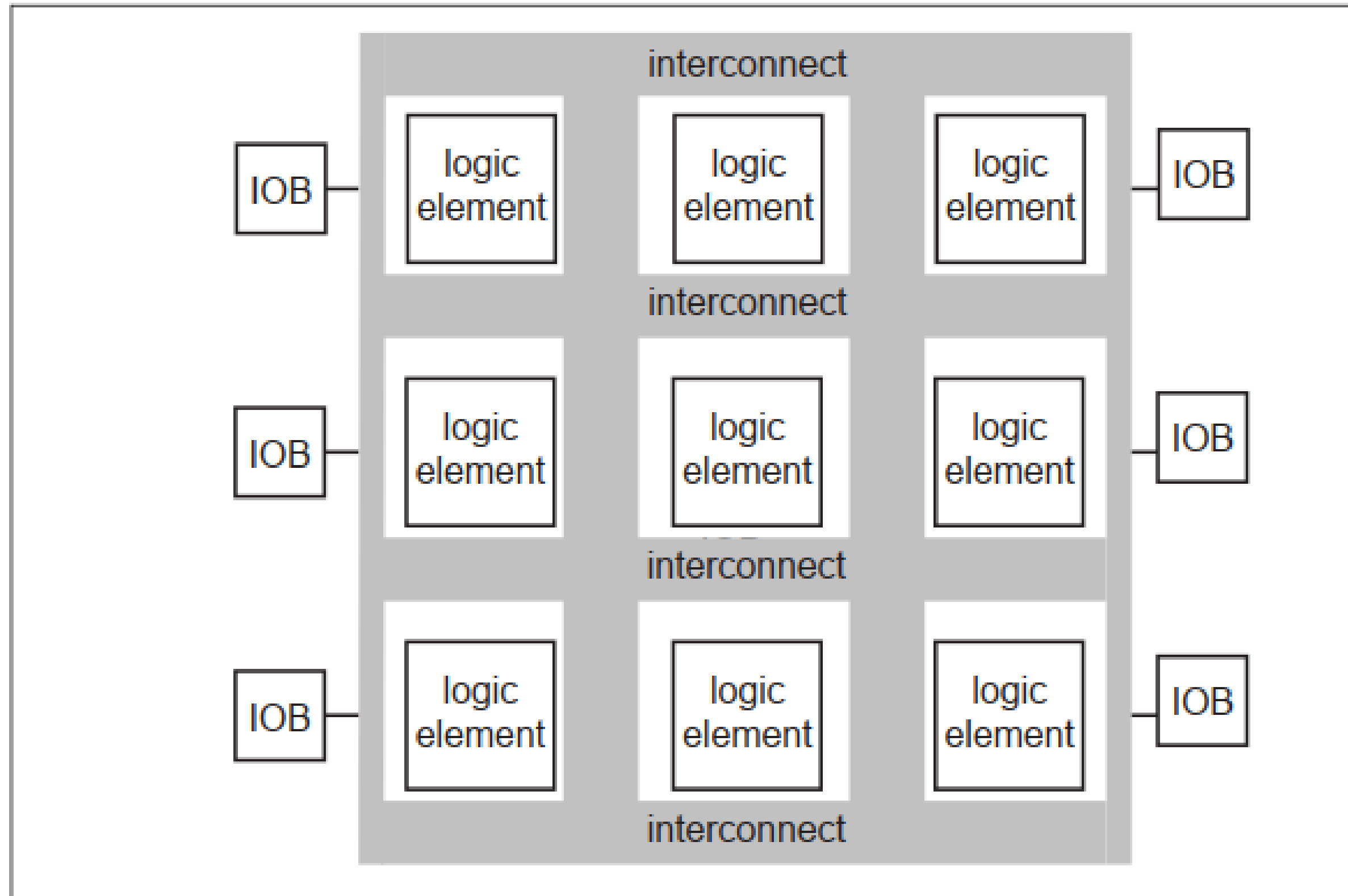
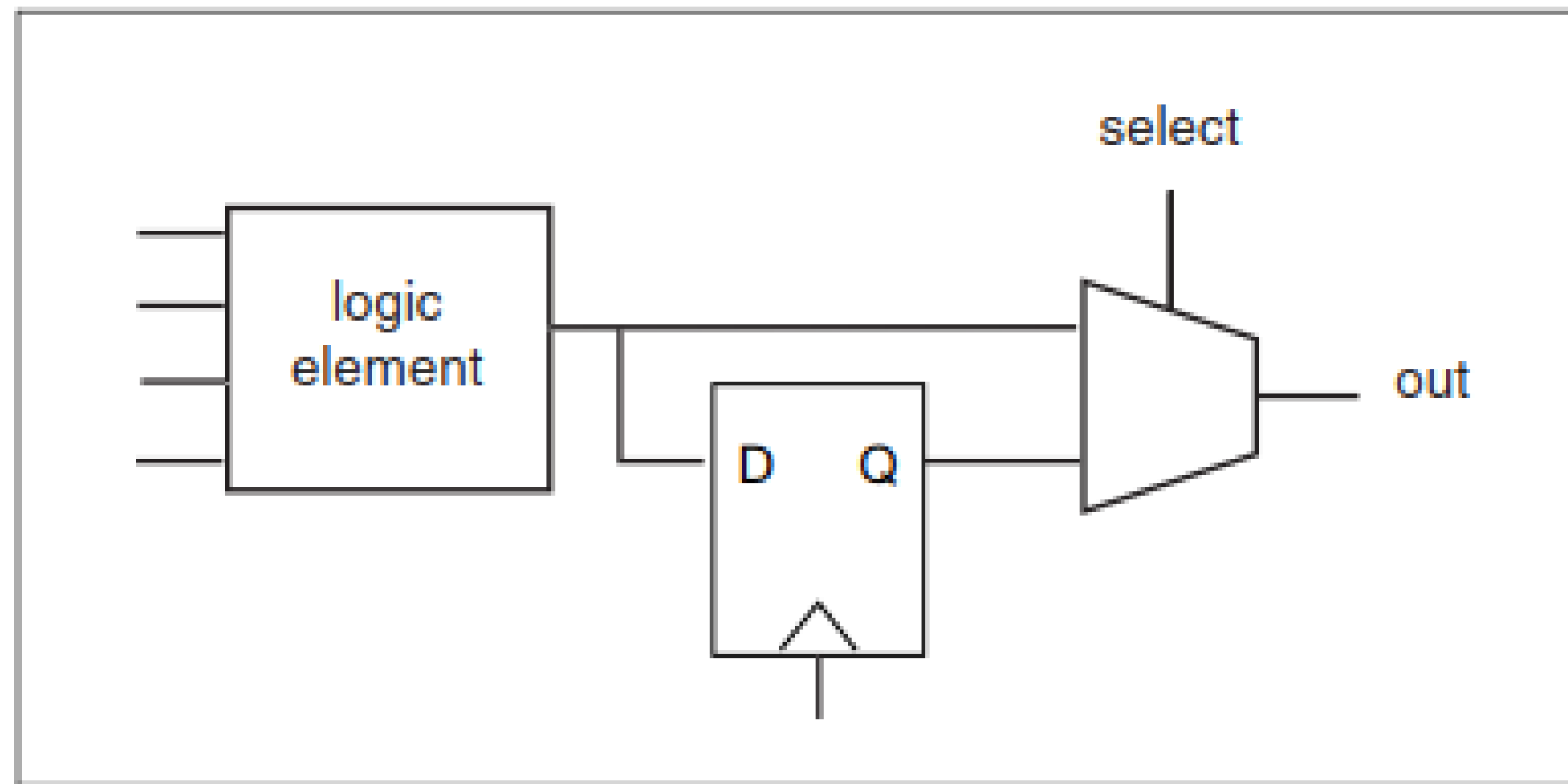


Figure 3-1 Generic structure of an FPGA fabric.

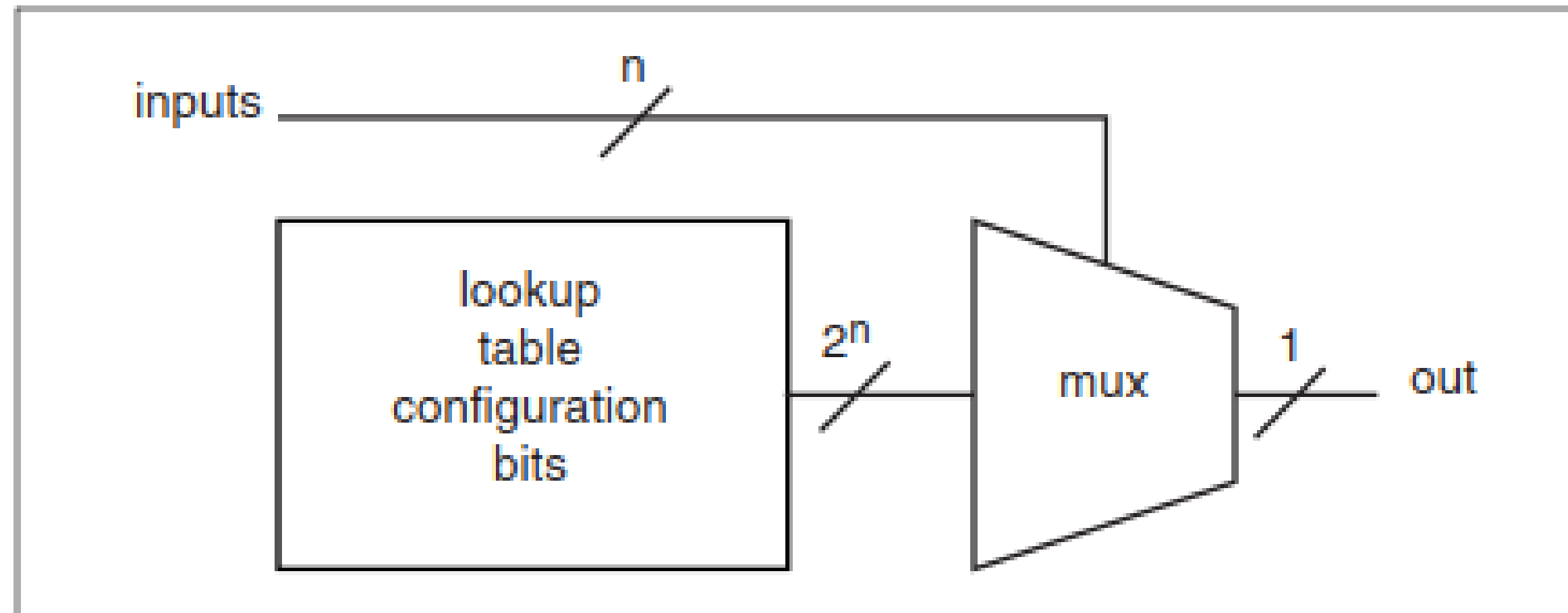
Configurable Logic Blocks (CLBs) / Logic Elements (LEs)

Figure 3-5 A flip-flop in a logic element.



Look Up Tables (LUT)

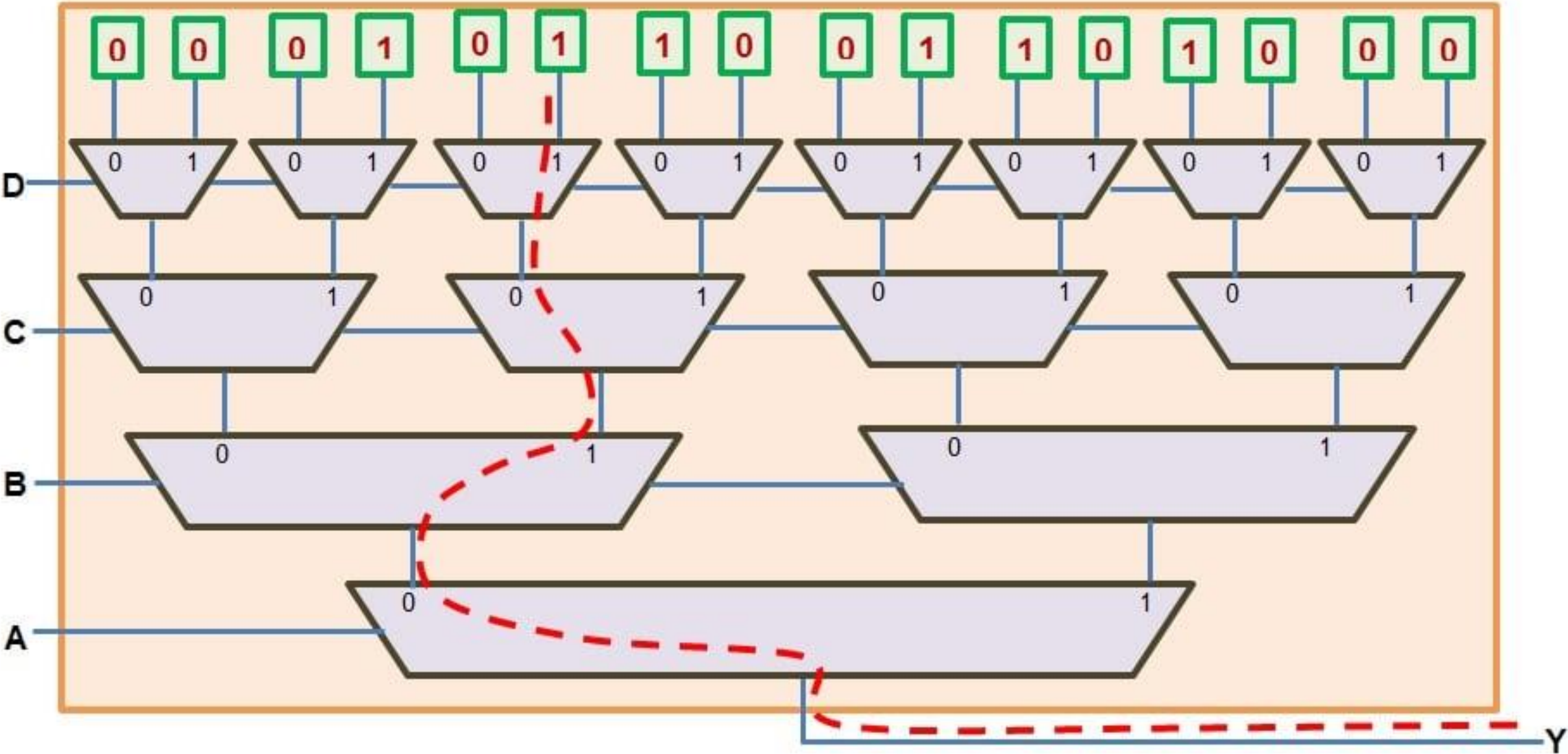
Figure 3-4 A lookup table.



Look Up Tables (LUT)

Truth Table

Inputs				Output
A	B	C	D	Y
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0



Look Up Tables (LUT)

Truth Table

Inputs				Output
A	B	C	D	Y
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	1
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

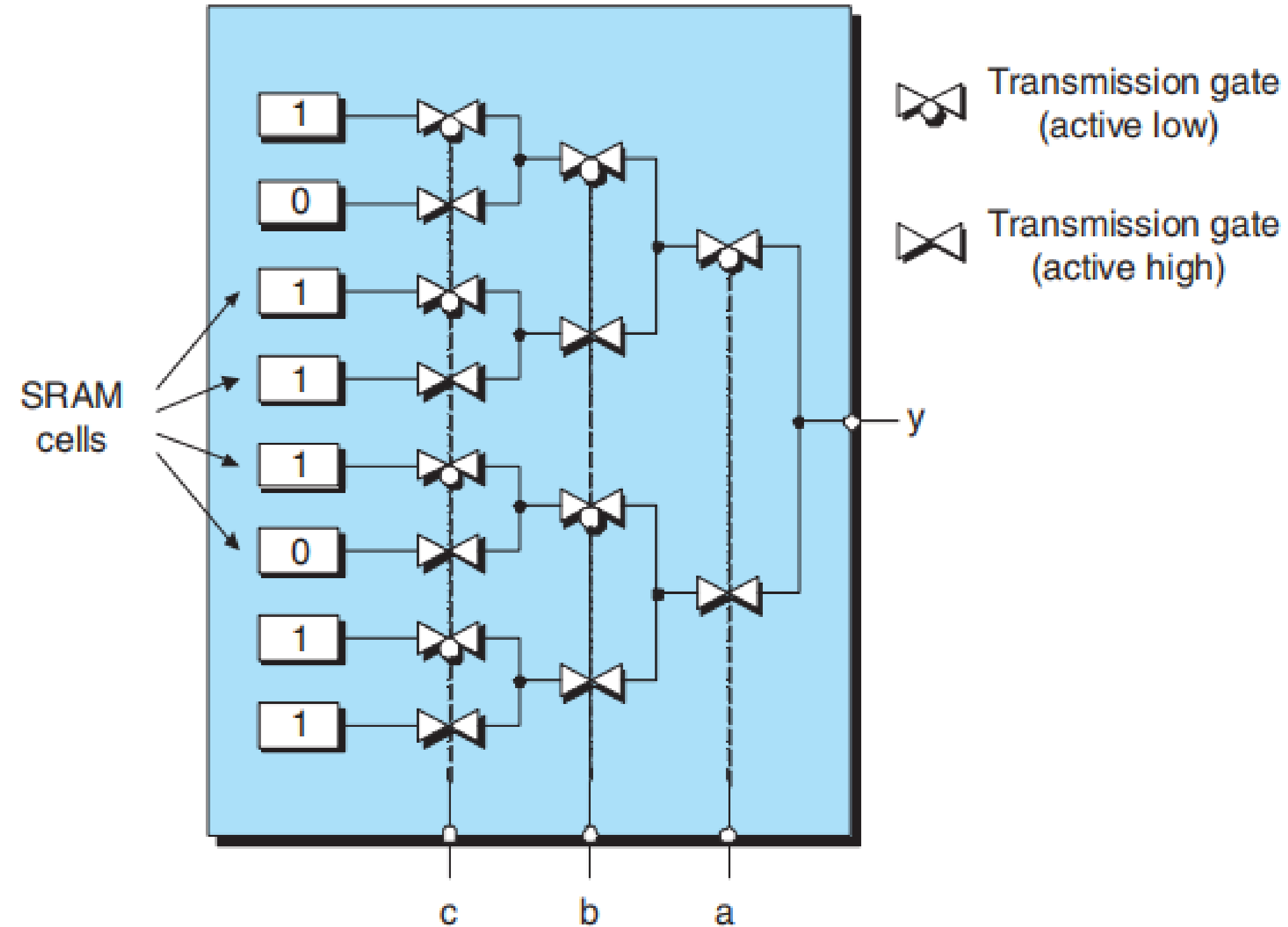
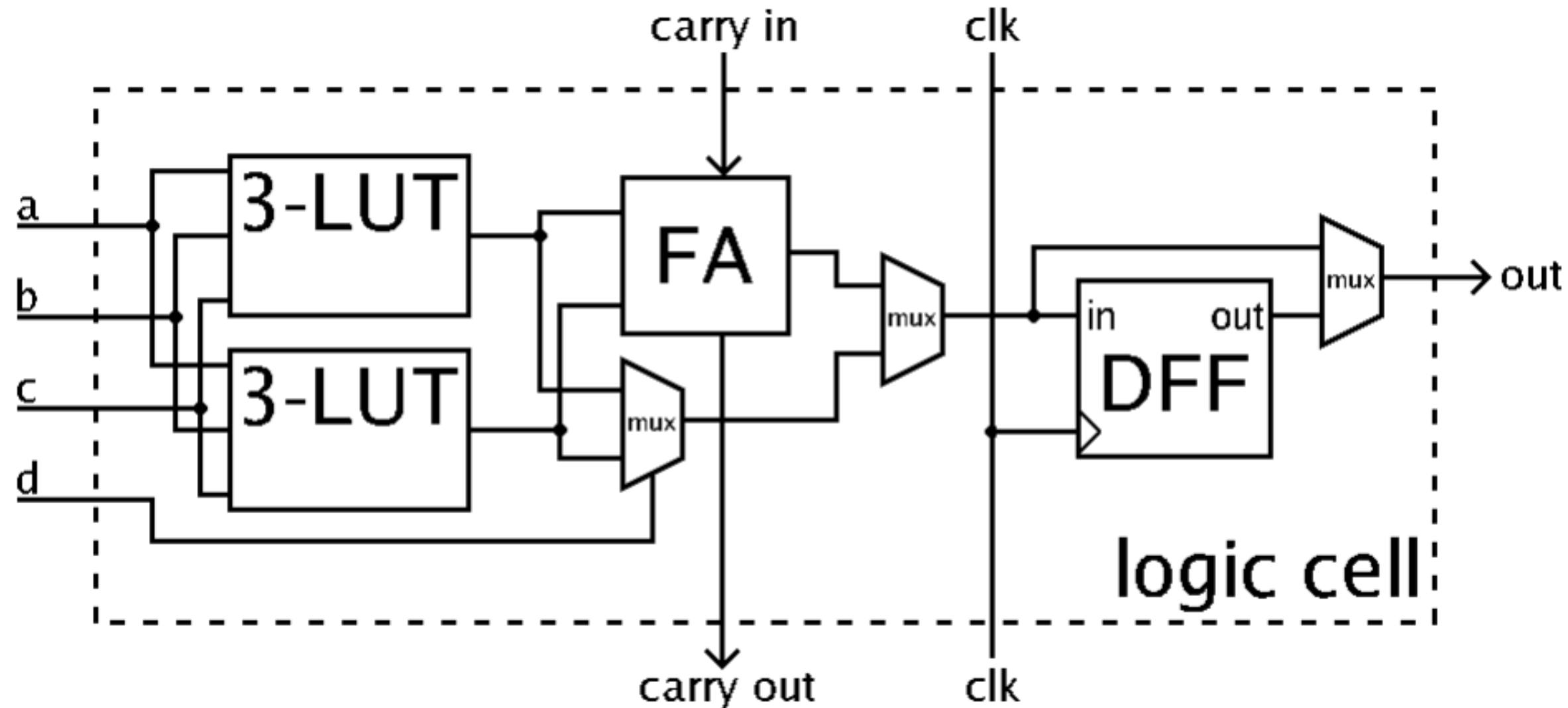


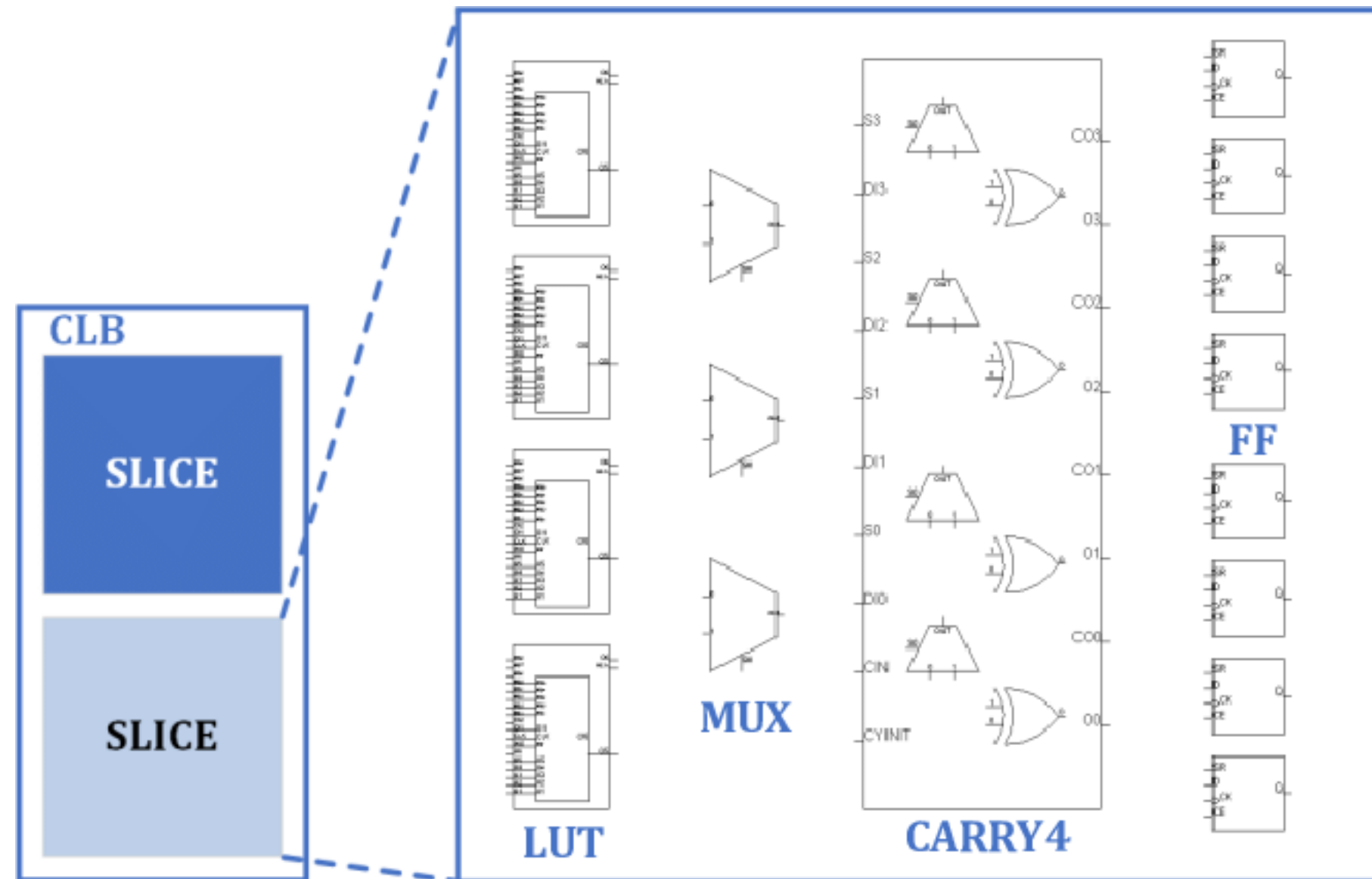
FIGURE 2-3 A transmission gate-based LUT (programming chain omitted for purposes of clarity).
© Eng. Ahmed Hussein

Complex logic elements

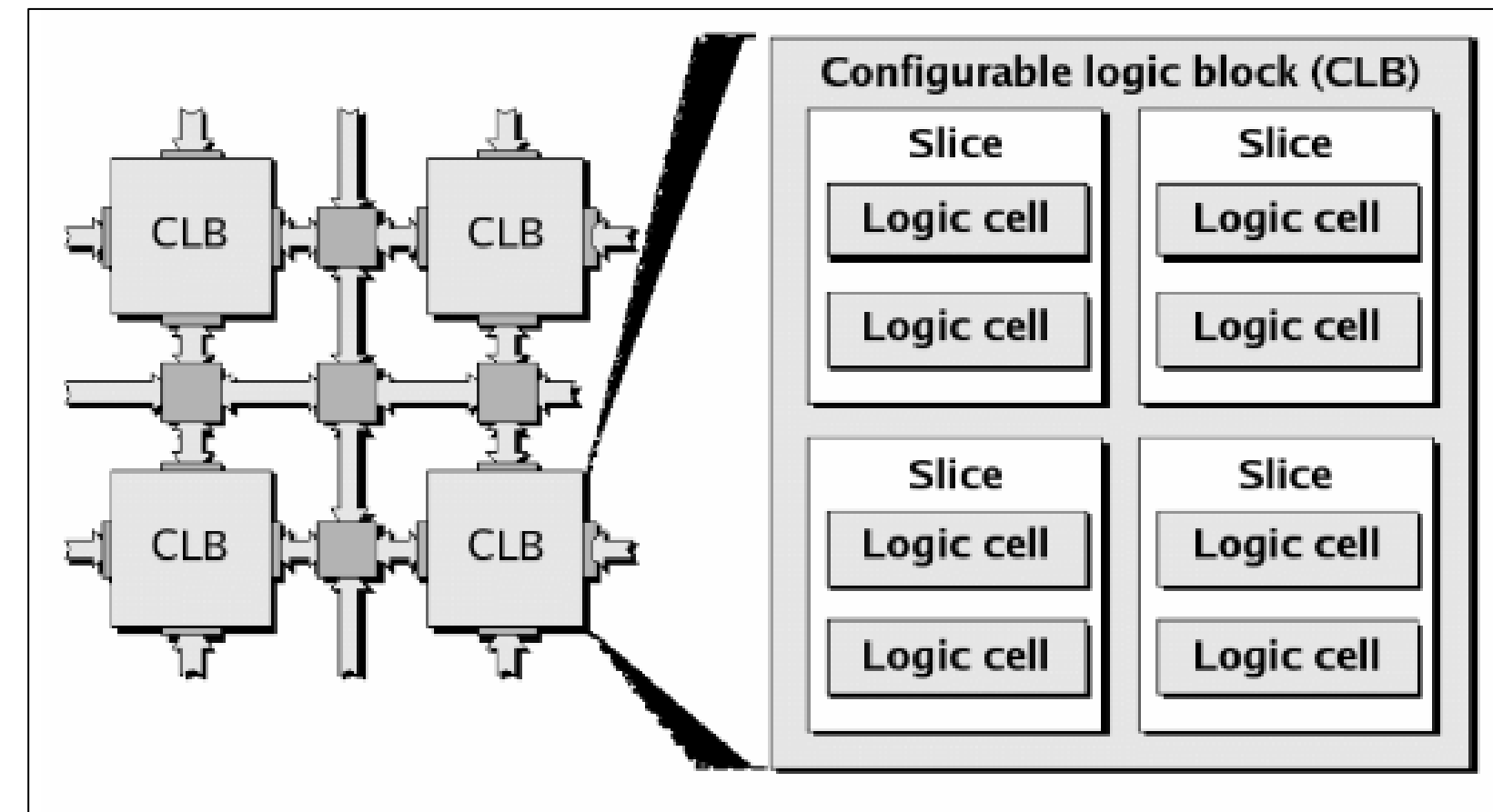
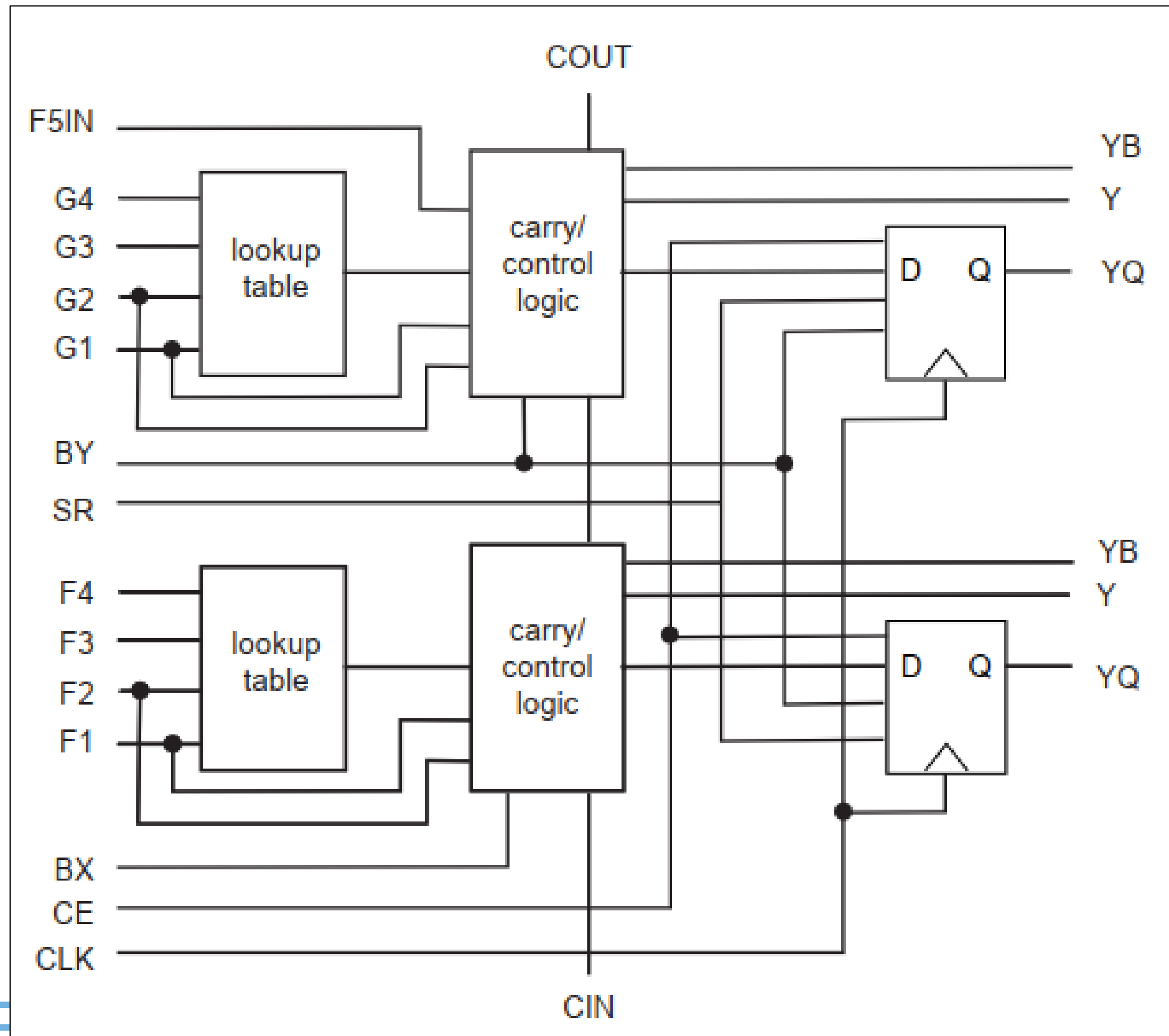


- **Xilinx** calls them **logic cells (LC)**
- **Altera** calls them **logic elements (LE)**.
- **A configurable logic block (CLB, Xilinx)**
- **logic array block (LAB, Altera)**

configurable logic block (CLB, Xilinx)

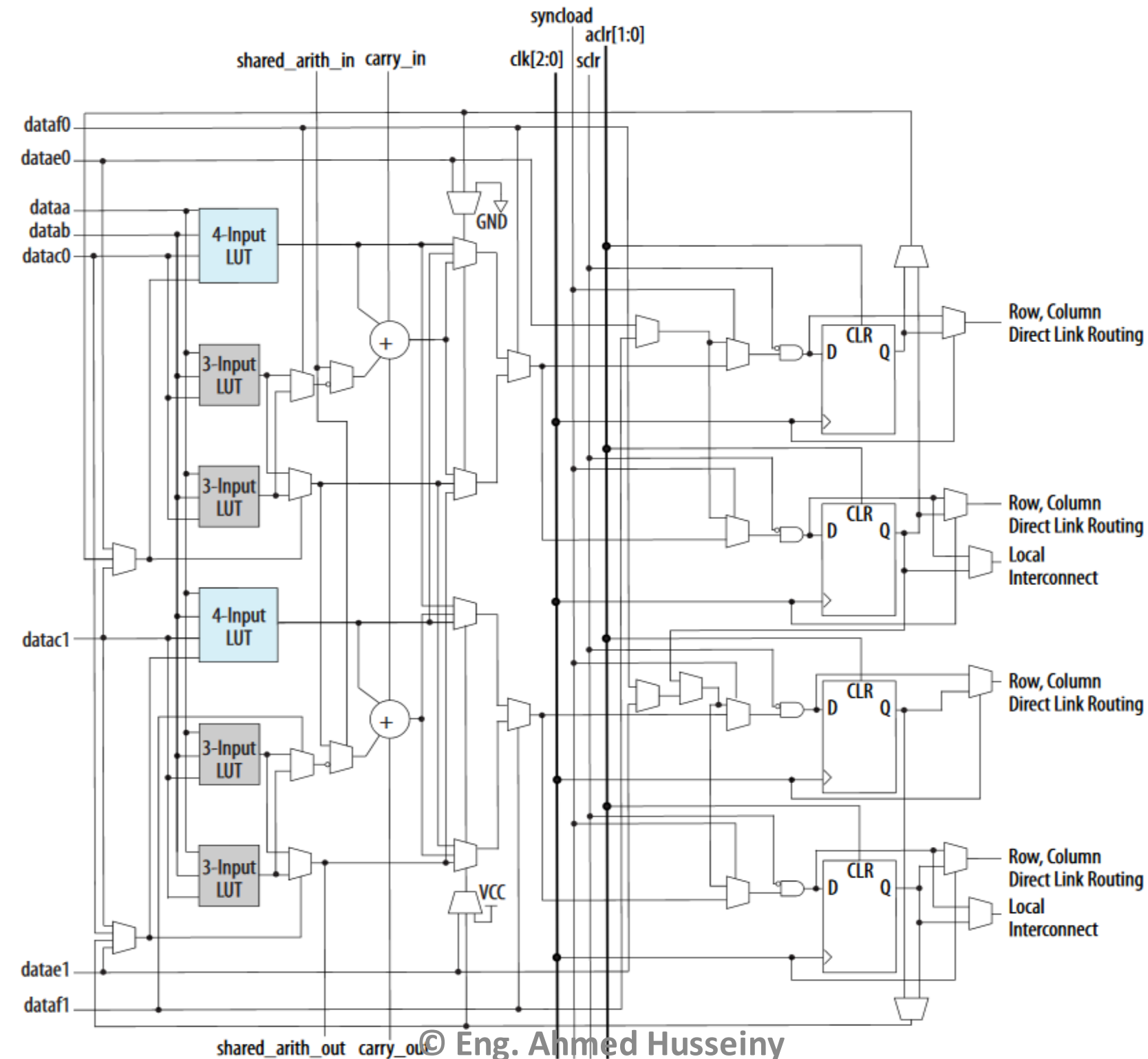


Xilinx Spartan-II combinational logic block



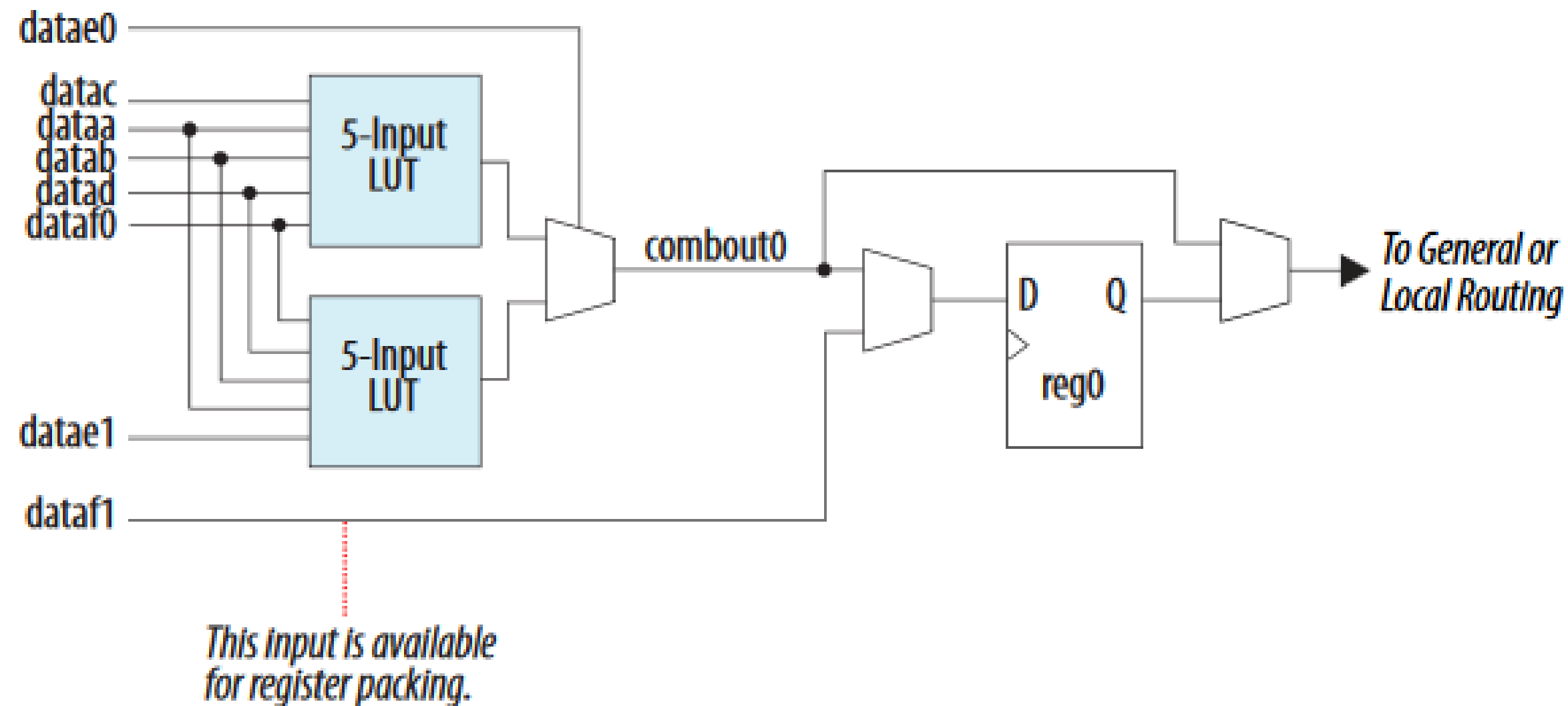
ALTERA Cyclone V

adaptive logic modules (ALMs)

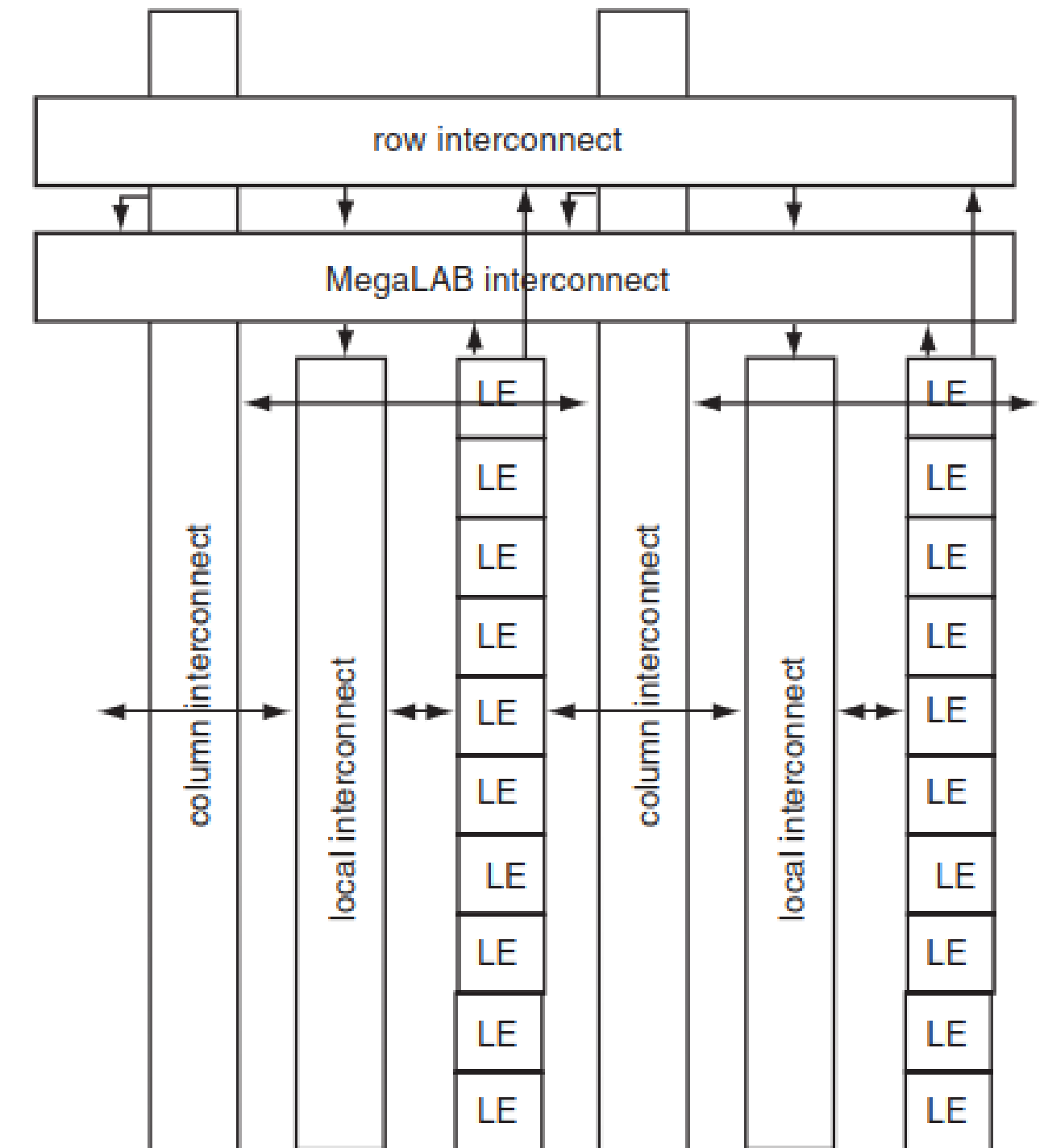
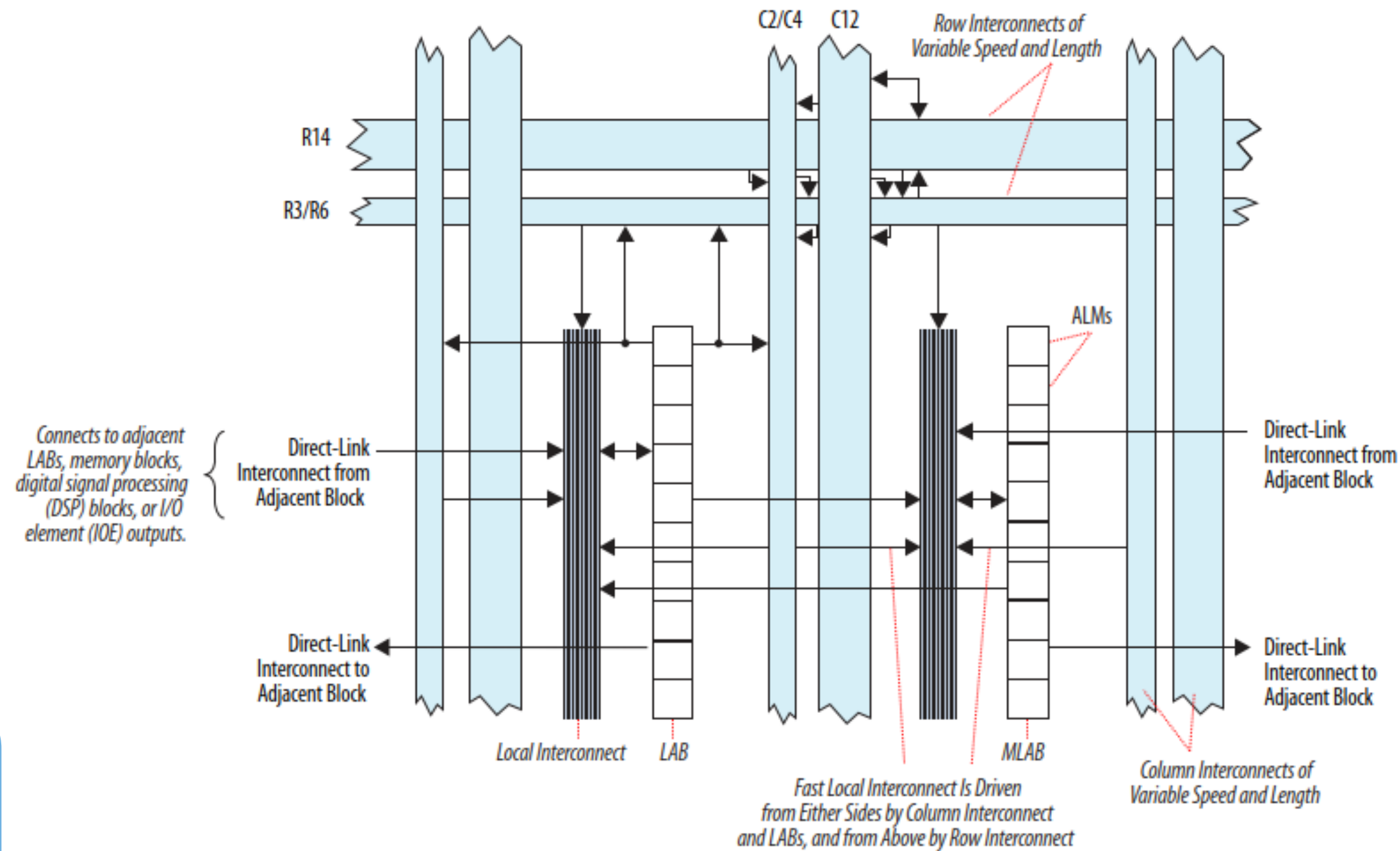


ALTERA Cyclone V

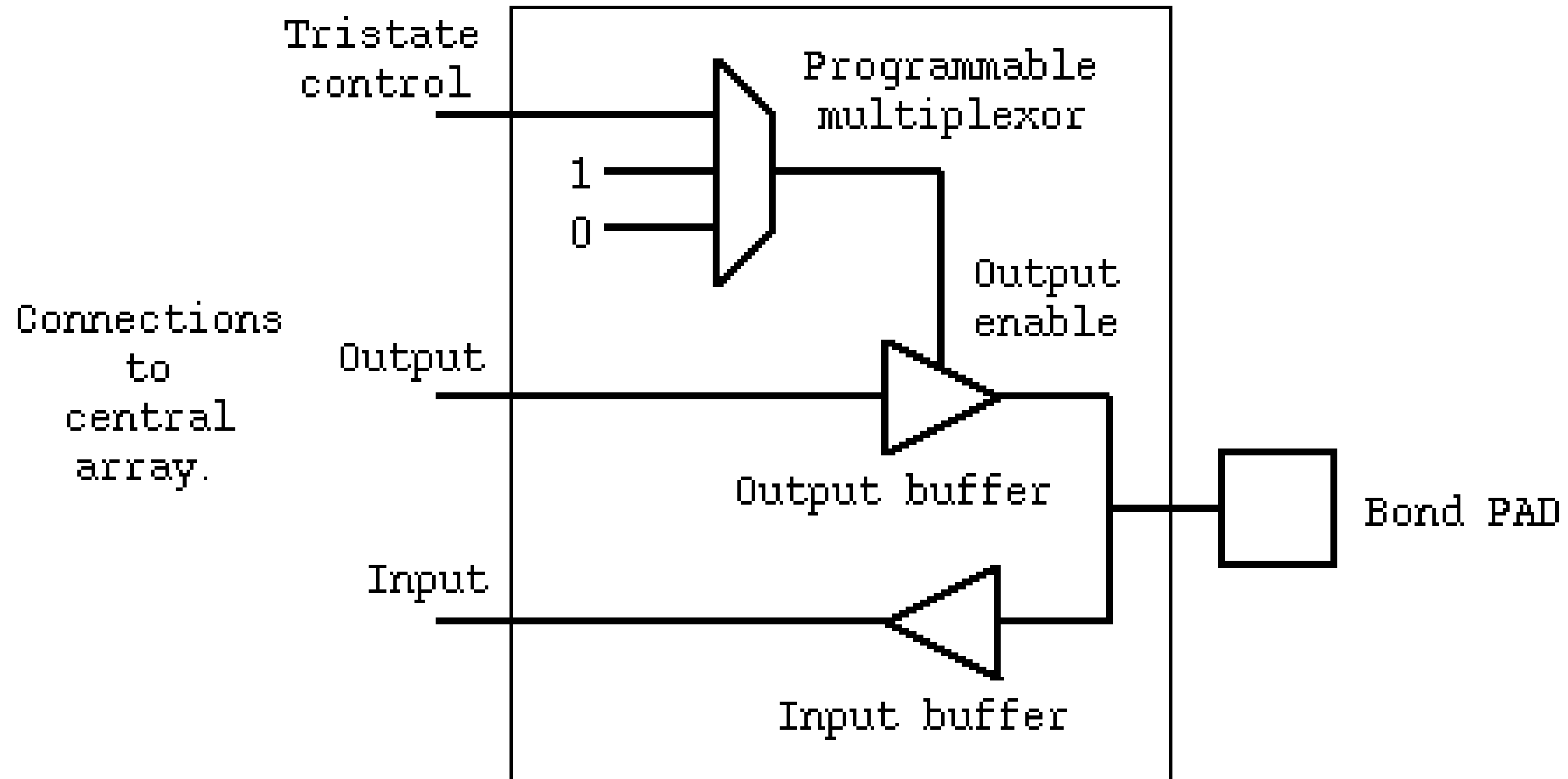
adaptive logic modules (ALMs)



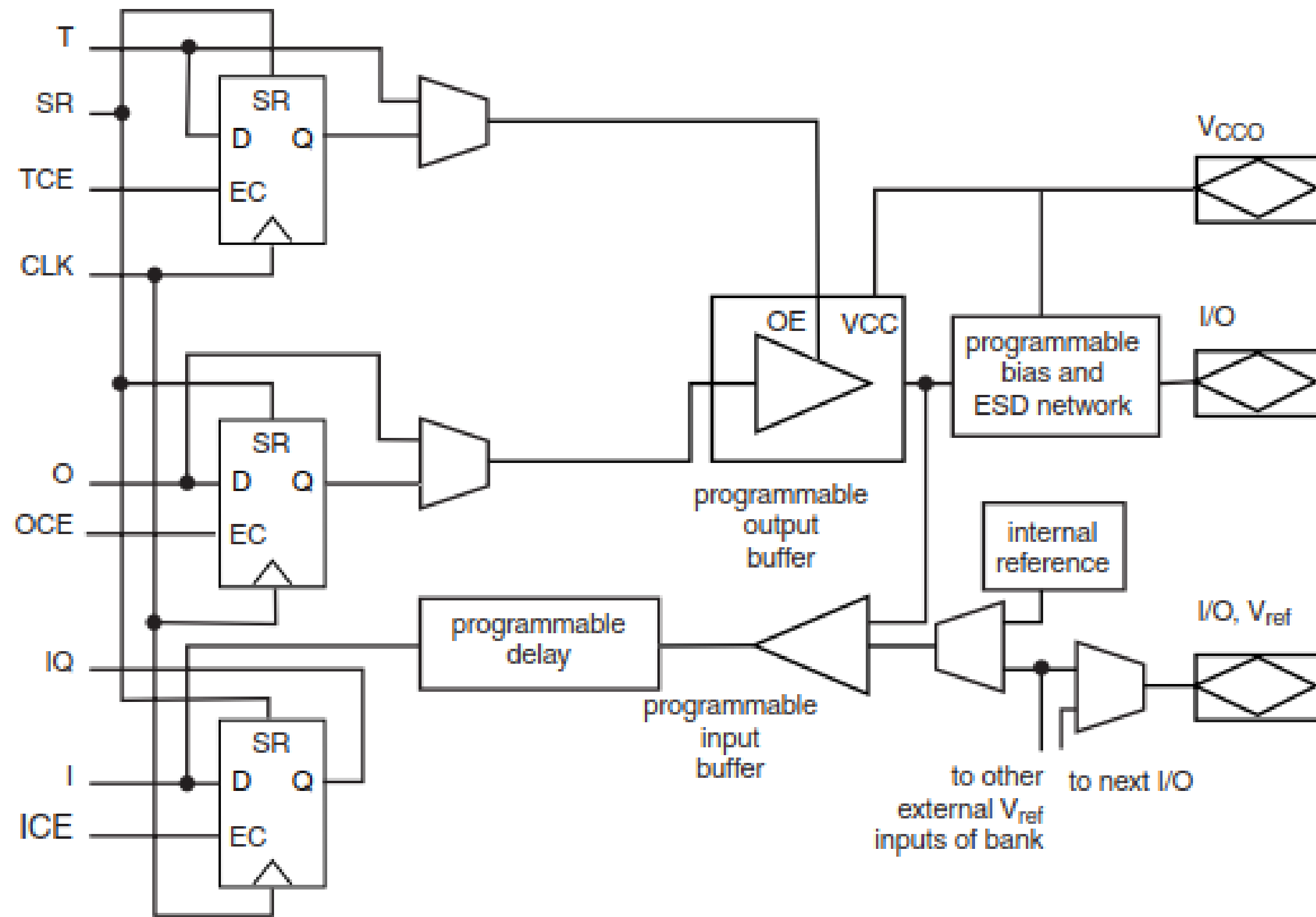
ALTERA Cyclone V logic array block (LAB)



I/O Blocks (IOBs)



I/O Blocks (IOBs)

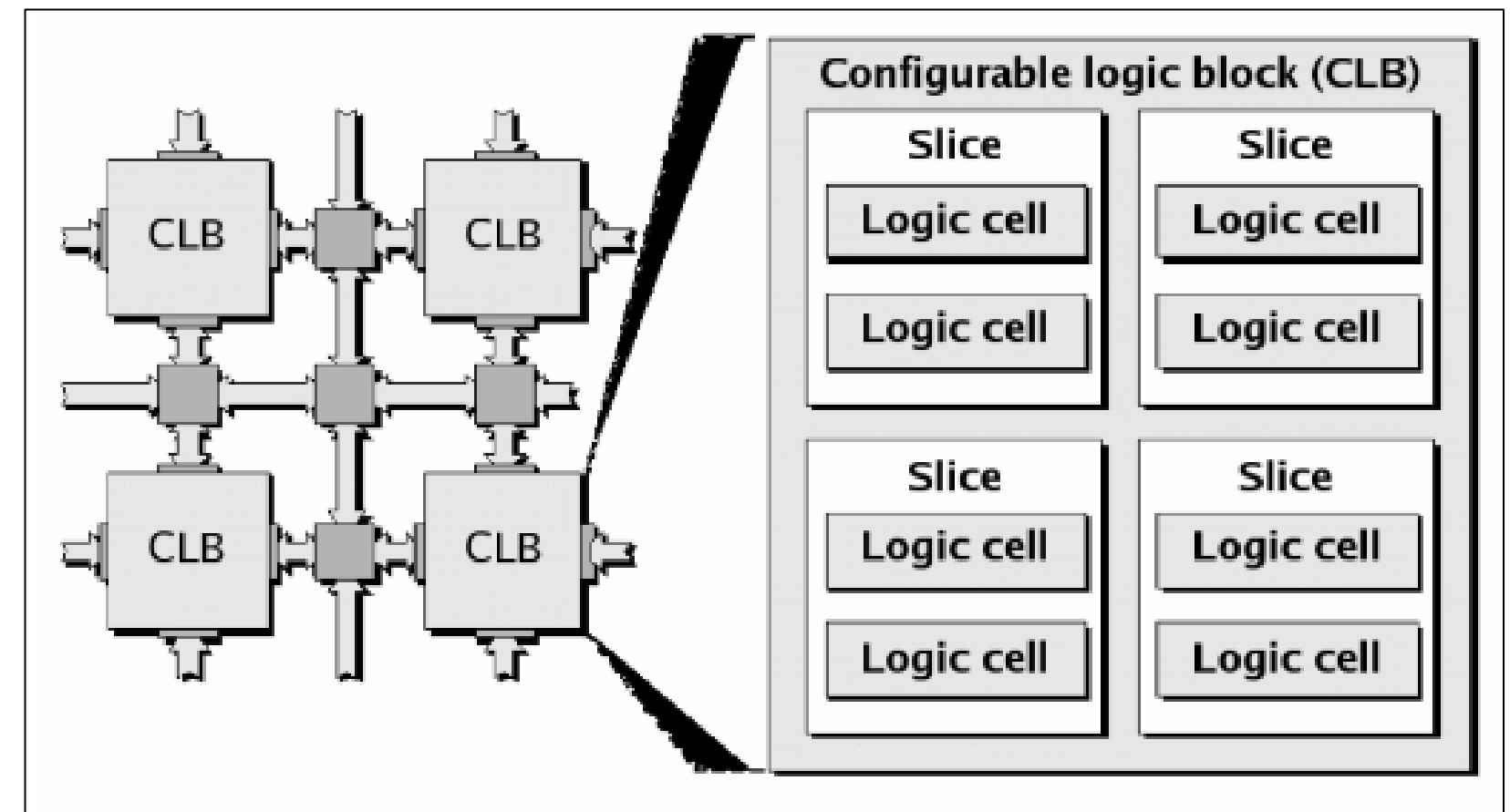


Distributed RAMs

- LUT (normally used for **logic**) or any other **memory** within a configurable cell used as a **distributed RAM**
 - **LUTRAM**: LUT as used **RAM**
- **fast**
- allows **collocating memory** and **computation**
- can **reduce routing**
- **DISADVANTAGE**: consumes resources for otherwise **used for logic implementation**

The shown **CLB** can be treated as, (**4 - input LUT , 2 LUT / Slice , 4 Slices/CLB**)

- **Single-port 16 x 8** bit RAM
- **Single-port 32 x 4** bit RAM
- **Single-port 64 x 2** bit RAM
- **Single-port 128 x 1** bit RAM
- **Dual-port 16 x 4** bit RAM
- **Dual-port 32 x 2** bit RAM
- **Dual-port 64 x 1** bit RAM



Distributed RAMs

You can use an MLAB ALM as a regular LAB ALM or configure it as a dual-port SRAM.

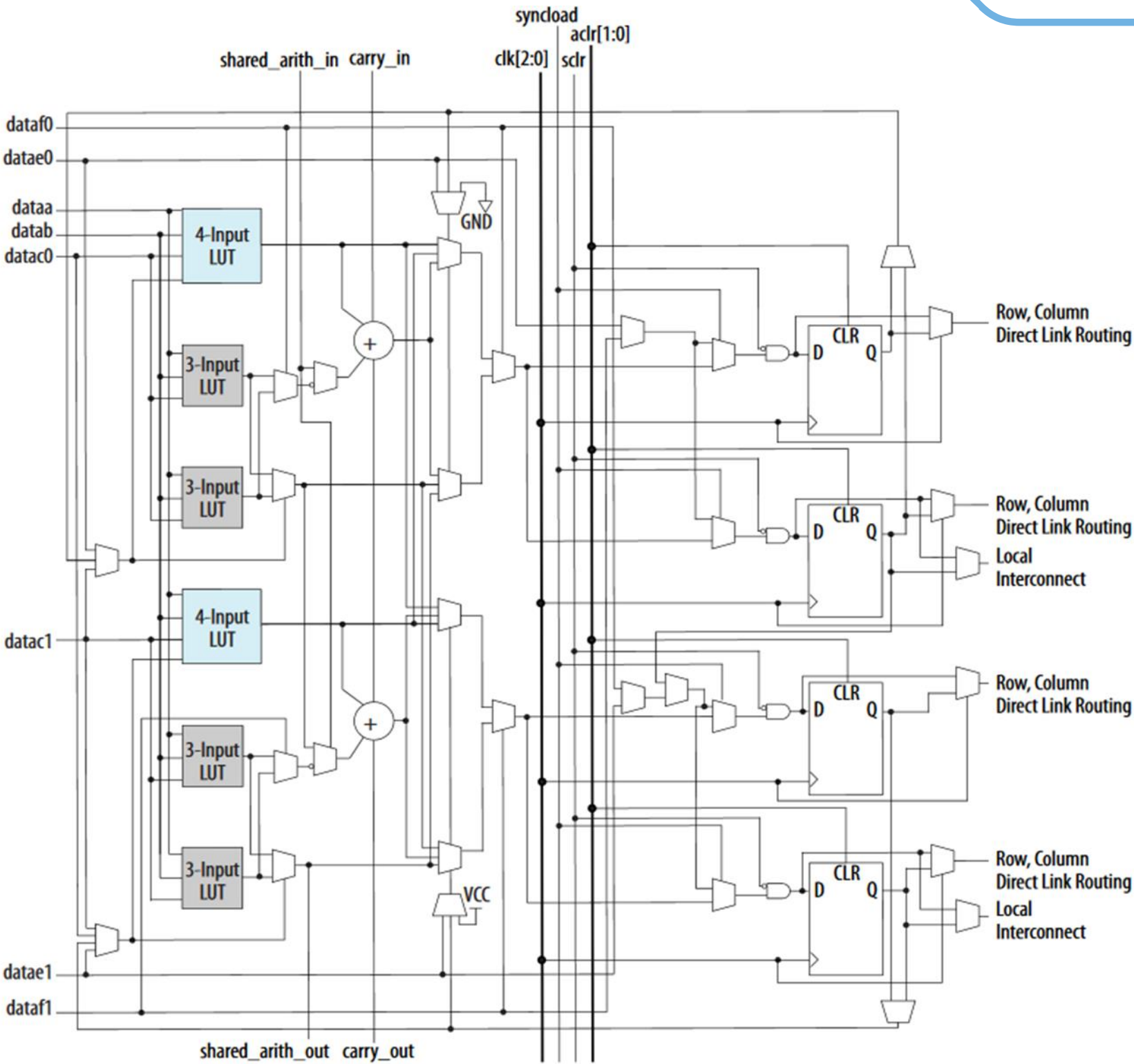
You can use an MLAB ALM as a regular LAB ALM or configure it as a dual-port SRAM.

LUT-Based-32 x 2 Simple Dual-Port SRAM	ALM
LUT-Based-32 x 2 Simple Dual-Port SRAM	ALM
LUT-Based-32 x 2 Simple Dual-Port SRAM	ALM
LUT-Based-32 x 2 Simple Dual-Port SRAM	ALM
LUT-Based-32 x 2 Simple Dual-Port SRAM	ALM
LAB Control Block	LAB Control Block
LUT-Based-32 x 2 Simple Dual-Port SRAM	ALM
LUT-Based-32 x 2 Simple Dual-Port SRAM	ALM
LUT-Based-32 x 2 Simple Dual-Port SRAM	ALM
LUT-Based-32 x 2 Simple Dual-Port SRAM	ALM
LUT-Based-32 x 2 Simple Dual-Port SRAM	ALM

MLAB

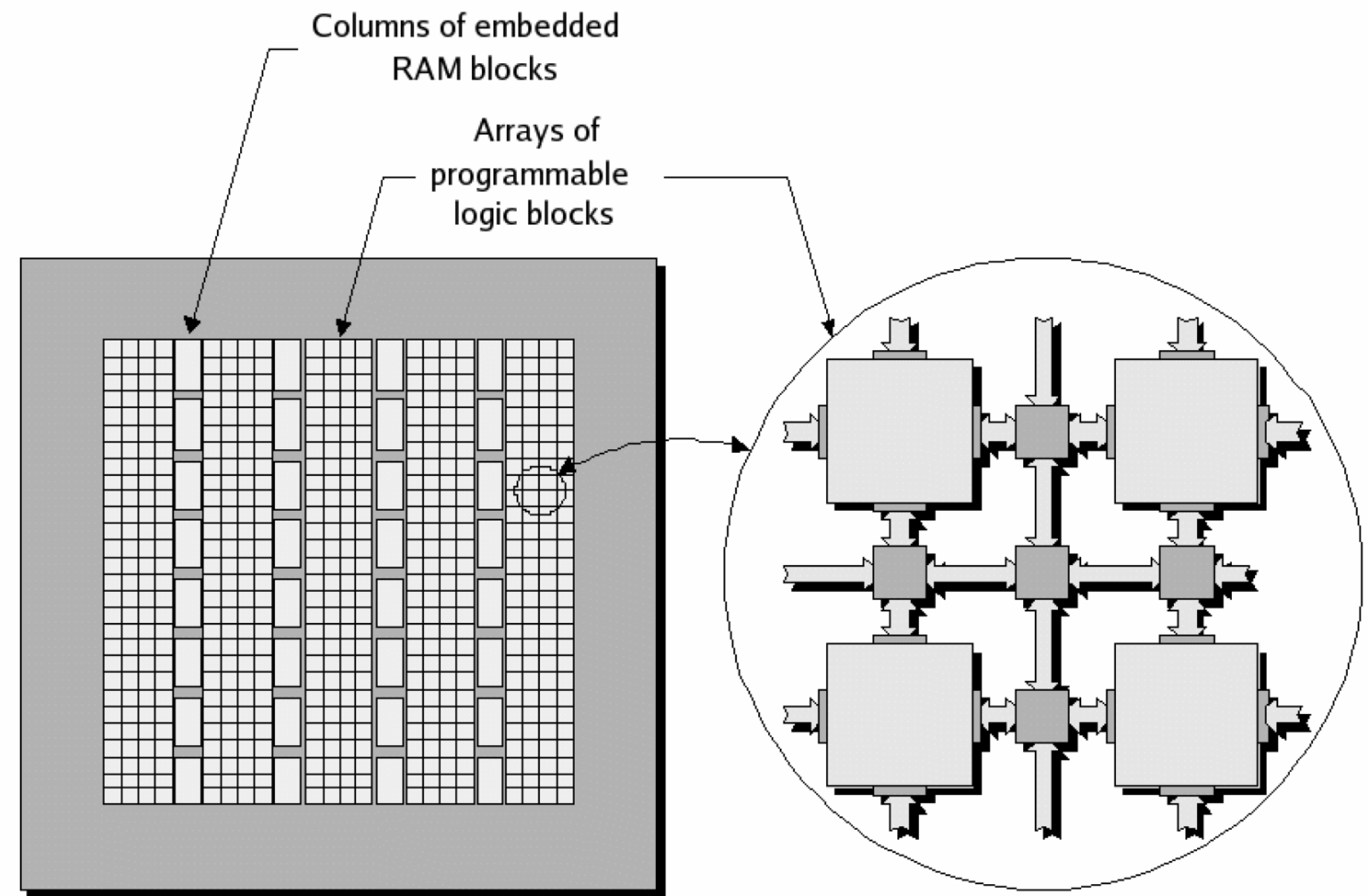
LAB

© Eng. Ahmed Hussein

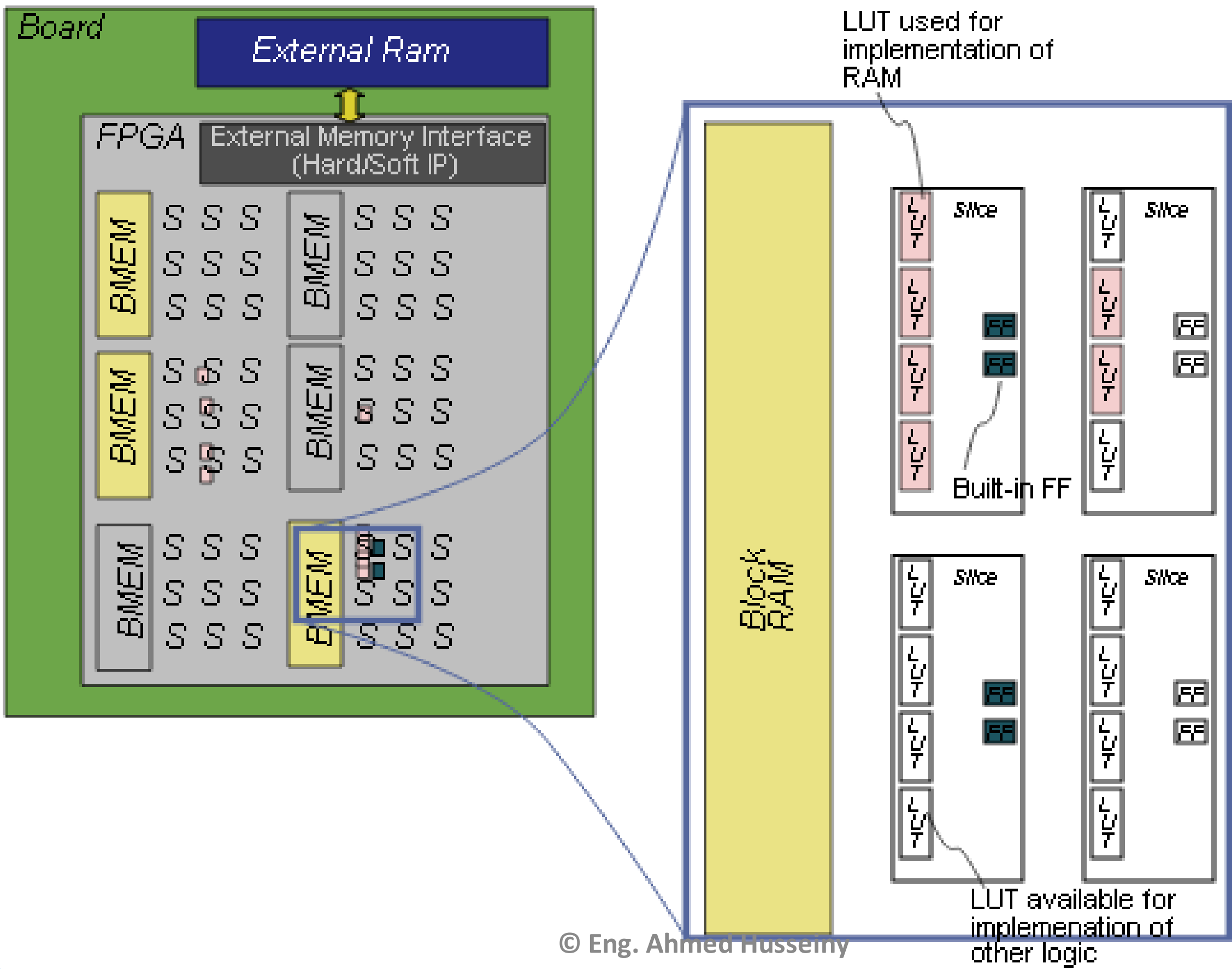


Embedded RAMs

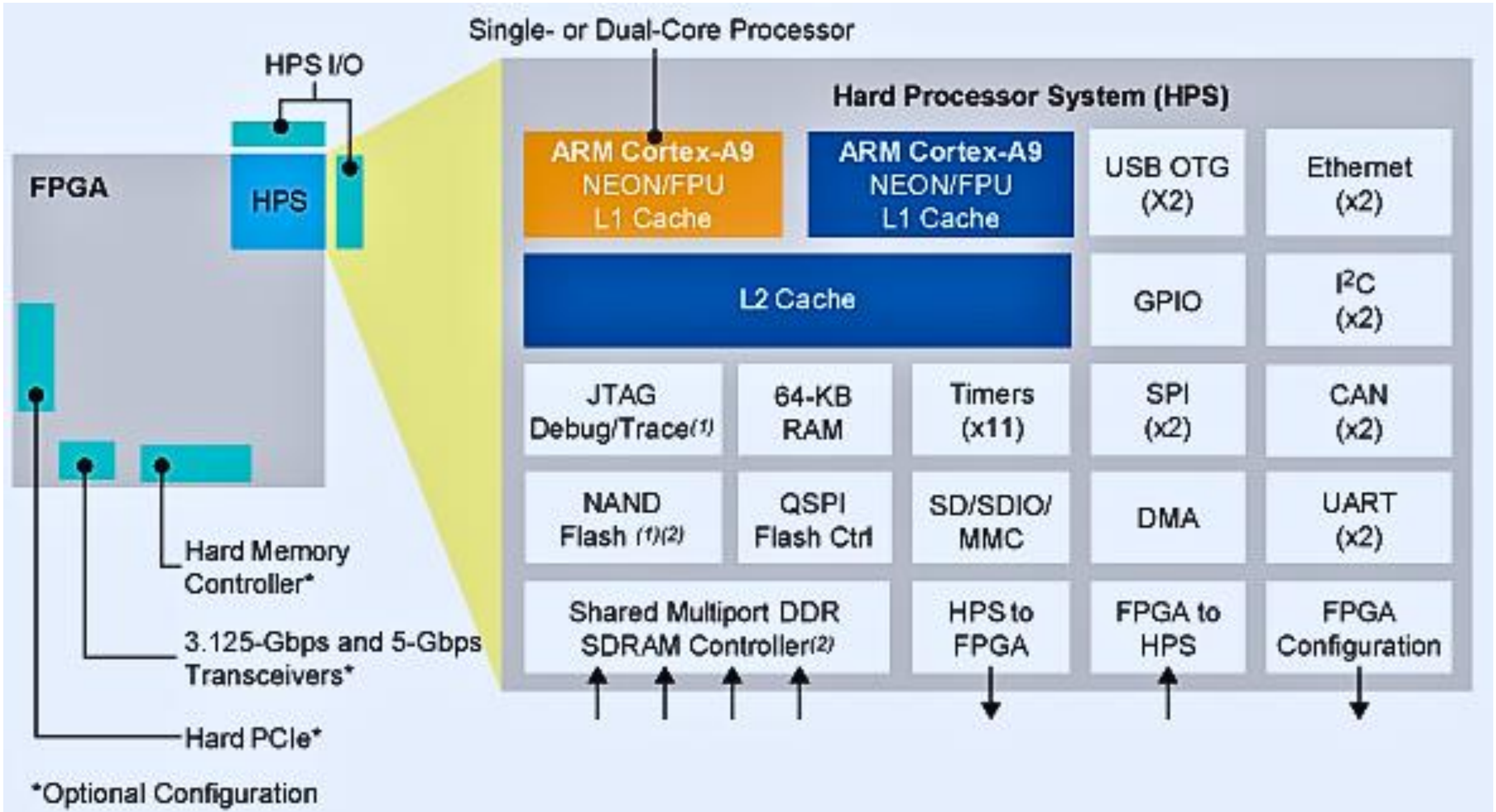
- A lot of applications require **the use of memory**, so FPGAs now include relatively **large chunks of embedded RAM** called **e-RAM** Or **block RAM**
- Each **e-RAM** can be used **independently**, or **multiple** blocks can be combined together to implement **larger blocks**.
- These blocks can be used for a **variety of purposes**, such as implementing standard **single-** or **dual-port RAMs**, **first-in first-out (FIFO)** functions, **state machines**, and so forth



Embedded RAMs



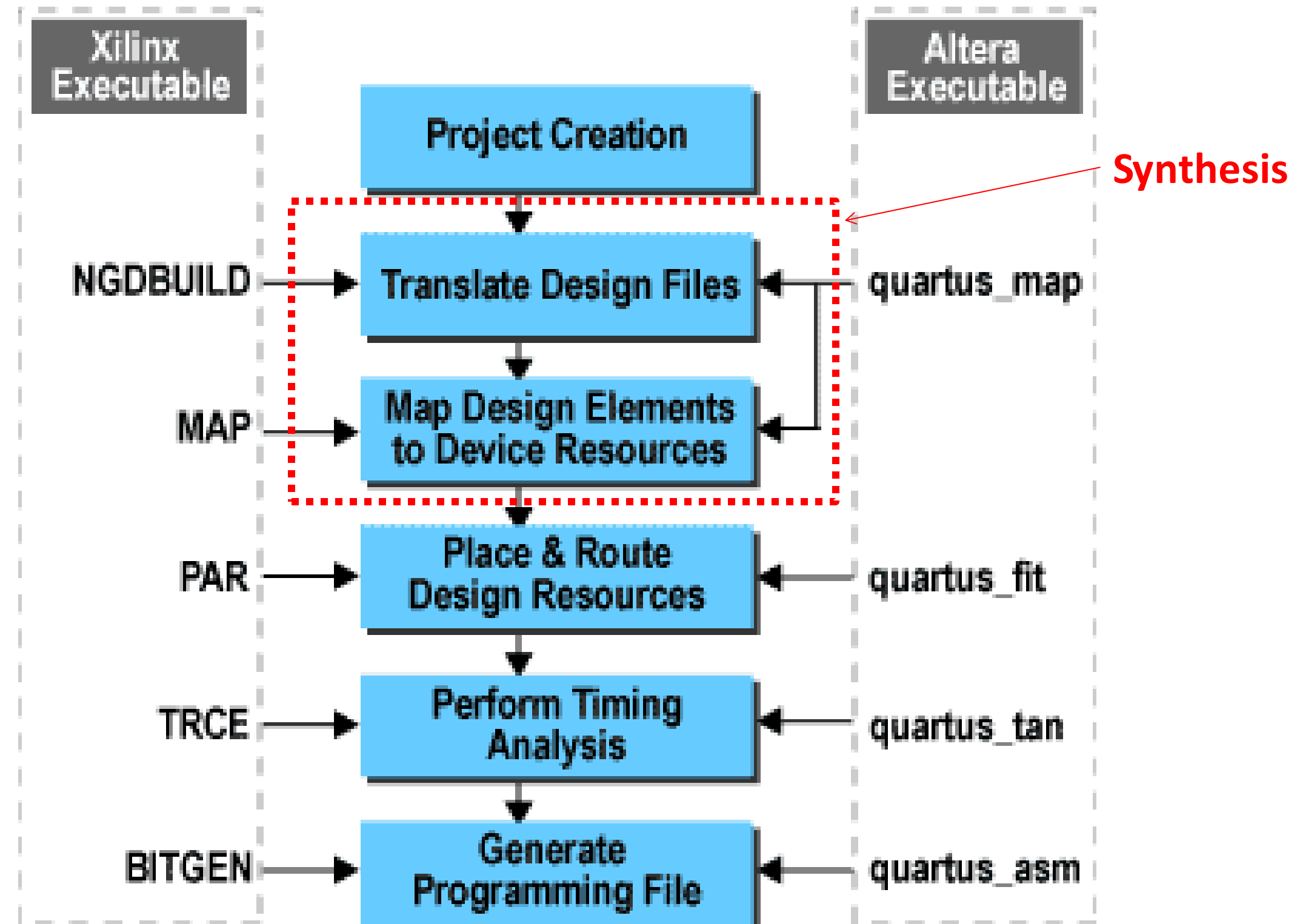
Hard-Processor



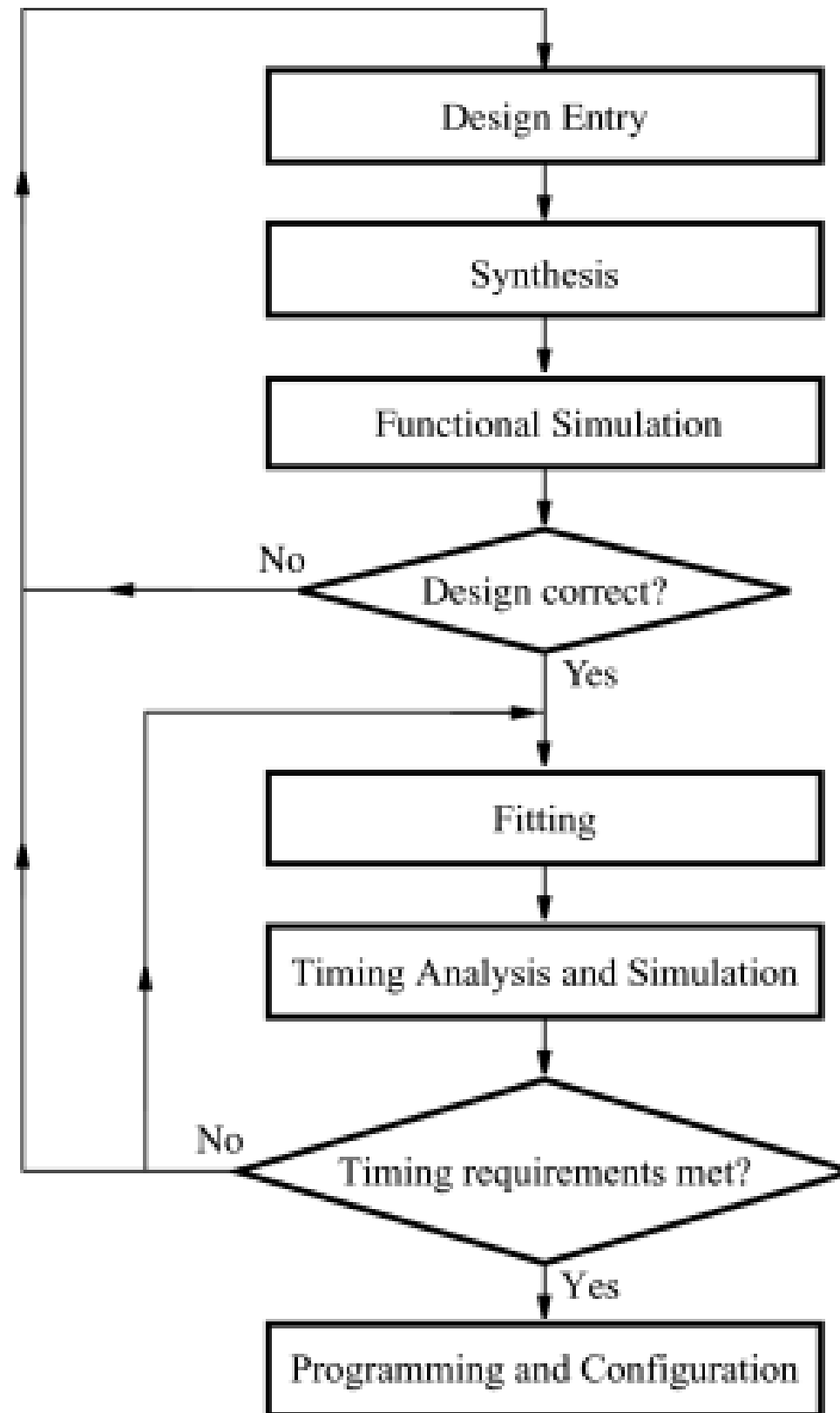
Altera's cyclone v soc

FPGA Design Flow

FPGA Design Flow



FPGA Design Flow



Tasks	Compilation
Task	
✓	▶ Compile Design
✓	> ▶ Analysis & Synthesis
✓	> ▶ Fitter (Place & Route)
✓	> ▶ Assembler (Generate programm
✓	> ▶ Timing Analysis
✓	> ▶ EDA Netlist Writer
	■ Edit Settings

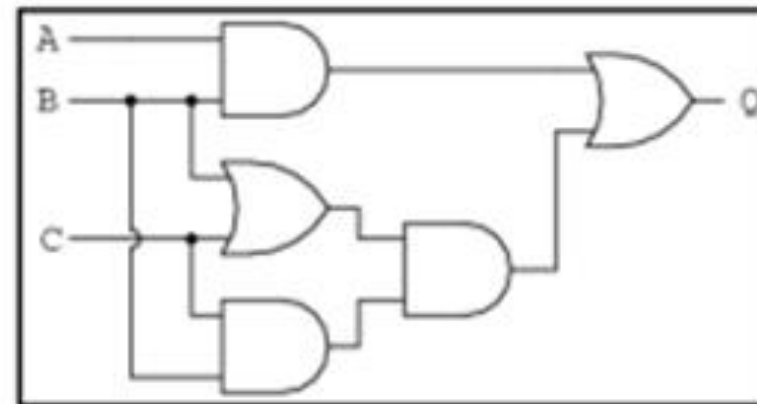
DESIGN FLOW [SYNTHESIS]

```
Residue = 16'h0000;  
If(high_bits==2'b10)  
    residue = state_table[i];  
Else  
    state_table[i] = 16'h0000;
```

HDL Source(RTL)

Translate

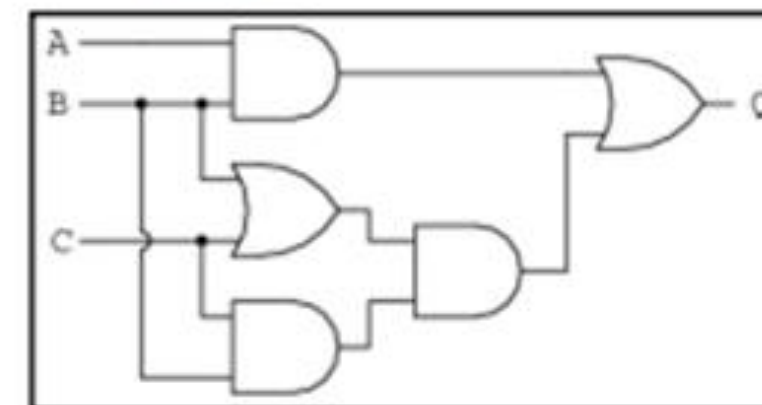
NO Timing Info =>



Generic Boolean

Optimize + Map
(Design Compiler)

Timing Info =>



Target Technology

DESIGN FLOW [SYNTHESIS]

- **Analyze / Compilation**

- The first step in synthesis is to read and analyze the HDL code and the other inputs to ensure nothing is missing or corrupted.
- The analyze step will check that:
 - HDL codes contain no syntax issues.
 - All modules/include files/functions/etc referenced inside the codes are available and nothing is missing.

- **Elaboration / Translate**

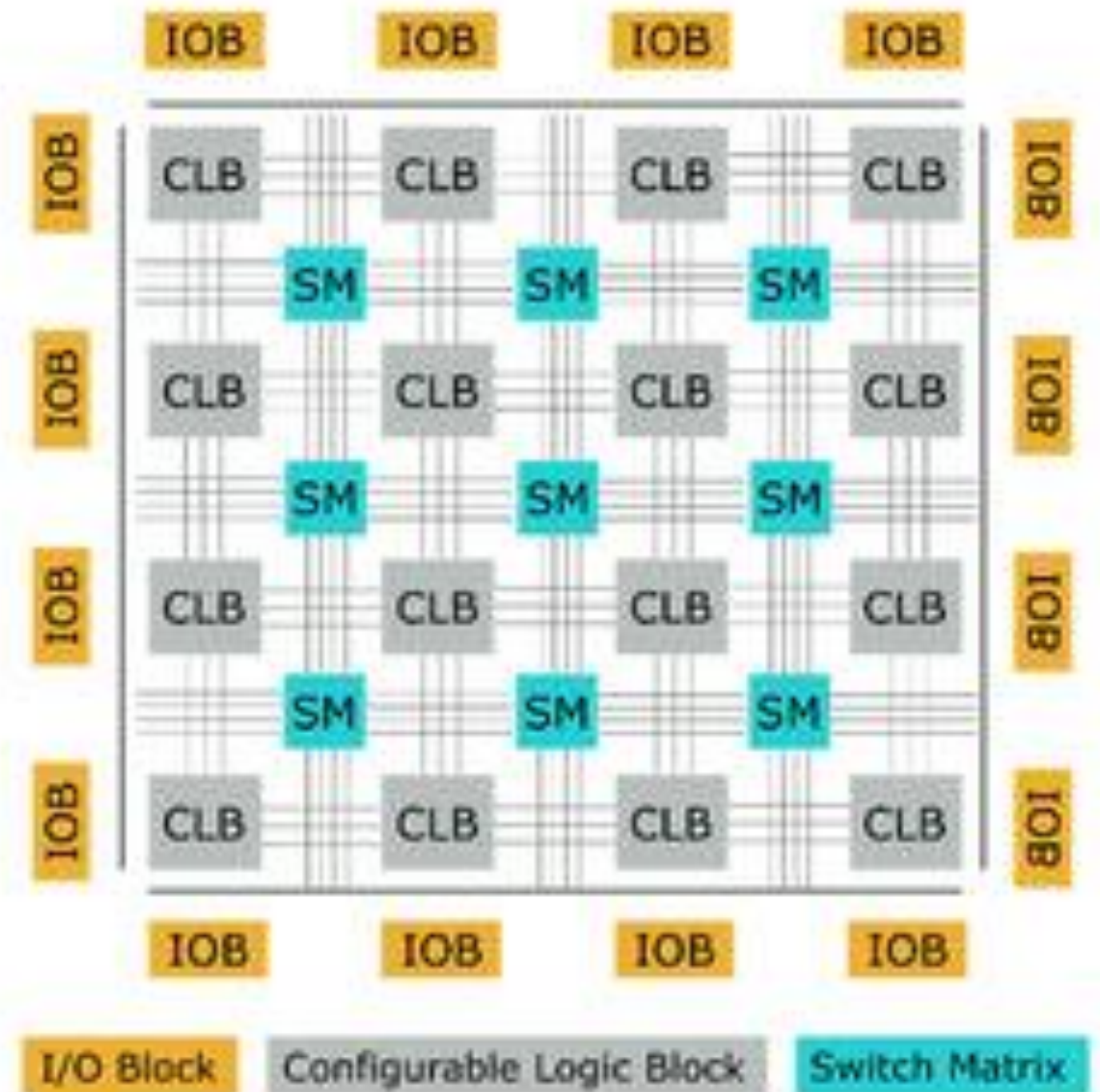
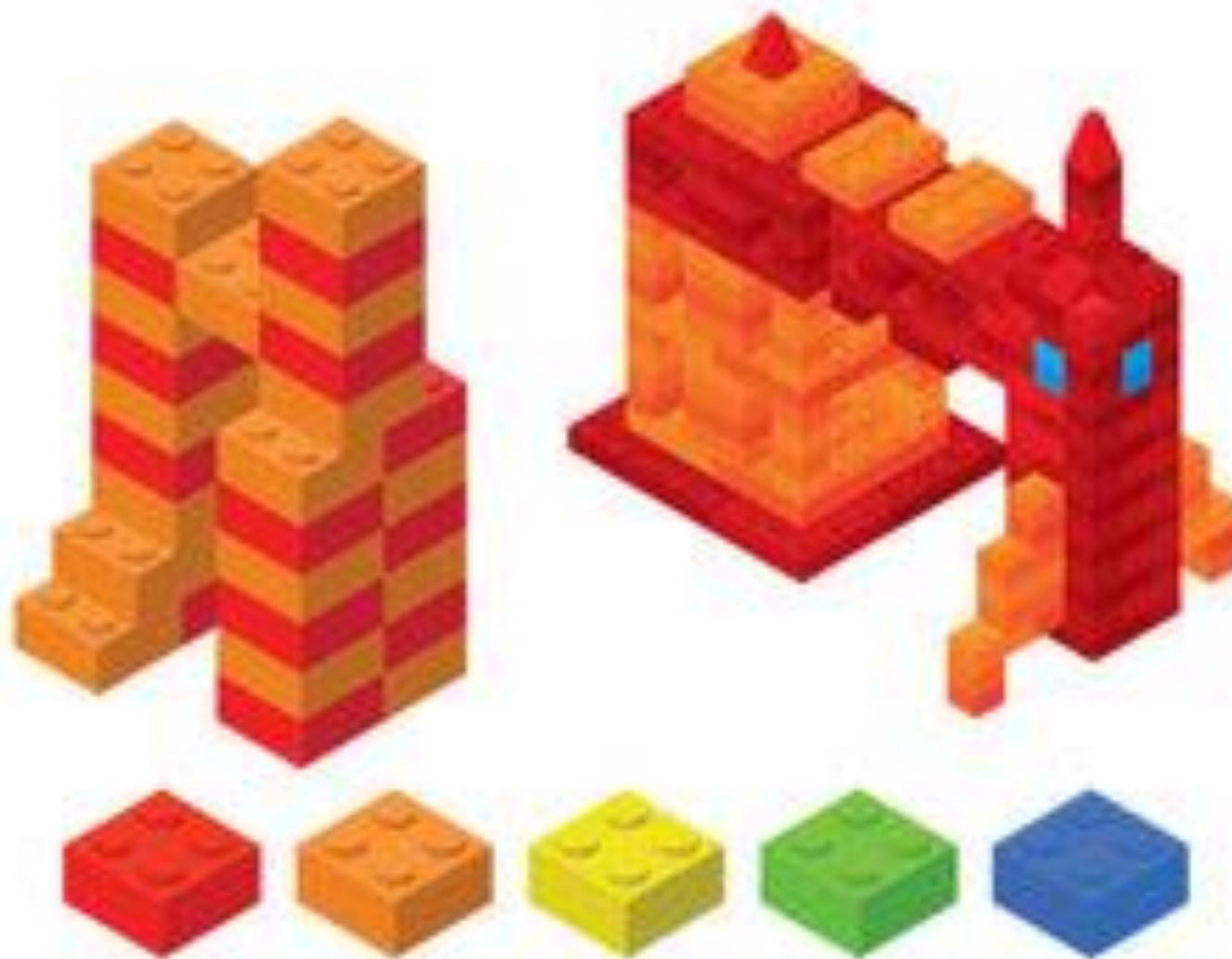
- It resolves all module instances, parameters, and generates a complete design hierarchy.
- The elaboration step outputs are:
 - A technology-independent (G-Tech) netlist without any optimizations done.
 - no timing, power, or physical information exist.

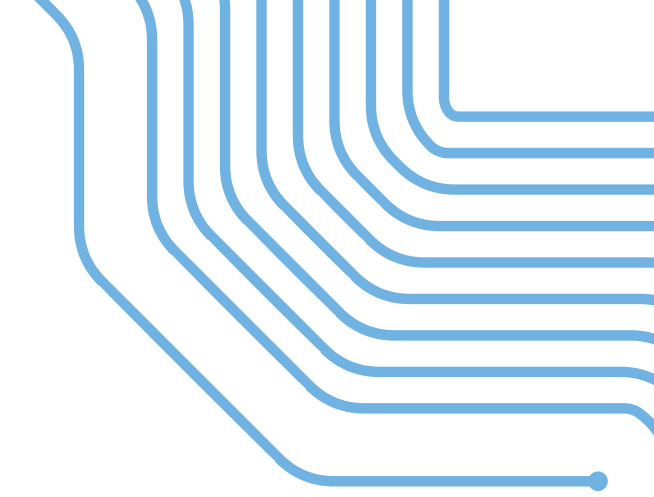
DESIGN FLOW[DEVICE PROGRAMMING]



Why FPGA?

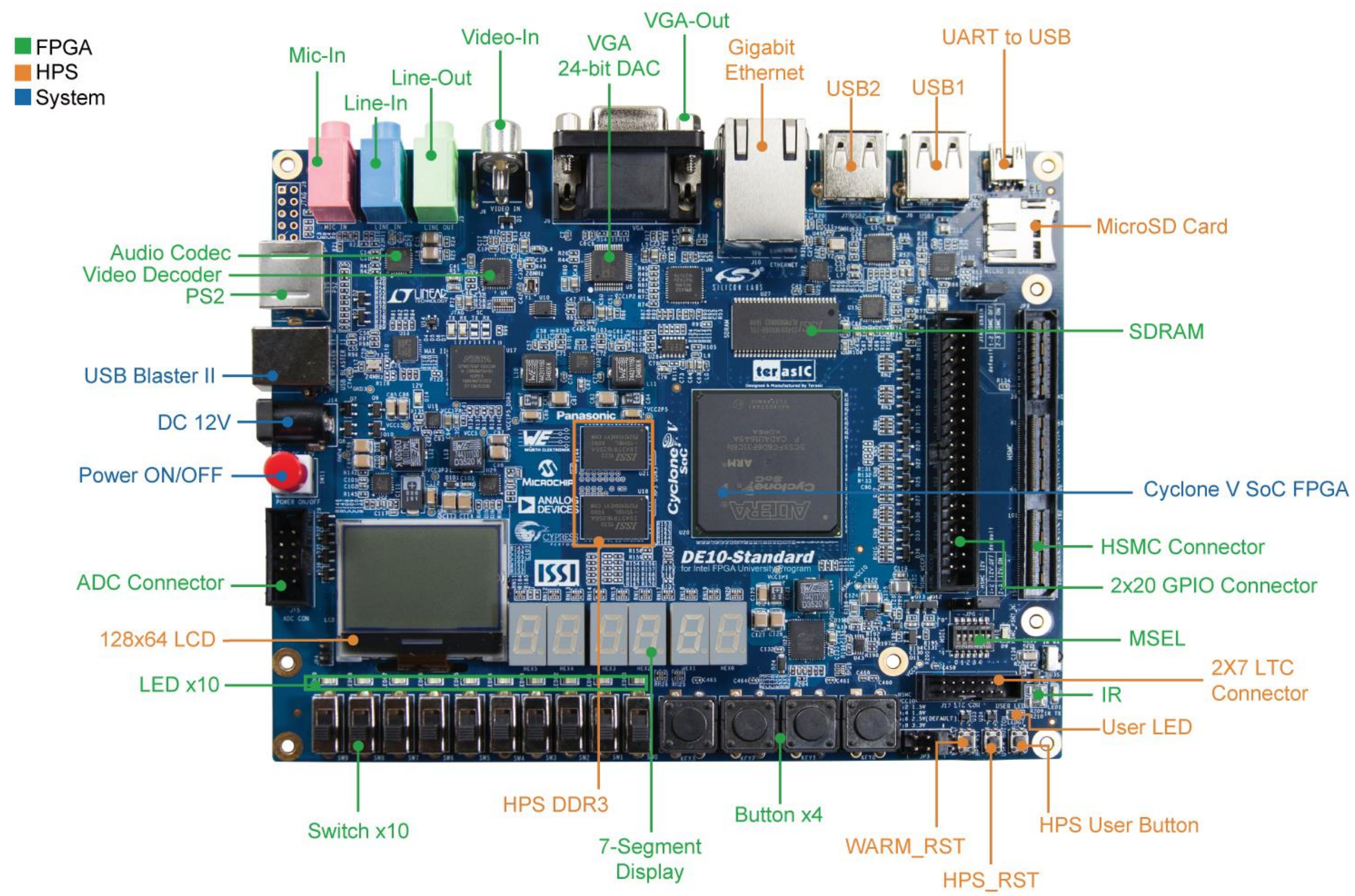
Think of **FPGAs** as **digital LEGOs**



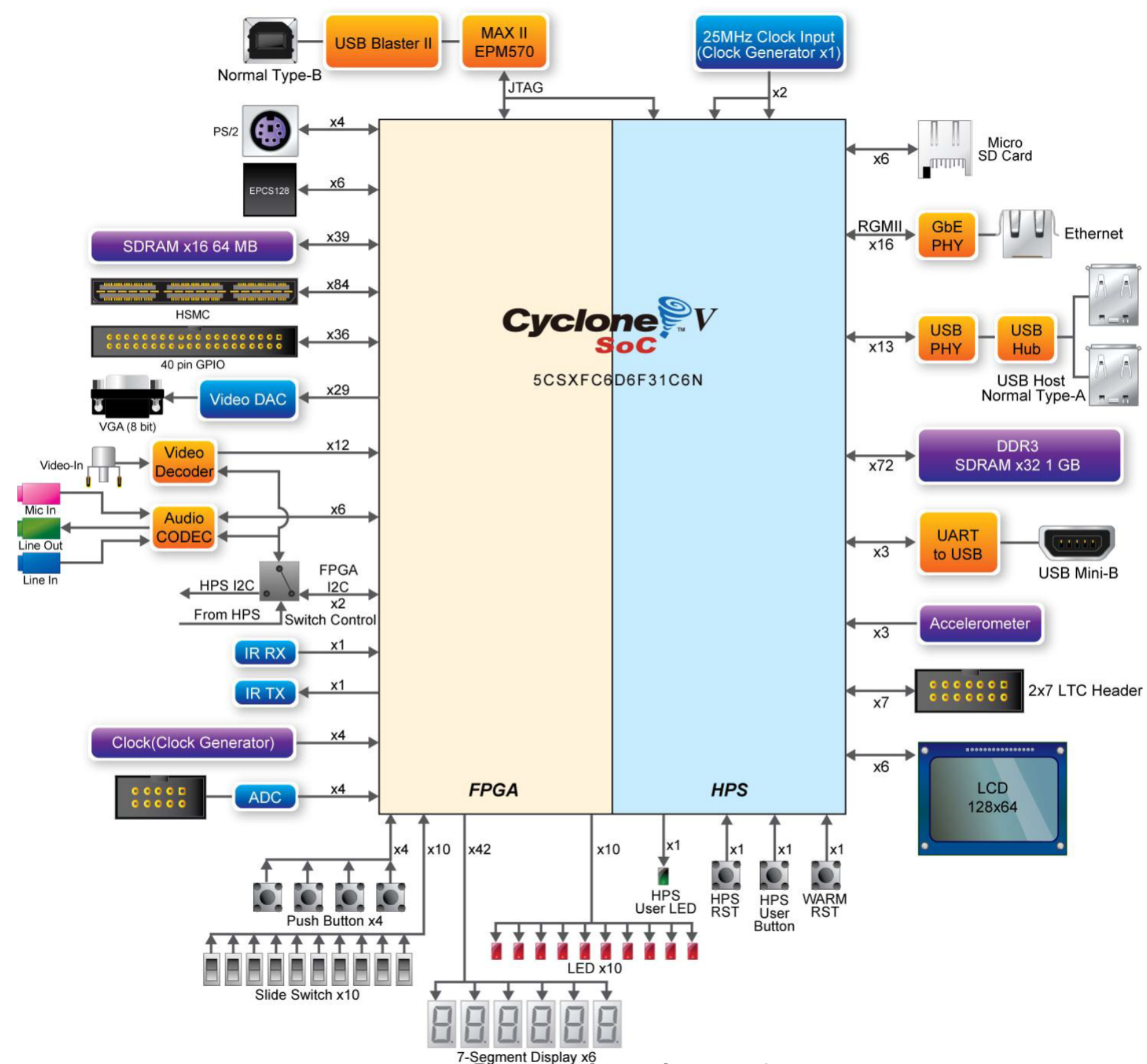


DE10-Standard development board

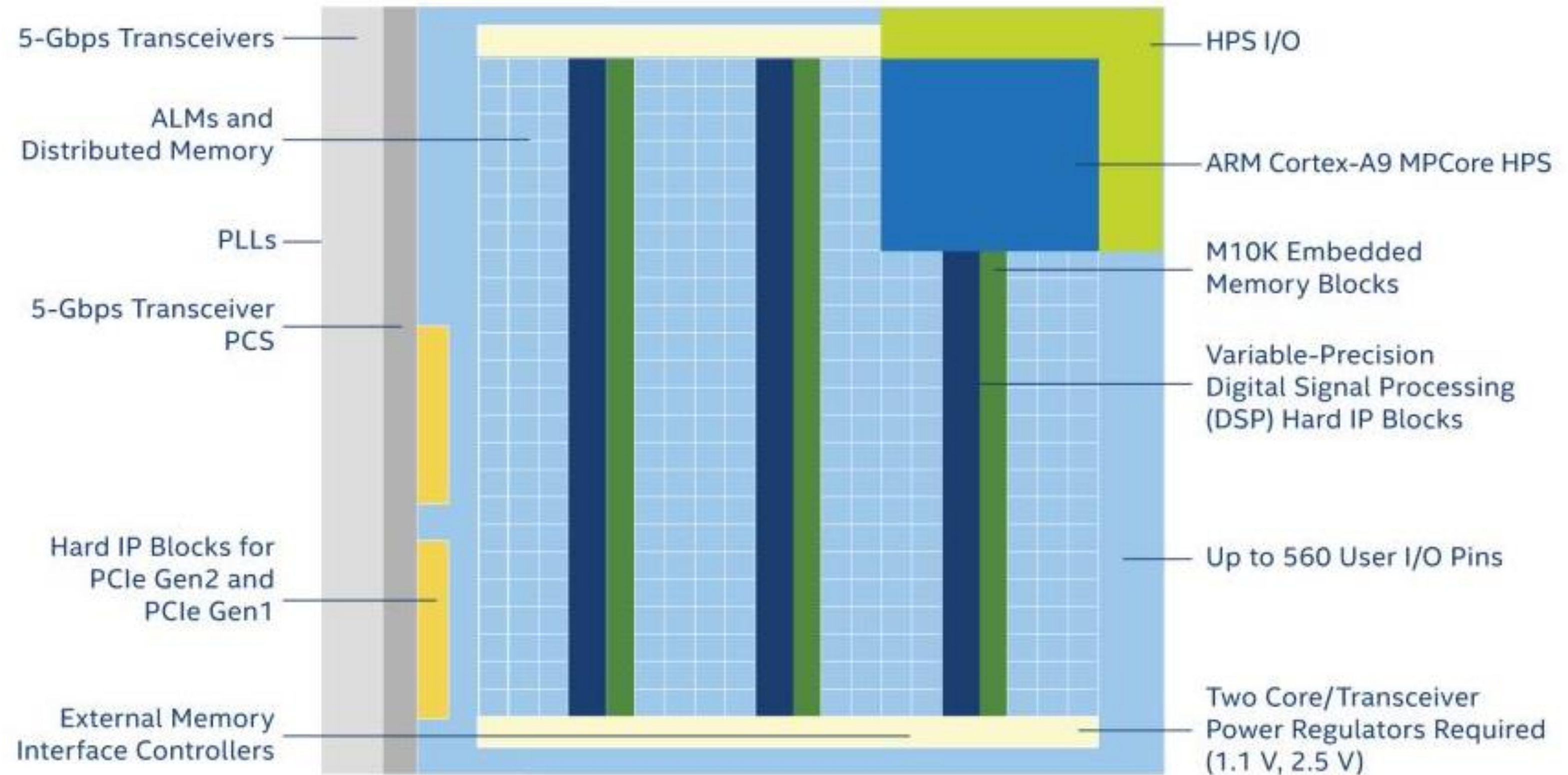
DE10-Standard development board (top view)



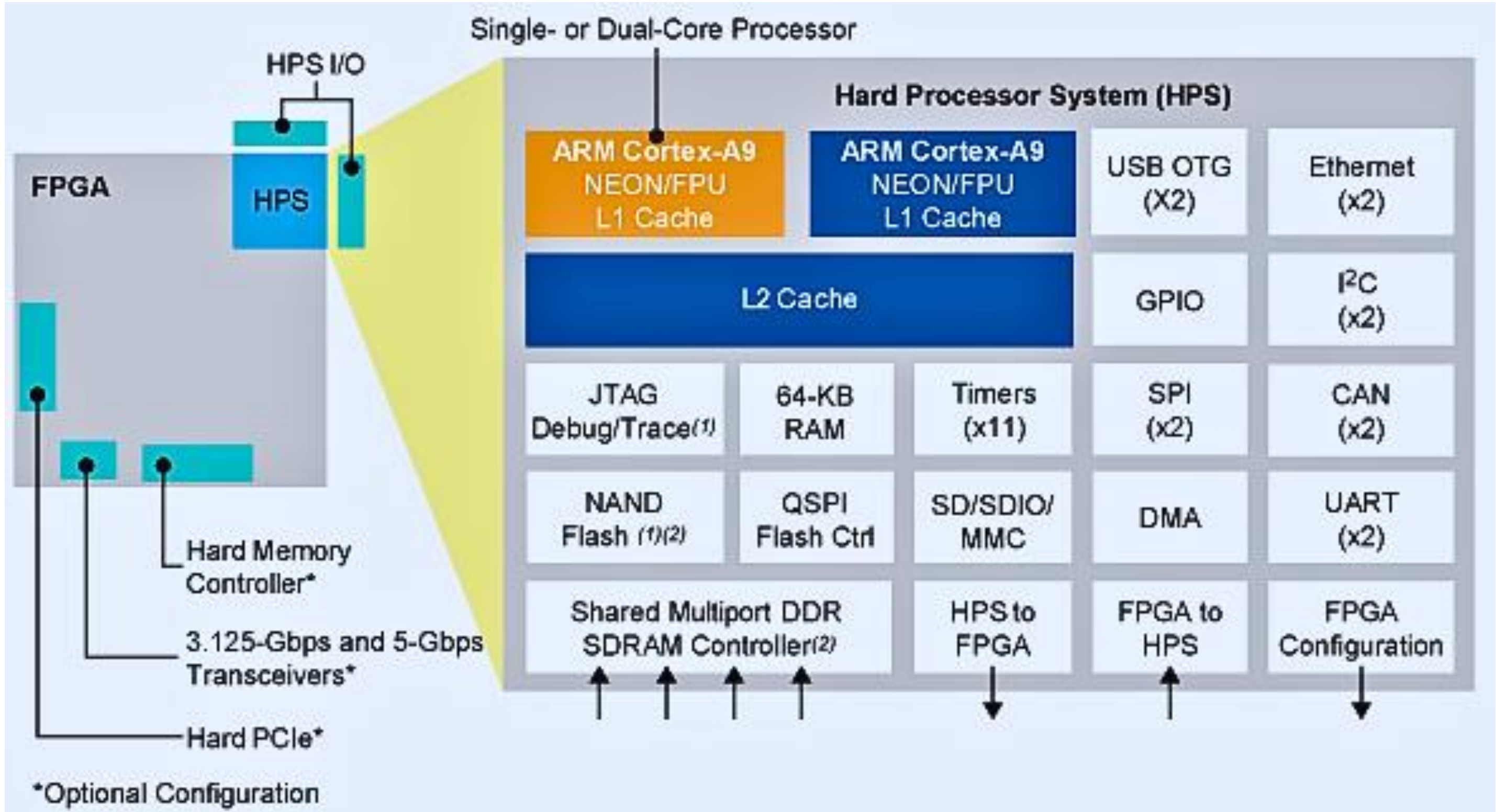
Block diagram of DE10-Standard



Altera Cyclone V architecture



Hard-Processor



Altera's cyclone v soc

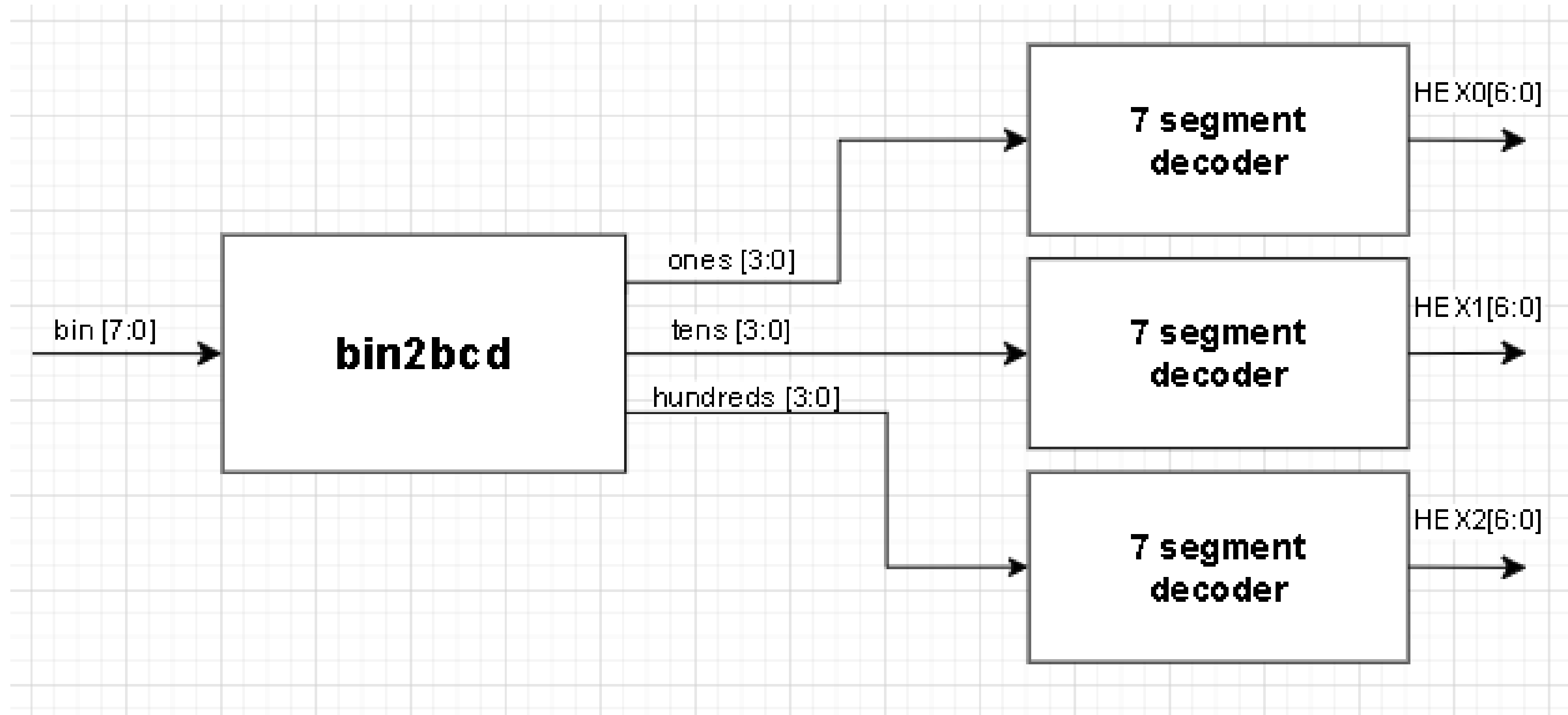
Altera Cyclone V resources table

Product Line		Cyclone V SE SoCs ¹				Cyclone V SX SoCs ¹				Cyclone V ST SoCs ¹	
		5CSEA2	5CSEA4	5CSEA5	5CSEA6	5CSXC2	5CSXC4	5CSXC5	5CSXC6	5CSTD5	5CSTD6
Resources	LEs (K)	25	40	85	110	25	40	85	110	85	110
	ALMs	9,430	15,880	32,070	41,910	9,430	15,880	32,070	41,910	32,070	41,910
	Registers	37,736	60,376	128,300	166,036	37,736	60,376	128,300	166,036	128,300	166,036
	M10K memory blocks	140	270	397	557	140	270	397	557	397	557
	M10K memory (Kb)	1,400	2,700	3,970	5,570	1,400	2,700	3,970	5,570	3,970	5,570
	MLAB memory (Kb)	138	231	480	621	138	231	480	621	480	621
	Variable-precision DSP blocks	36	84	87	112	36	84	87	112	87	112
	18 x 18 multipliers	72	168	174	224	72	168	174	224	174	224
Clocks, Maximum I/O Pins, and Architectural Features	Processor cores (Arm Cortex-A9)	Single or dual	Single or dual	Single or dual	Single or dual	Dual	Dual	Dual	Dual	Dual	Dual
	Maximum CPU clock frequency (MHz)	925	925	925	925	925	925	925	925	925	925
	Global clock networks	16	16	16	16	16	16	16	16	16	16
	PLLs ² (FPGA)	5	5	6	6	5	5	6	6	6	6
	PLLs (HPS)	3	3	3	3	3	3	3	3	3	3
	I/O voltage levels supported (V)	1.1, 1.2, 1.5, 1.8, 2.5, 3.3									
	I/O standards supported	LVTTTL, LVCMOS, PCI, PCI-X, LVDS, mini-LVDS, RSDS, LVPECL, SSTL-18 (I and II), SSTL-15 (I and II), SSTL-2 (I and II), HSTL-18 (I and II), HSTL-15 (I and II), HSTL-12 (I and II), Differential SSTL-18 (I and II), Differential SSTL-15 (I and II), Differential SSTL-2 (I and II), Differential HSTL-18 (I and II), Differential HSTL-15 (I and II), Differential HSTL-12 (I and II), Differential HSUL-12, HiSpi, SLVS, Sub-LVDS									
	Maximum LVDS pairs (receiver/transmitter)	37/32	37/32	72/72	72/72	37/32	37/32	72/72	72/72	72/72	72/72
	Transceiver count (3.125 Gbps)	–	–	–	–	6	6	9	9	–	–
	Transceiver count (6.144 Gbps)	–	–	–	–	–	–	–	–	9 ³	9 ³
	PCI Express hardened IP blocks (1.0)	–	–	–	–	2	2	2 ⁴	2 ⁴	–	–
	PCI Express hardened IP blocks (2.0)	–	–	–	–	–	–	–	–	2	2
	GPIOs (FPGA)	145	145	288	288	145	145	288	288	288	288
	GPIOs (HPS)	181	181	181	181	181	181	181	181	181	181
	Hard memory controllers ⁵ (FPGA)	1	1	1	1	1	1	1	1	1	1
Hard memory controllers ⁵ (HPS)	1	1	1	1	1	1	1	1	1	1	
Memory devices supported	DDR3, DDR2, LPDDR2										
Package Options and I/O Pins: General-Purpose I/O (GPIO) Count, and Transceiver Count											
U484 pin (19 mm, 0.8 mm pitch)		66, 151 0	66, 151 0	66, 151 0	66, 151 0						
U672 pin (23 mm, 0.8 mm pitch)		145, 181 0	145, 181 0	145, 181 0	145, 181 0	145, 181 6	145, 181 6	145, 181 6	145, 181 6		
F896 pin (31 mm, 1.0 mm pitch)				288, 181 0	288, 181 0			288, 181 9	288, 181 9	288, 181 9	288, 181 9

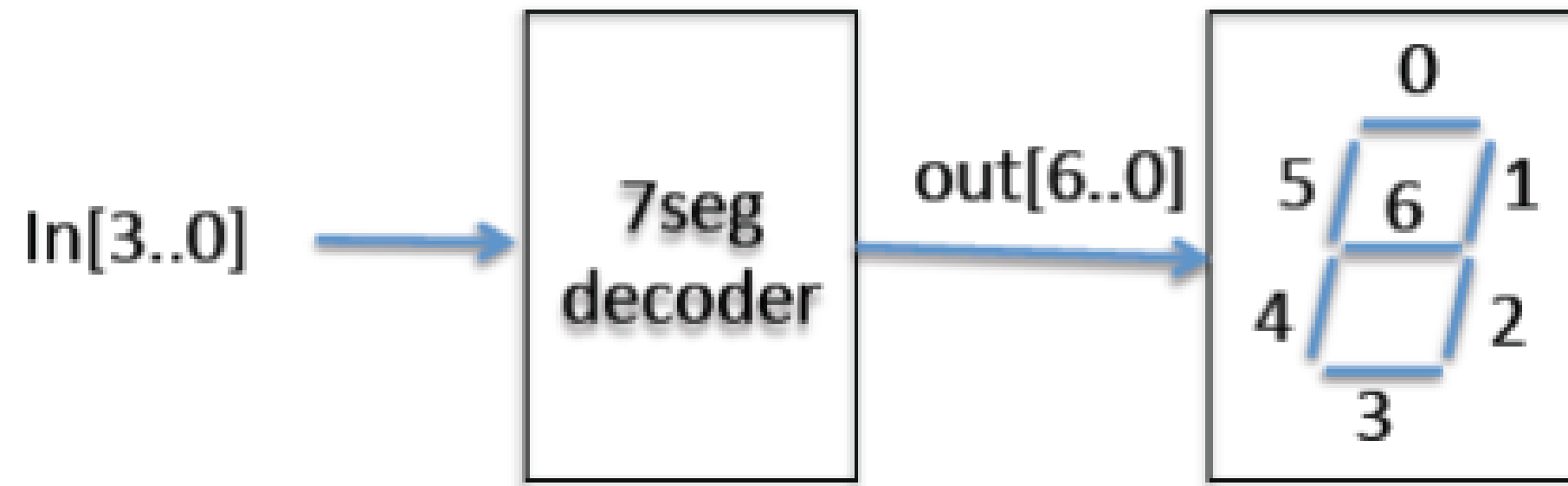
The labs

- 1. Try to turn some led s**
 - Try switches and keys**
- 2. My_first fpga lab.**
- 3. bcd to 7 segment decoder.**
- 4. A one-second counter + 7 segment decoder.**
- 5. Verilog VGA driver for the DE10-standard Fpga.**

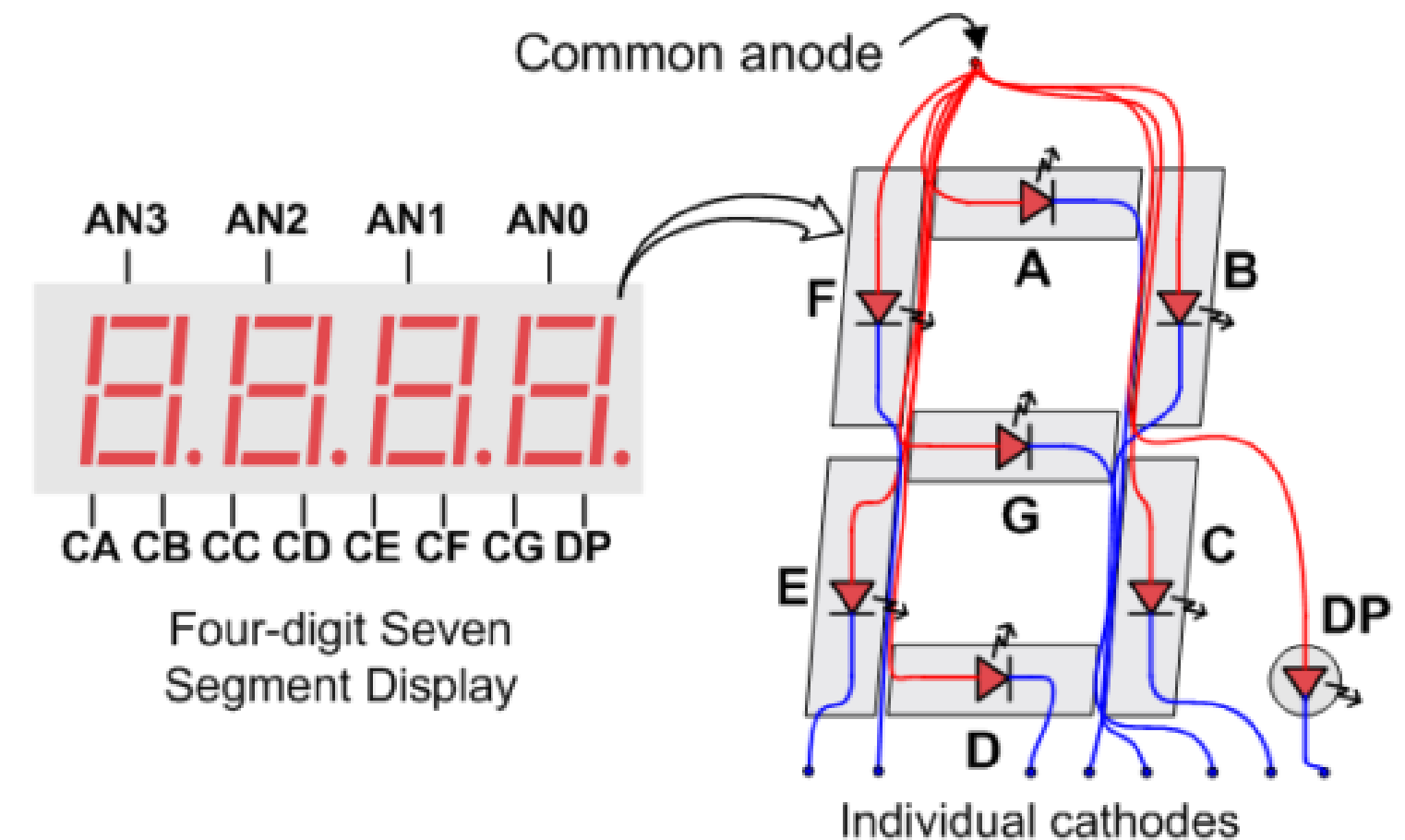
1. bcd to 7 segment decoder lab.



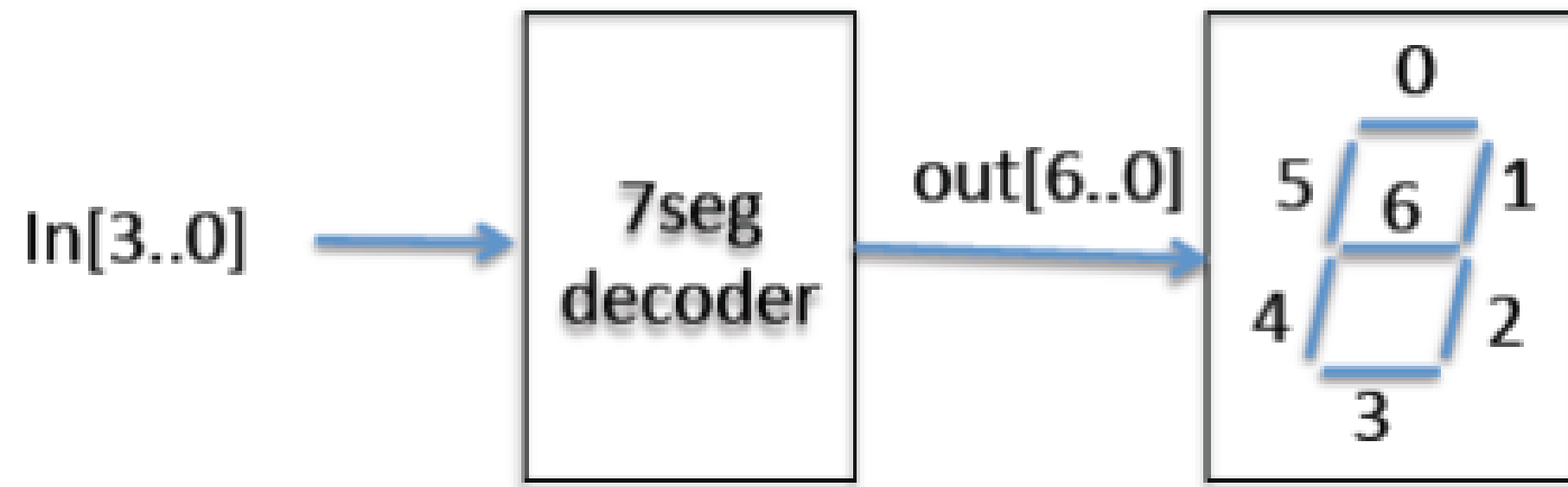
1. bcd to 7 segment decoder lab.



in[3..0]	out[6:0]	Digit	in[3..0]	out[6:0]	Digit
0000	1000000	0	1000	0000000	8
0001	1111001	1	1001	0010000	9
0010	0100100	2	1010	0001000	A
0011	0110000	3	1011	0000011	b
0100	0011001	4	1100	1000110	C
0101	0010010	5	1101	0100001	d
0110	0000010	6	1110	0000110	E
0111	1111000	7	1111	0001110	F



1. bcd to 7 segment decoder lab.



```

module hex_to_7seg (out,in);

    output [6:0] out; // low-active out
    input [3:0] in; // 4-bit binary input

    reg [6:0] out; // make out a variable

    always @ (*)
        case (in)
            4'h0: out = 7'b1000000;
            4'h1: out = 7'b1111001;
            4'h2: out = 7'b0100100;
            4'h3: out = 7'b0110000;
            4'h4: out = 7'b0011001;
            4'h5: out = 7'b0010010;
            4'h6: out = 7'b0000010;
            4'h7: out = 7'b1111000;
            4'h8: out = 7'b0000000;
            4'h9: out = 7'b0011000;
            4'ha: out = 7'b0001000;
            4'hb: out = 7'b0000011;
            4'hc: out = 7'b1000110;
            4'hd: out = 7'b0100001;
            4'he: out = 7'b0000110;
            4'hf: out = 7'b0001110;
        endcase
    endmodule
    
```

```

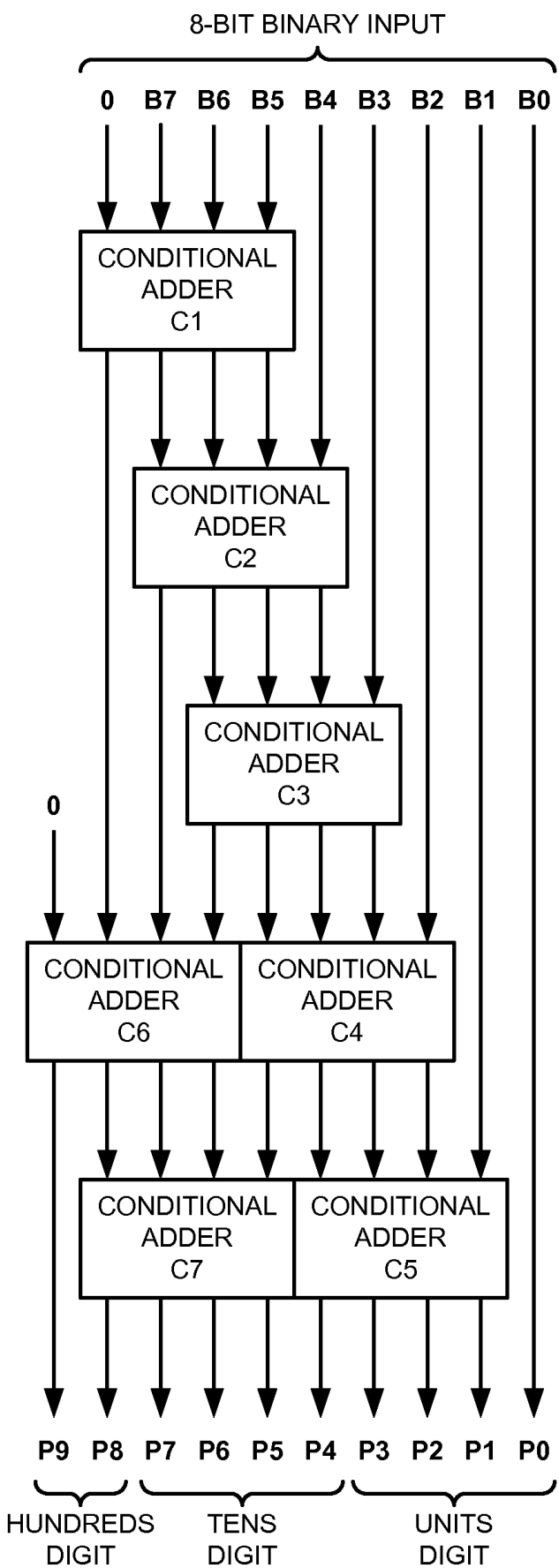
// --0 --
// |   |
// 5   1
// |   |
// --6 --
// |   |
// 4   2
// --3 --
    
```

in[3..0]	out[6:0]	Digit	in[3..0]	out[6:0]	Digit
0000	1000000	0	1000	0000000	8
0001	1111001	1	1001	0010000	9
0010	0100100	2	1010	0001000	A
0011	0110000	3	1011	0000011	b
0100	0011001	4	1100	1000110	C
0101	0010010	5	1101	0100001	d
0110	0000010	6	1110	0000110	E
0111	1111000	7	1111	0001110	F

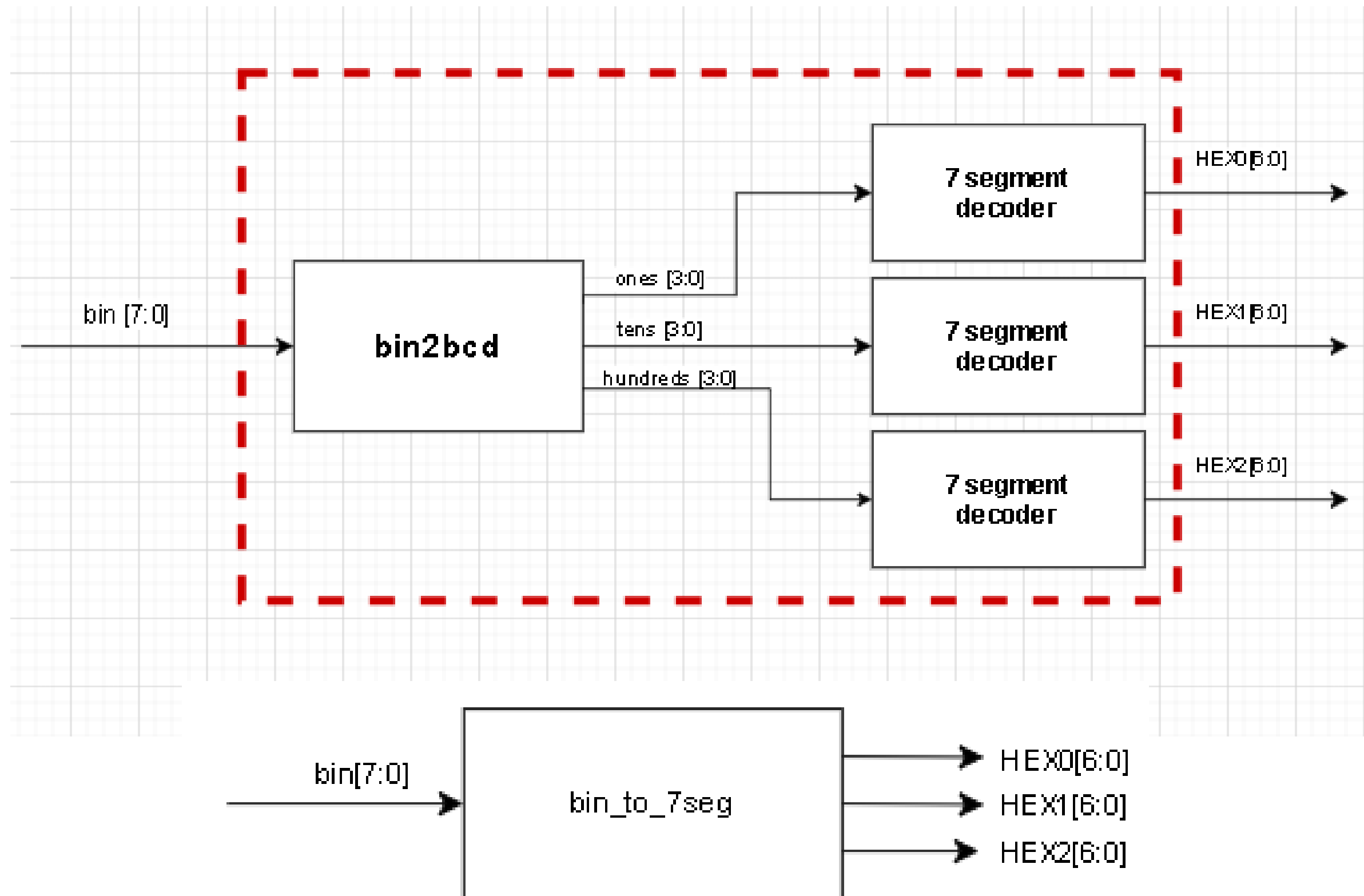
bcd to 7 segment decoder lab.

BCD2	BCD1	BCD0	Binary Input	
0000	0000	0000	10010111	
0000	0000	0001	0010111.	← ①
0000	0000	0010	010111..	← ②
0000	0000	0100	10111...	← ③
0000	0000	1001	0111....	← ④
0000	0000	1100	0111....	ADD
0000	0001	1000	111.....	← ⑤
0000	0001	1011	111.....	ADD
0000	0011	0111	11.....	← ⑥
0000	0011	1010	11.....	ADD
0000	0111	0101	1.....	← ⑦
0000	1010	1000	1.....	ADD ADD
0001	0101	0001		← ⑧

The Double Dabble Algorithm



2. digital clock on 7 segment



2. digital clock on 7 segment

